

Ж.-П. ЛАМУАТЬЕ

УПРАЖНЕНИЯ ПО ПРОГРАММИРОВАНИЮ НА ФОРТРАНЕ IV





Exercices de programmation en Fortran IV,

par

Jean-Pierre LAMOITIER

Ingénieur Arts et Métiers

Ingénieur à la Compagnie Française PHILIPS

Ж.-П. ЛАМУАТЬЕ

УПРАЖНЕНИЯ
ПО ПРОГРАММИРОВАНИЮ
НА ФОРТРАНЕ IV

Перевод с французского

В. А. Баяковской

Под редакцией

Ю. М. Баяковского

ИЗДАТЕЛЬСТВО «МИР»
МОСКВА 1978

Книга подчинена совершенно определенной цели: ускорить практическое освоение Фортрана, который в настоящее время чрезвычайно широко применяется при решении научных и инженерных задач. В ней рассматривается более 30 различных по сложности задач численного, нечисленного и статистического анализа, а также задач из области финансовых расчетов. Тщательно прослеживаются все фазы решения каждой задачи: постановка, составление алгоритма и блок-схемы, программирование на Фортране, анализ результатов после выполнения программы.

Книга будет ценным пособием для всех, кто изучает Фортран (в вузе или самостоятельно) с целью его практического применения.

Редакция литературы по математическим наукам

ОТ РЕДАКТОРА ПЕРЕВОДА

Около года назад почти одновременно вышли три книги, посвященные языку Фортран: Грунд Ф., Программирование на языке Фортран IV, М., «Мир», 1976; Карпов В. Я., Алгоритмический язык Фортран, М., «Наука», 1976; Салтыков А. И., Макаренко Г. И., Программирование на языке Фортран, М., «Наука», 1976. Почти 200-тысячный общий тираж этих книг, по-видимому, не удовлетворил полностью потребность в литературе по Фортрану. Уже один этот факт свидетельствует о том, что Фортран еще надолго останется основным языком программирования инженерных и научных вычислений, а также о том, что однобокая ориентация на Алгол в начале 60-х годов нанесла ощутимый ущерб как прикладному, так и системному программированию в нашей стране.

Широкое распространение Фортрана заставило искать новые пути рационального и быстрого обучения этому языку специалистов, различающихся как по уровню подготовки, так и по запросам. В этих условиях не удастся ограничиться основным учебником, содержащим описание грамматики языка и немногочисленные и, как правило, искусственные примеры.

Книга Ж.-П. Ламуатье не учебник, а только дополнение к учебнику. Это и не задачник. Если обратиться к аналогии, ее можно назвать «книгой для чтения» на Фортране: при изучении иностранных языков без такого рода литературы просто нельзя обойтись. При изучении же языка программирования отсутствие подобной литературы пытаются компенсировать листингами каких-либо случайных программ, далеко не всегда совершенных с методической и стилистической точек зрения.

В книге более 30 задач-миниатюр. Часть из них представляет непосредственный практический интерес, другие являются чисто учебными и содержат развлекательный элемент.

В качестве основного учебника автор рекомендует свою книгу о Фортране IV, вышедшую во Франции, но не переведенную на русский язык. Однако любой из трех учебников, упомянутых в начале предисловия, может служить вполне эквивалентной заменой.

Как известно, для языка Фортран характерно наличие многих диалектов, порой ничтожно отличающихся друг от друга. В тексте автор дает отдельные указания на особенности того диалекта, которым пользовался он сам. И все-таки тем из читателей, кто пожелает вслед за автором провести экспериментальную проверку программ, содержащихся в книге, следует внимательно просмотреть руководство по Фортрану для своей машины.

Мне хотелось бы выразить признательность автору книги и французскому издательству «Дюно», проявившим максимум внимания и доброжелательности к переводу книги и предоставившим в наше распоряжение второе ее издание сразу же после его выхода во Франции (первое издание вышло в 1974 г.).

Ю. Баяковский

Декабрь 1977 г.

ПРЕДИСЛОВИЕ

Нельзя изучать программирование без повседневной практики, а практика возможна только при наличии вычислительной машины.

Эта книга, задуманная как необходимое дополнение к основному учебнику¹⁾, позволит читателю проделать большое число упражнений и покажет ему, каким образом с помощью довольно коротких программ можно справиться с задачами, которые не удастся решить вручную.

Начинающие программисты обычно с предубеждением относятся к составлению блок-схем, поскольку видят в нем дополнительную и чаще всего бесполезную работу, предшествующую программированию. Однако опыт показывает, что составление блок-схемы позволяет точнее сформулировать алгоритм, еще не приступая к программированию, иногда дает возможность обнаружить некоторые ошибки анализа, облегчает последующее программирование и эксплуатацию программы.

Поэтому первая глава посвящена блок-схемам, и каждое упражнение содержит блок-схему, соответствующий листинг программы на Фортране и, как правило, пример ее выполнения.

Поскольку начинающий программист при поиске подходящего алгоритма часто испытывает затруднения, при выборе упражнений учитывалась их полезность как в плане программирования, так и в плане приложений. В связи с этим вторая глава посвящена целым числам, третья и четвертая — численному анализу, пятая — элементарной статистике, шестая — финансовым расчетам. Две последние главы содержат задачи из различных областей.

¹⁾ В нашем предисловии упоминалось несколько книг, которые можно использовать в качестве основного учебника. — *Прим. ред.*

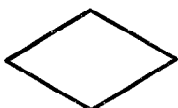
В некоторых упражнениях показано, как решаются простые задачи, с которыми можно встретиться в численном анализе: задача о влиянии ошибок округления, вычисление определенного интеграла, решение уравнений и т. д. Чтобы познакомить читателя с задачами нечисленного анализа, в заключительной главе приводятся несколько более сложных упражнений (задача о ханойской башне, задача о восьми ферзях).

Ж.-П. Ламуатье

Символы, используемые в блок-схемах



операция обработки



операция проверки



операция ввода-вывода



вызов подпрограммы



ссылка



начало и конец программы



подготовка

Символы, используемые в блок-схемах
для обозначения данных



перфокарта } чтение карт
 } перфорация карт



колода перфокарт



вывод на печать



магнитная лента



магнитный барабан



магнитный диск



слияние



разделение



сортировка



ручной ввод (обозначение терминала)

ГЛАВА 1. БЛОК-СХЕМЫ

1.0. Введение

Когда хотят написать доклад, диссертацию и тому подобное, то начинают с составления плана, в котором находят отражение основные идеи и порядок их изложения. В программировании тоже существует подобный предварительный этап, и связан он с составлением блок-схем.

Многие программисты пренебрегают этим очень полезным для более четкого понимания задачи этапом, не позволяющим оставить без внимания детали и особые случаи, которые не следует забывать.

Построение блок-схемы не представляет трудностей для профессионального программиста, но оно оказывается простым делом для начинающего. Поэтому мы начнем с двух простых и забавных примеров, в которых показано, как составляется блок-схема.

1.1. Задача об автомате и лабиринте

Пусть имеется упрощенный лабиринт только с одним входом и одним выходом и такой, что внутри него нет «кольцевых» коридоров.

Автомат, который не видит перед собой, может осуществить следующие элементарные действия:

- продвинуться на шаг вперед;
- повернуться на 90° (налево или направо);
- узнать, не натолкнется ли он при следующем шаге на стену;
- проверить, вышел ли он из лабиринта (уловив поток свежего воздуха).

Этот автомат в какой-то мере похож на слепого, который ощупывает тростью перед собой, но сходство неполное, поскольку наш автомат не может помнить ранее совершенные действия, у него нет памяти.

Задача

а) Какие правила должен применять наш автомат, чтобы пройти от входа лабиринта до выхода из него?

б) Эти правила необходимо записать кратко, но так, чтобы они допускали только одну интерпретацию.

Решение

а) Среди возможных решений следующий вариант является одним из самых простых:

- состояние 1) 1 Автомат продвигается на один шаг (в первый раз этот шаг необходим, чтобы войти в лабиринт) и переходит в состояние 2.
- состояние 2 Автомат осуществляет проверку: вышел ли он из лабиринта?
- если да, то задача закончена и автомат останавливается,
 - если нет, то он переходит в состояние 3.
- состояние 3 Автомат делает поворот направо и переходит в состояние 4.
- состояние 4 Автомат осуществляет проверку: не стена ли перед ним?
- если нет, то он переходит в состояние 1,
 - если да, то он переходит в состояние 5.
- состояние 5 Автомат делает поворот налево, затем переходит в состояние 4.

Начиная с рис. 1.1 попытаемся проследить, как наш автомат выйдет из лабиринта.

Он входит в лабиринт, делает поворот направо (состояние 3), обнаруживает перед собой стену (состояние 4), делает поворот налево (состояние 5), продвигается на один шаг (состояние 1) и т. д. до тех пор, пока не найдет проход, позволяющий ему войти в коридор В. Там он поворачивает направо (состояние 3) и продолжает свой путь по коридору В, пока не натолкнется на стену (рис. 1.2).

Оказавшись вблизи В₁, автомат обнаруживает стену справа (состояния 3, 4, 5), затем находит ее перед собой (состояние 4), поворачивается снова налево (состояние 5), вновь обнаруживает стену (состояние 4), совершает еще один поворот налево и теперь может снова продвигаться (ему придется возвращаться). Наконец, он находит проход, который ведет в коридор С, и процесс продолжается.

Наш метод представляется вполне удовлетворительным.

б) Теперь остается сформулировать этот метод в лаконичной форме. Для этого мы можем воспользоваться схемой, представленной на рис. 1.3.

¹⁾ Мы используем здесь слово «состояние», поскольку оно употребляется в теории автоматов. В этом примере нас устроило бы также слово «фаза».

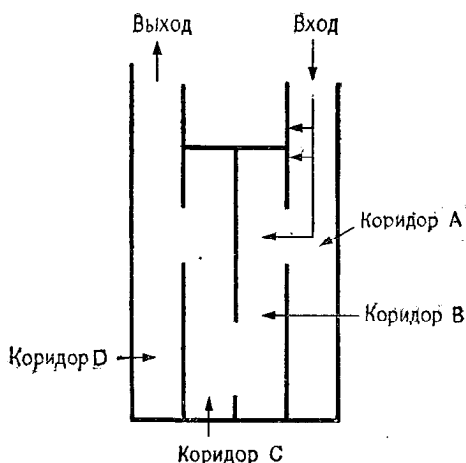


Рис. 1.1.

Пример лабиринта без кольцевых коридоров.

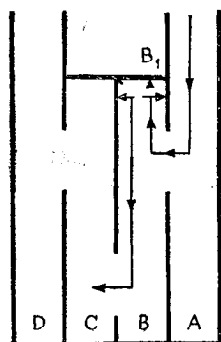


Рис. 1.2.

Часть пути, пройденного автоматом.

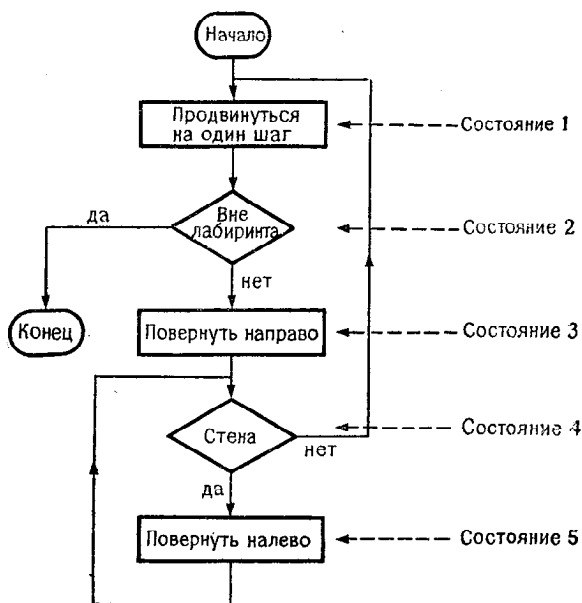


Рис. 1.3.

Эта схема показывает, как происходит переход из одного состояния в другое и как строится цепочка различных операций, выполняемых автоматом.

В информатике такие схемы, как на рис. 1.3, называются *блок-схемами*. Геометрические фигуры, из которых они построены, имеют обычно следующие значения:

- прямоугольник содержит операцию для выполнения (обработка);
- ромб содержит операцию проверки;
- стрелки, которые соединяют различные фигуры, указывают порядок перехода от одной фигуры к другой (или из одного состояния в другое).

В следующем более сложном примере мы вновь вернемся к составлению блок-схемы.

1.2. Задача о поездах и мухе

Пусть два города А и В удалены один от другого на расстояние $d = 600$ км. В то же самое время из каждого города отходит скорый поезд в направлении к другому городу. Поезд, отправившийся из города А, имеет скорость $v_1 = 40$ км/ч, скорость поезда, отправившегося из В, $v_2 = 60$ км/ч. Одновременно из пункта А вылетает исключительно быстрая муха со скоростью $V = 200$ км/ч и летит навстречу поезду, отправившемуся из пункта В. При встрече с этим поездом муха делает полуоборот и летит к поезду, идущему из пункта А. Когда она его встретит, она полетит в обратном направлении, и так продолжается до тех пор, пока поезда не встретятся.

Задание:

- 1) определить длину различных отрезков пути, которые пролетит муха,
- 2) определить общее расстояние, которое пролетит муха.

1.3. Предварительный анализ задачи

Очевидно, что можно легко ответить на второй вопрос (поезда встретятся через $600/(40 + 60) = 6$ ч, а муха пролетит за то же время $200 \times 6 = 1200$ км). Но оставим в стороне эту хитрость и будем применять элементарный метод, подсказываемый формулировкой задачи.

Самая простая идея состоит в том, чтобы производить действия следующим образом:

- вычислить длину первого отрезка пути, который пролетит муха;
- вычислить длину второго отрезка и прибавить к предыдущему результату;
- вычислить длину третьего отрезка, сложить с ранее полученной суммой и т. д.

Схематически этот метод представлен на рис. 1.4.

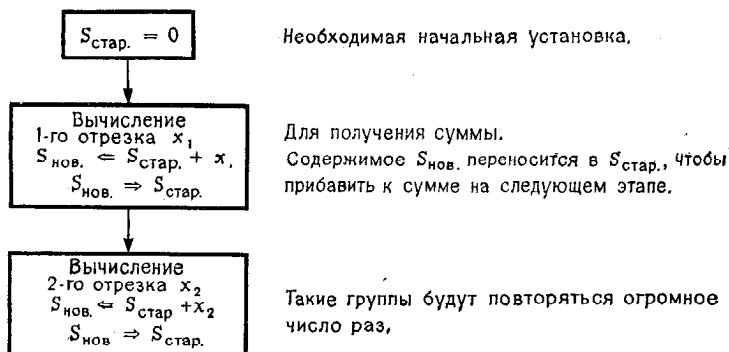


Рис. 1.4.

В связи с этой схемой возникают два замечания:

- если вычисление должно производиться с помощью вычислительной машины, то ей надо указать на то, какие величины должны быть напечатаны;
- так как схема должна иметь ограниченную длину, то средствами рисунка необходимо выразить понятие повторения.

Учет этих замечаний приводит к схеме на рис. 1.5.

Чтобы осуществить повторение последовательного вычисления, нужно ввести дополнительную переменную i , которая служит индексом и которая вначале устанавливается равной 1.

Вычислительные операции и вообще операции обработки заключаются в *прямоугольники*.

Операции *печати* и вообще операции *ввода-вывода* заключаются в *параллелограммы*.

С каждой фигурой на рис. 1.5 связаны с одной стороны дуга, указывающая, какое действие выполнялось непосредственно перед текущей операцией, и дуга, указывающая, какая операция должна выполняться, как только закончится текущая операция.

Случается, что после того, как операция закончена, необходимо вернуться к операции, которая была выполнена раньше. Это называется циклом. На рис. 1.5 цикл образует дуга,

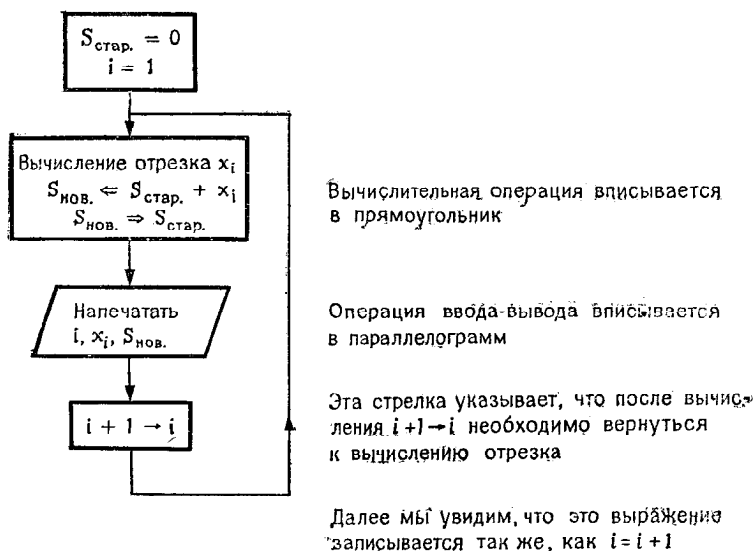


Рис. 1.5.

которая направлена от операции $i + 1 \rightarrow i$ к операции вычисления нового отрезка пути.

Замечание

Операция $i + 1 \rightarrow i$ обозначает:

- взять содержимое ячейки памяти с именем i ,
- прибавить 1,
- поместить результат в ячейку памяти с именем i .

Эта операция называется присваиванием и в блок-схеме ее обозначают по-разному:

$$i + 1 \rightarrow i,$$

$$i \leftarrow i + 1,$$

или еще

$$i = i + 1,$$

что должно быть истолковано следующим образом:

- вычислить выражение, расположенное справа от знака $=$,
- присвоить результат переменной, расположенной слева от знака $=$.

В этом случае знак $=$ является *символом присваивания*.
В более общем виде мы можем записать

$V = \text{выражение}$
 $\uparrow$ $\uparrow$
 переменная арифметическое выражение, которое может содержать эту же переменную

При проверке же знак $=$ интерпретируется как символ сравнения: вычисляются выражения, находящиеся соответственно слева и справа от знака $=$ и затем сравниваются полученные результаты. Сравнения встретятся нам в последующих блок-схемах (например, на рис. 1.6).

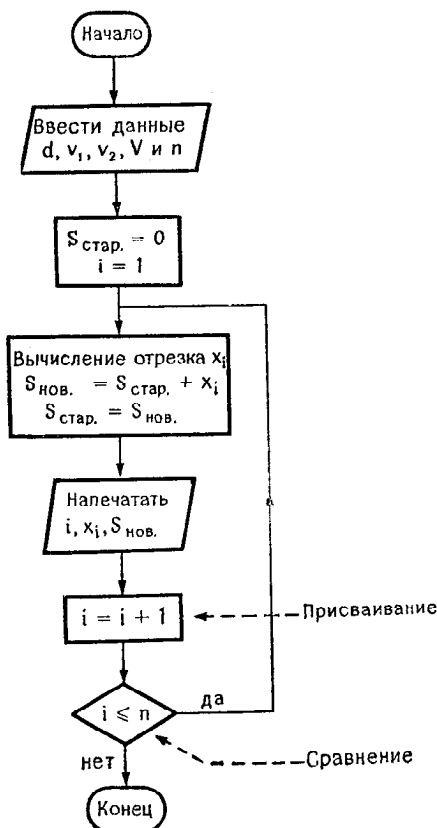


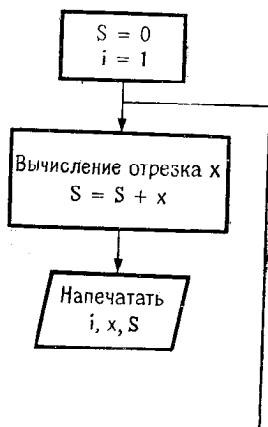
Рис. 1.6.

1.4. Первая блок-схема

В блок-схеме на рис. 1.5 легко обнаруживается весьма серьезное неудобство: вычислительный процесс никогда не заканчивается, поскольку цикл вычислений не может прекратиться. Это свидетельствует о недостаточном анализе задачи.

Очевидно, что различные отрезки пути, которые пролетает муха, будут становиться все меньше и меньше, а число отрезков с математической точки зрения бесконечно. (По сути дела, нам необходимо вычислить сумму ряда.) Начиная с некоторого момента отрезки станут настолько короткими, что можно будет прекратить «итерации». Останется только точно определить критерий окончания.

Начало такое же, как на рис. 1.6



Продолжение такое же, как на рис. 1.6

Рис. 1.6 bis.

Чтобы не слишком усложнять задачу, примем следующий критерий окончания: остановимся после n итераций, где n — некоторая дополнительно задаваемая величина.

Блок-схема теперь примет вид, показанный на рис. 1.6.

Таким образом, нам придется попеременно работать с $S_{\text{стар}}$ и $S_{\text{нов}}$. Величина $S_{\text{нов}}$, соответствующая отрезку x_i , становится $S_{\text{стар}}$ для следующего отрезка.

Впрочем, x_i и $S_{\text{нов}}$ нас более не интересуют, как только их значения напечатаны; следовательно, можно использовать одну переменную S вместо $S_{\text{нов}}$ и $S_{\text{стар}}$ и одну переменную x для всех отрезков x_i .

Эти замечания приводят к блок-схеме, указанной на рис. 1.6 bis.

После каждой итерации возникает вопрос: « $i \leq n$?». Этот вопрос заключен в ромб, имеющий в данном случае один вход и два выхода. Выбор выхода зависит от ответа на этот вопрос.

1.5. Подробный анализ отрезков пути

Чтобы пояснить, как вычисляется длина каждого отрезка x_i , прибегнем к графической интерпретации, приведенной на рис. 1.7.

На этом рисунке y_1 обозначает расстояние, разделяющее два поезда после того, как муха пролетит первый отрезок пути, а $t_1 = d/(V + v_2)$ и $y_1 = y_0 - t_1(V + v_2)$.

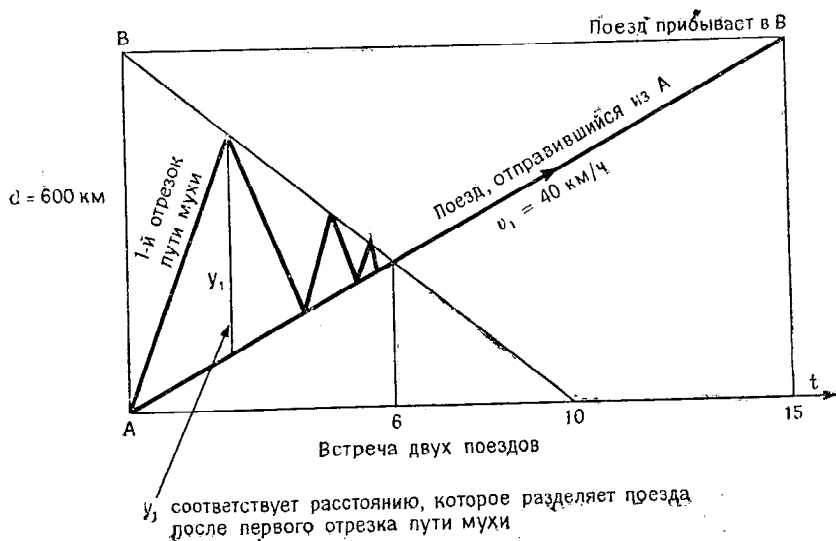


Рис. 1.7.

В общем случае пусть y — расстояние, которое в момент встречи с одним поездом отделяет муху от встречного поезда в самом начале нового отрезка пути. Тогда:

- если встречным поездом будет поезд В, то продолжительность полета равна

$$t = y/(V + v_2),$$

следовательно, муха пролетит расстояние

$$x = t \cdot V;$$

- если встречным поездом будет поезд А, то используются следующие формулы:

$$t = y/(V + v_1), \quad x = t \cdot V.$$

Вначале расстояние y равно d .

Когда муха встретит поезд после полета в течение t , ее будет отделять от другого поезда расстояние

$$y - t(v_1 + v_2).$$

Это новое значение будет использоваться при вычислении следующего отрезка. После всех уточнений мы можем построить еще одну промежуточную блок-схему, изображенную на рис. 1.8.

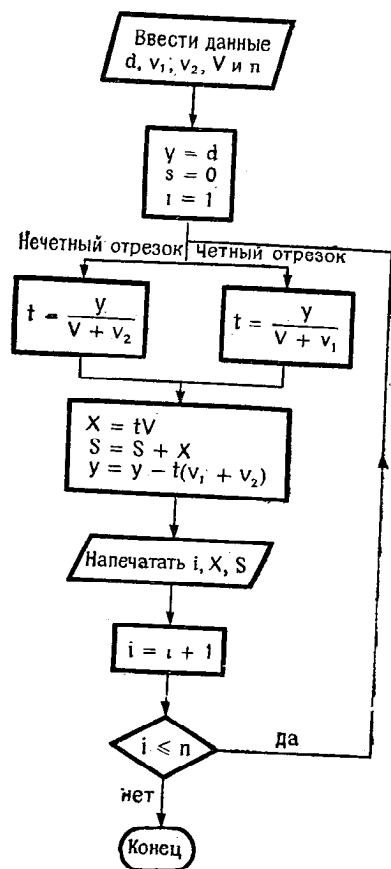


Рис. 1.8.

1.6. Окончательная блок-схема

В блок-схеме на рис. 1.8 остается еще один последний недостаток: как указать вычислительной машине, что надо проходить левую ветвь на нечетных отрезках пути (при встрече с поездом В) и правую ветвь в другом случае (при встрече с поездом А)?

Для этого мы воспользуемся вспомогательной переменной, значение которой указывает, какую ветвь следует проходить. Этот элементарный прием отражен на фрагментах блок-схем, приведенных на рис. 1.9. и 1.10.

На рис. 1.9 используется логическая (или булева) переменная, принимающая только 2 значения: истина и ложь.

На рис. 1.10 используется числовая переменная (следует предпочесть переменную целого типа). Программисты часто предпочитают последнее, менее элегантное с теоретической точки зрения решение, поскольку в этом случае сравнение можно осуществить как с помощью логического оператора IF, так и с помощью арифметического оператора IF.

Тем не менее основной принцип в обоих случаях один и тот же, в каждой из ветвей этой переменной присваивается такое значение, которое вынуждает в следующий раз пройти по другой ветви. Однако первоначально переменная должна получить соответствующее значение.

Кроме того, на схеме 1.10 есть два кружка, в которые вписана буква А. Символ $\rightarrow \textcircled{A}$ говорит о том, что далее следует направиться в точку, содержащую символ $\leftarrow \textcircled{A}$. Это условное обозначение позволяет сделать блок-схемы более наглядными, особенно в тех случаях, когда не хватает места.

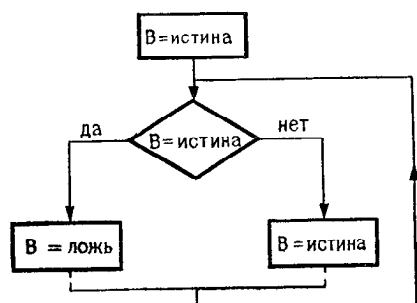


Рис. 1.9.

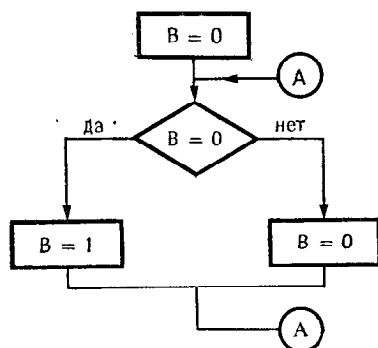


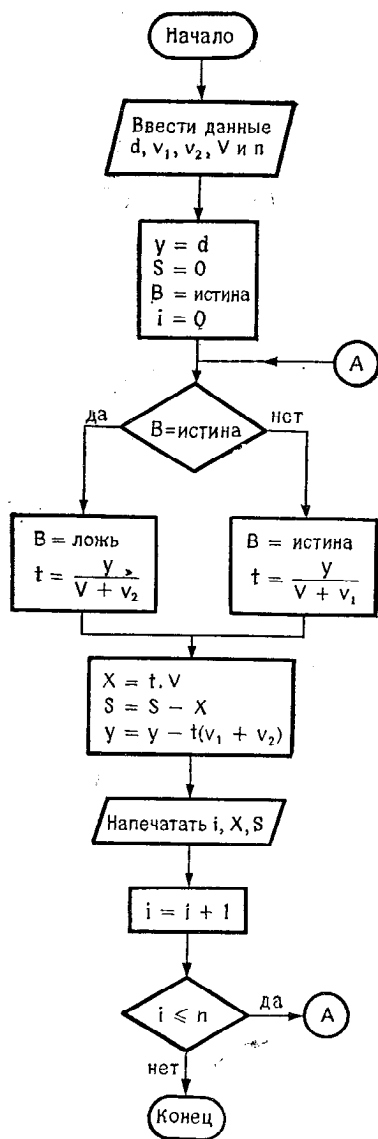
Рис. 1.10.

Предыдущие схемы позволяют нарисовать окончательную блок-схему. На рис. 1.11 и 1.12 представлены два примера, в которых используются соответственно логическая и числовая переменные. Образец программы на Фортране, согласующийся с блок-схемой на рис. 1.11, содержится в приложении к этой главе (см. стр. 27).

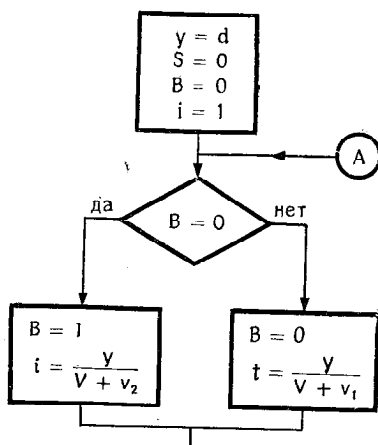
1.7. Методология решения научных задач с помощью вычислительной машины

Мы советуем при первом чтении разд. 1.7—1.9 не задерживаться надолго на трудных для понимания вопросах; к этим разделам следует вернуться еще раз, когда будет проделано несколько упражнений.

В простом примере, рассмотренном ранее, показано, как цепочка умозаключений приводит к окончательной блок-схеме. Решение более сложных задач распадается на несколько этапов.



Начало такое же, как на рис. 1.11



Продолжение такое же, как на рис. 1.11

Формулирование задачи: эта первая часть состоит в четком изложении условия задачи и составлении соответствующих уравнений. На этом этапе уточняются значения данных, при которых будут производиться вычисления.

Физический и математический анализ: этот анализ должен подтвердить существование и единственность решения и определить, какие теоретические методы будут использоваться при решении задачи.

Численный анализ: на этом этапе выбирают метод, который представляется наиболее приемлемым, разрабатывают его во всех деталях и затем записывают в виде алгоритма.

Анализ алгоритма: учитывая ограничения, налагаемые вычислительной машиной (время выполнения операций, объем оперативной памяти), на этом этапе уточняют алгоритм и рисуют принципиальную блок-схему.

Программирование и отладка: этот этап включает:

- составление детальных блок-схем;
- собственно программирование;
- синтаксическую отладку программы;
- логическое тестирование с помощью выполнения программы в некоторых частных случаях, позволяющих проверить ее правильность;
- подготовку технического описания программы и документации для пользователей.

Продолжительность этого этапа часто недооценивается. Техническое описание программы не редко вообще не делается, хотя оно необходимо в дальнейшем для эксплуатации программы.

На практике границы между этими различными этапами не всегда четко очерчены, особенно если речь идет о легких задачах. Кроме того, этапы не всегда следуют в нормальном порядке, и иногда требуются возвраты к предыдущему этапу.

Упражнения в следующих главах посвящены в основном анализу алгоритмов (который во всех рассмотренных примерах достаточно прост) и программированию. Кроме того, попутно высказываются краткие замечания, касающиеся тех задач численного анализа, с которыми можно встретиться на практике.

1.8. Советы по составлению блок-схемы

Даже профессионалу трудно с первого раза начертить правильную блок-схему, которая строится постепенно. Опыт показывает, что блок-схема обычно начинается с инициализаций

(начальных установок), которые полностью определяют только тогда, когда нарисовано «ядро блок-схемы». Поэтому следует начать такой рисунок с центральной части, оставив место для последовательности инициализации. Стоит ли удивляться тому, что при рисовании блок-схем карандаш и резинка оказываются предпочтительнее ручки!

Как показано в предыдущем примере, блок-схема строится в несколько этапов, а именно вначале рисуется принципиальная блок-схема и уже потом подробная блок-схема.

В процессе этого построения желательно посмотреть, в какой степени подпрограммы могут упростить подробные блок-схемы, а затем и программирование.

Этап составления блок-схемы непосредственно предшествует программированию. А при написании программ можно, как правило, использовать лишь латинские прописные буквы; в вычислительных машинах обычно не допускаются строчные буквы и греческий алфавит. Чтобы облегчить переход от блок-схемы к программе и обратно, желательно использовать в блок-схеме только те литеры, которые допустимы в программе.

Когда программа слишком длинная, блок-схему не удастся разместить на одной странице, и ее приходится делить на части, применяя для этой цели ссылки. Чтобы читателю не приходилось путаться в деталях очень большой блок-схемы, полезно добавить более лаконичную принципиальную блок-схему, в которой лучше просматривается основная идея выбранного метода.

В процессе составления блок-схемы вводятся переменные, смысл которых может быть забыт. Желательно, как только блок-схема станет довольно длинной, обновить таблицу используемых переменных. В этой таблице, кроме имени, следует указать физический смысл переменной, если таковой имеется, и ее назначение. Полезно также уточнить способ установки начального значения этой переменной:

- операцией ввода данных;
- посредством инструкции DATA;
- с помощью инструкции присваивания.

Тогда в программе не будет переменных с неопределенным начальным значением; кроме того, такой анализ позволит обнаружить некоторые упущения, сделанные во время составления блок-схемы.

Наконец, эта таблица будет очень полезна в дальнейшем при эксплуатации и развитии программ.

Очевидно, при решении простой задачи можно обойтись без промежуточных этапов и тем более без таблицы переменных.

1.9. Советы по конструированию программы

Издержки, связанные с отладкой программы, часто недооцениваются в том, что касается затрат человеческого времени, расхода машинного времени и общих задержек по отношению к намеченным срокам. Когда программа становится длинной или сложной (большое наслоение проверок, медленная сходимости и т. д.), необходимо действовать систематически и попытаться уменьшить стоимость отладки. Теперь мы дадим несколько общих рекомендаций, направленных на снижение стоимости отладки и последующей эксплуатации программы¹⁾.

Необходимо обеспечить полное соответствие между программой и блок-схемой, для этого:

- постарайтесь программировать, скрупулезно следуя блок-схеме;
 - если во время программирования вы заметите ошибку в блок-схеме или даже найдете, как можно ее упростить, то исправьте блок-схему и только тогда продолжайте программирование;
 - используйте одни и те же имена переменных в блок-схеме и в программе;
 - сохраняйте блок-схему чистой и не работайте с черновиком;
- внесите в блок-схему основные метки программы с тем, чтобы облегчить ее последующую эксплуатацию.

Программа должна быть наглядной и удобочитаемой, для этого:

- постарайтесь, чтобы блок-схема нашла свое отражение в структуре программы. С этой целью
 - соответствующим образом располагайте инструкции, сдвигая, например, вправо все инструкции, относящиеся к одному ДО-циклу;
 - поместите комментарий перед каждой важной частью программы;
 - обратите внимание на начальные установки, которые часто программируются неполностью;

¹⁾ После того как закончена отладка и исправлены все найденные ошибки, начинается эксплуатация программы. Однако в процессе эксплуатации будут, по-видимому, обнаруживаться ошибки и их придется исправлять. Кроме того, постепенно изменяется и усложняется сама задача (как и всякая техника) и поэтому программу нужно модернизировать. Эта работа называется поддержанием (эксплуатацией) программы.

- разместите метки в возрастающем порядке от начала к концу программы. Рекомендуется выбирать метки в возрастающем порядке с шагом 10 или 20. Это позволит в дальнейшем вставить новые метки, не меняя принятого порядка;
- сгруппируйте форматы в начале или в конце программы;
- используйте там, где это возможно, инструкцию DO, а не три эквивалентные ей инструкции;
- всякий раз, когда в программе применяется трюк с целью экономии при ее выполнении пространства в оперативной памяти или времени счета, объясните этот трюк в комментарии или в техническом описании.
- В подпрограммах:
 - укажите входные параметры и их тип; укажите, изменяются ли они в подпрограмме, обратив внимание на «побочный эффект»;
 - укажите выходные параметры и их тип;
 - пишите подпрограммы, не смешивая параметры, перечислите вначале входные параметры, затем входные и выходные параметры и, наконец, выходные параметры.

Следует постоянно помнить о трудностях отладки, поэтому:

- не используйте идентификаторы, начинающиеся с буквы Ø;
- избегайте одновременного использования переменных, отличающихся только буквой Ø и цифрой 0.
Например. Старайтесь не использовать в программе таких идентификаторов, как XØ и X0;
- стремитесь называть целые переменные идентификаторами, начинающимися с букв I, J, K, L, M, N, чтобы не противоречить опыту другого случайного читателя;
- старайтесь выбирать «мнемонические» идентификаторы (не забывая при этом предыдущий совет);
- предусмотрите временную промежуточную печать, чтобы облегчить отладку. Для таких временных инструкций очень удобны X-карты¹⁾ (условная компиляция), если, конечно, компилятор их воспринимает.

Чтобы явно продемонстрировать некоторые трудности, мы не всегда следовали этим советам в последующих упражнениях. Но это не может служить оправданием для читателя!

Следующая программа написана для компилятора с Фортрана, допускающего расширение стандартной формы инструкции READ. Действительно, в этой инструкции можно задать метку, которая указывает, куда следует перейти в слу-

¹⁾ См. книгу того же автора «Le langage FORTRAN IV», вышедшую в издательстве Dunod.

чае ошибки при чтении данных (если, например, встречается некорректный формат)¹⁾.

Программа выполнялась с терминала в режиме разделения времени.

Приложение: листинг, соответствующий блок-схеме 1.11.

```

LOGICAL B
READ(1,100)D,V1,V2,V,N
WRITE(1,102)D,V1,V2,V,N
Y=D
S=0.
B=.TRUE.
DO 20 I=1,N
IF(B)GO TO 10
B=.TRUE.
T=Y/(V+V1)
GO TO 15
10 B=.FALSE.
T=Y/(V+V2)
15 X=T*V
S=S+X
Y=Y-T*(V1+V2)
20 WRITE(1,101)I,X,S
100 FORMAT(4F4.0,13)
101 FORMAT(13,F10.2,F15.2)
102 FORMAT(3H D=,F5.0,2X,3HV1=,F3.0,2X,3HV2=,F3.0,
1 3X,2HV=,F5.0,' ЧИСЛО ОТРЕЗКОВ. =',I3,' I
2 S'/)
END

```

Пример выполнения программы:

D= 600. V1=40. V2=60. V= 200. ЧИСЛО ОТРЕЗКОВ =25

I	X	S
1	461.54	461.54
2	307.69	769.23
3	165.68	934.91
4	110.45	1045.36
5	59.48	1104.84
6	39.65	1144.49
7	21.35	1165.84
8	14.23	1180.07
9	7.66	1187.74
10	5.11	1192.85
11	2.75	1195.60
12	1.83	1197.43
13	.99	1198.42
14	.66	1199.08
15	.35	1199.43
16	.24	1199.67
17	.13	1199.80
18	.08	1199.88
19	.05	1199.93
20	.03	1199.96
21	.02	1199.97
22	.01	1199.98
23	.01	1199.99
24	.00	1199.99
25	.00	1200.00

¹⁾ Стандартный, ныне действующий Фортран не позволяет писать ERR= и END= в инструкциях ввода/вывода. Эти возможности появятся в новом стандарте Фортрана, проект которого в настоящее время подготовлен.

```

LOGICAL B
5 READ(1,100,ERR=1000)D,V1,V2,V,N
WRITE(1,102)D,V1,V2,V,N
Y=D
S=0.
B=.TRUE.
DO 20 I=1,N
IF(B)GO TO 10
B=.TRUE.
T=Y/(V+V1)
GO TO 15
10 B=.FALSE.
T=Y/(V+V2)
15 X=T*V
S=S+X
Y=Y-T*(V1+V2)
20 WRITE(1,101)I,X,S
100 FORMAT(4F4.0,13)
101 FORMAT(I3,F10.2,F15.2)
102 FORMAT(3H D=,F5.0,2X,3HV1=,F3.0,2X,3HV2=,F3.0,
1 3X,2HV=,F5.0,' ЧИСЛО ОТРЕЗКОВ =',I3/' I
2 S'/)
STOP 00
1000 WRITE(1,103)
GO TO 5
103 FORMAT(' ОШИБКА В ФОРМАТЕ, ПОВТОРИТЕ')
END

```

← Специальная литера, указывающая, что IBM ждет ввода

* AB60 40. 60. 200. 25 ← Ввод букв вместо цифр вызывает
ОШИБКА В ФОРМАТЕ, ПОВТОРИТЕ переход к инструкции
с меткой 1000

* 600. 40. 60. 200. 25

D= 600. V1=40. V2=60. V= 200. ЧИСЛО ОТРЕЗКОВ = 25

I	X	S
1	461.54	461.54
2	307.69	769.23
3	165.68	934.91
4	110.45	1045.36
5	59.48	1104.84
6	39.65	1144.49
7	21.35	1165.84
8	14.23	1180.07
9	7.66	1187.74
10	5.11	1192.85
11	2.75	1195.60
12	1.83	1197.43
13	.99	1198.42
14	.66	1199.08
15	.35	1199.43
16	.24	1199.67
17	.13	1199.80
18	.08	1199.88
19	.05	1199.93
20	.03	1199.96
21	.02	1199.97
22	.01	1199.98
23	.01	1199.99
24	.00	1199.99
25	.00	1200.00

STOP 00

ГЛАВА 2. УПРАЖНЕНИЯ С ЦЕЛЫМИ ЧИСЛАМИ

2.1. Таблица Пифагора

Таблица Пифагора — это квадратная матрица из 10 строк и 10 столбцов, каждый элемент которой определяется формулой

$$A(I, J) = I \times J.$$

Пример

Пример			J							
	1	2	3	4	5	6	7	8	9	10
1										
2										
3										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										
⋮										

Задача

Построить блок-схему и написать программу на Фортране, которые позволяют напечатать таблицу Пифагора.

Решение

Первое решение заключается в построении двумерного массива, который затем печатается по некоторому

соответствующим образом выбранному формату. Это решение приводит к блок-схеме, изображенной на рис. 2.1.

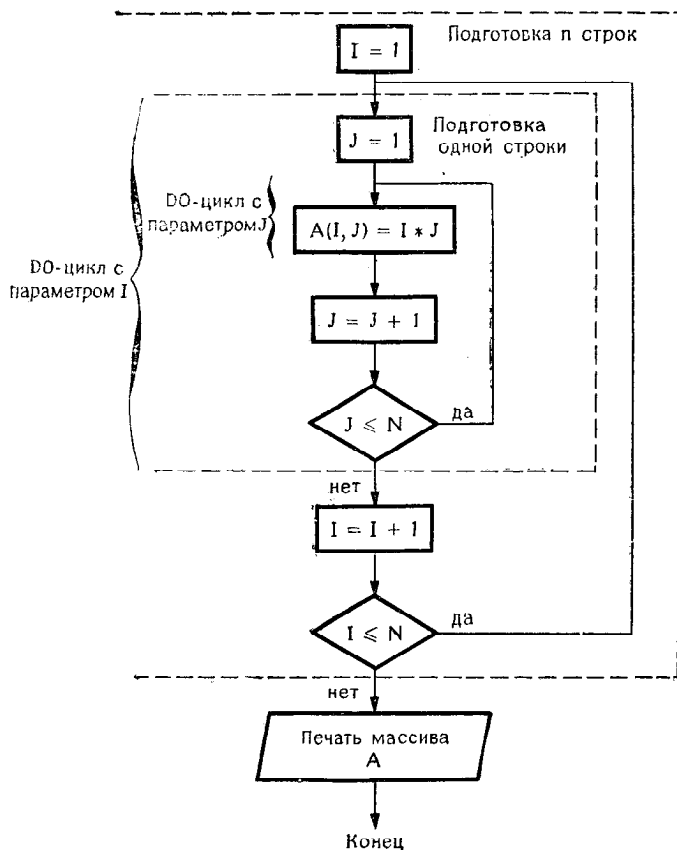


Рис. 2.1.

Очевидно, что массив A будет содержать значения типа INTEGER.

Если известно, что матрица симметрическая, то без всякого риска мы можем напечатать ее с помощью инструкции `WRITE( ) A`, которой соответствует `FORMAT (10I5)`¹⁾

¹⁾ Таким образом, печатается транспонированная матрица, которая в данном случае не отличается от исходной. — *Прим. ред.*

Таким образом, мы получим следующую программу:

```

INTEGER A(10,10)
DATA IMP/1/
DO 10 I=1,10
DO 10 J=1,10
10   A(I,J)=I*J
WRITE(IMP,20)A
20   FORMAT(10I5/)
END

```

Значение зависит от используемого устройства вывода

Можно использовать эту простую форму, поскольку матрица A симметрическая

Пропуск строки для более наглядного представления результатов

Эта программа дает следующий результат:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

При другом решении задачи каждая подготовленная строка немедленно печатается, что позволяет обойтись лишь одномерным массивом.

Блок-схема, соответствующая этому решению, несколько отличается от предыдущей и представлена на рис. 2.2.

Такой подход позволяет:

- более экономично использовать оперативную память и
- печатать даже несимметрическую матрицу с сохранением порядка строк и столбцов.

Блок-схеме соответствует следующая программа на Фортране:

```

INTEGER A(10)
DATA IMP/4/
DO 20 I=1,10
DO 10 J=1,10
10   A(J)=I*J
20   WRITE(IMP,30)A
30   FORMAT(10I5/)
STOP
END

```

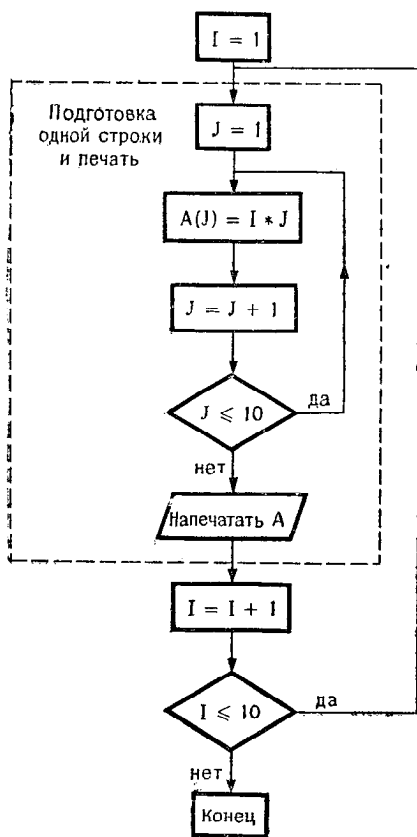



Рис. 2.2.

Нам осталось только расположить таблицу в начале страницы и предпослать ей заголовок, например, такой:

ТАБЛИЦА ПИФАГОРА

После внесения небольших изменений программа, написанная в соответствии с первым методом, будет выглядеть так:

```

10  INTEGER A(10,10)
    DATA IMP/4/
    DO 10 I=1,10
      DO 10 J=1,10
        A(I,J)=I*J
        WRITE(IMP,20)A
20  FORMAT(1H1,10X,'ТАБЛИЦА ПИФАГОРА'////(10I5/))
    STOP
    END
  
```

Повторяется только заключительная спецификация формата (10I5/) и поэтому заголовок печатается всего один раз

Для перехода к новой странице

2.2. НОК двух целых чисел

Метод, позволяющий получить Наименьшее Общее Кратное (НОК) двух чисел A и B , сводится к вычислению Наибольшего Общего Делителя (НОД) чисел A и B и затем к определению НОК по формуле

$$\text{НОК}(A, B) = \frac{A \times B}{\text{НОД}(A, B)}.$$

Пример

$$A = 24 \quad \text{НОД} = 6$$

$$B = 30$$

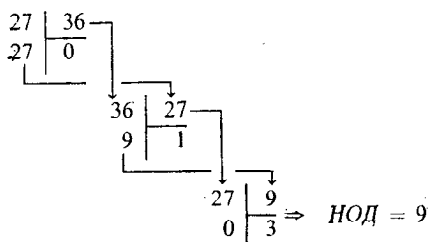
$$\text{НОК} = \frac{24 \times 30}{6} = 120.$$

Чтобы найти НОД, можно поступить следующим образом¹⁾:

- разделить A на B и получить частное Q и остаток R ;
- если $R = 0$, то делитель и есть НОД;
- если $R \neq 0$, то разделить B на Q и продолжить процесс до тех пор, пока не получится остаток, равный нулю. Последний делитель и есть НОД.

Пример

НОД 27 и 36.



Задача

1. нарисовать блок-схему, соответствующую этому методу вычисления НОК;
 2. написать программу, выполняющую это вычисление;
 3. написать главную программу, которая читает A и B и печатает НОК, а также подпрограмму типа FUNCTION, которая вычисляет НОК.
- Эта функция НОК имеет два параметра, соответствующие каждому из чисел.

¹⁾ Алгоритмы нахождения НОД подробно обсуждаются в книге Д. Кнута «Искусство программирования для ЭВМ», т. 2, М., «Мир», 1977, п. 4.5.2. — *Прим. ред.*

Решение

Прежде чем нарисовать блок-схему, следует провести тщательный анализ задачи. Итак, последовательность делений можно организовать следующим образом:

1. Ввести A и B ;
2. Вычислить Q и R , такие, что $A = B \cdot Q + R$;
3. Если $R = 0$, то установить $\text{НОД} = B$ и перейти к 4.
 Если $R \neq 0$, то
 - переслать значение B в A ,
 значение Q в B ;
 - перейти к 2;

4. вычислить НОК.

Но теперь уже нельзя вычислить НОК, поскольку начальные значения A и B потеряны. Чтобы этого не случилось, надо сохранить значения A и B и использовать две вспомогательные переменные X и Y , которым вначале присваиваются соответственно значения A и B .

Итогом этого анализа будет блок-схема на рис. 2.3.

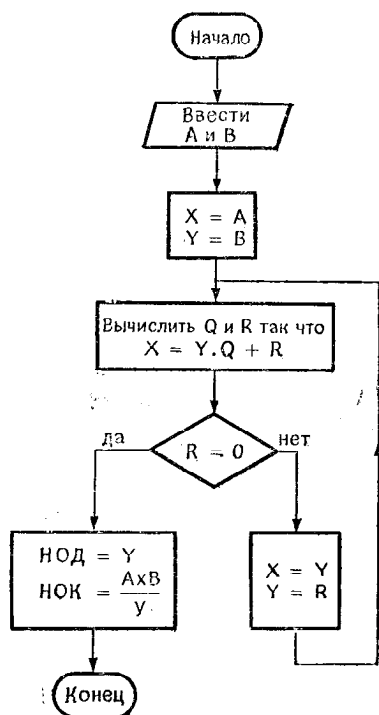


Рис. 2.3.

Существуют различные способы вычисления Q и R .

1-й способ

$$\left. \begin{array}{l} Q = A/B \\ R = A - Q * B \end{array} \right\} \text{ имея в виду, что } Q, A, B \text{ типа INTEGER.}$$

2-й способ

$$Q = A/B$$

$$R = \text{MOD}(A, B).$$

Учитывая, что функция MOD реализована не на всех вычислительных машинах, мы остановим свой выбор на первом способе и получим следующую программу:

```

      INTEGER FUNCTION HOK(A,B)
      INTEGER A,B,R,Q,X,Y
      X=A
      Y=B
10    Q=X/Y
      R=X-Q*Y
      IF(R.EQ.0) GO TO 20
      X=Y
      Y=R
      GO TO 10
20    HOK=A*B/Y
      RETURN
      END
  
```

Всякая программная компонента, в которой имеется обращение к функции HOK, должна содержать декларацию INTEGER HOK (без этой декларации результаты будут неверны):

```

      INTEGER HOK, U, V
      .
      .
      .
      I = HOK(24, 36)
      .
      .
      .
      I = HOK(U, V)
  
```

2.3. Совершенные числа

Число называется совершенным, если оно равно сумме своих делителей, включая 1, но не включая само себя. Например, 6 — совершенное число, потому что

$$6 = 1 + 2 + 3^1).$$

¹⁾ Первые совершенные числа суть 6, 28, 496, 8128; легко заключить, что они довольно редки.

Задача

Написать программу, которая позволит находить совершенные числа на отрезке, заданном двумя числами M и N . Естественно искать совершенные числа только среди четных чисел.

Решение

Метод заключается в нахождении всех делителей числа. Как только делитель найден, он прибавляется к переменной T , значение которой будет равно в конечном итоге сумме делителей. Если T равно проверяемому числу, то оно является совершенным.

Первая идея состоит в нахождении делителей числа I между 1 и $I/2$. В соответствии с другой идеей поиск ведется только между 1 и $\sqrt{I}$ и если делитель найден, то к T прибавляются как делитель, так и частное.

Пример

1-я идея:

$$I = 6 \quad I/2 = 3$$

$$T = 1 \quad I/2 = 3 \quad T \Rightarrow 1 + 2 = 3$$

$$I/3 = 2 \quad T \Rightarrow 3 + 3 = 6$$

2-я идея:

$$I = 6 \quad \sqrt{I} = 2,45$$

$$T = 1 \quad I/2 \Rightarrow 3 \quad T \Rightarrow 1 + 2 + 3.$$

Таким образом, вторая идея позволяет сократить количество проверок, и именно она реализована в блок-схеме 2.4.

Прежде чем написать программу, надо подумать еще и о том, что пользователь может задать значение $M > N$, а это ошибка, которую необходимо обнаруживать. Кроме того, в выбранном нами методе предполагается, что число M четное. Если же оно таковым не является, то следует взять наибольшее четное число, меньшее M^1). Поэтому мы пишем

$$M = 2 * (M/2).$$

¹⁾ Если следовать этому указанию автора, то результат не всегда будет соответствовать условию задачи. Так, при $M = 7$ и $N = 30$ вместо одного числа 28 мы получим 6 и 28. Как уточнить указание и исправить программу? (Не забудьте об условии $M > N$.) — *Прим. ред.*

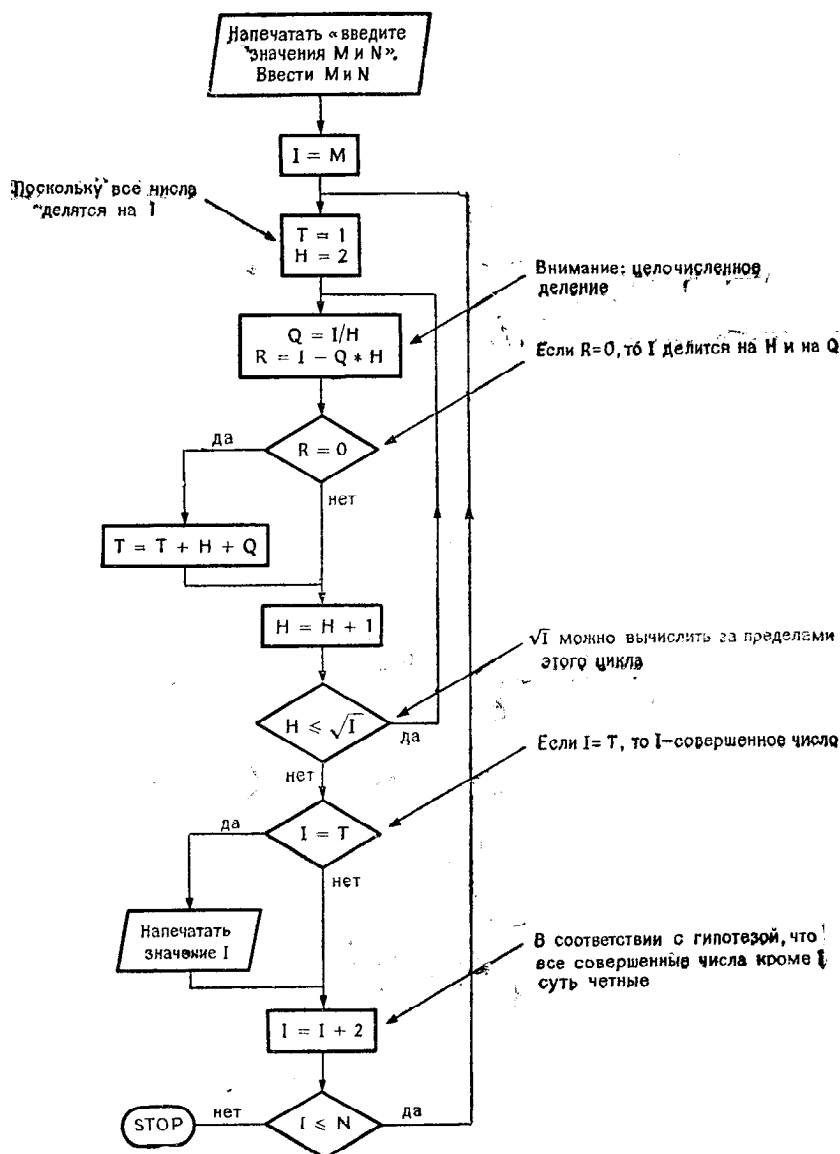


Рис. 2.4.

Эти замечания учтены в программе, которая приводится ниже в качестве примера:

```

INTEGER H,R,Q,T
WRITE(1,30)
READ(2,40)M,N
IF(M.GT.N) STOP 0001
M=2*(M/2)
DO 20 I=M,N,2
  T=1
  IS=SQR(FL0AT(1))
  DO 10 H=2,IS
    Q=I/H
    R=I-Q*H
  10   IF(R.EQ.0) T=T+Q+H
  20   IF(1.EQ.T) WRITE(1,50) I
STOP 0000
30  FORMAT(1X,15HВВЕДИТЕ М И N : )
40  FORMAT(215)
50  FORMAT(1X,15,12H СОВЕРШЕННОЕ /)
END

```

Об ошибке свидетельствует это значение, которое печатается при выполнении инструкции STOP

Получение четного значения M

Здесь функция FL0AT необходима, поскольку во время компиляции неизвестно, что SQR — стандартная функция

Форматы желательно группировать в начале или в конце программы

2.4. Поиск простых чисел

Существуют различные методы, позволяющие находить простые числа. В одном из быстрых методов используется тот факт, что эти числа (начиная с 5) можно представить как $6n \pm 1$, где n — целое положительное число.

Чтобы узнать, является ли число простым, достаточно делить его на ранее найденные простые числа до тех пор, пока не выполнится одно из следующих условий:

- деление без остатка: число не является простым;
- частное меньше или равно делителю: число простое.

Пример

Известно, что 1, 2, 3, 5, 7, 11, 13, 17, 19 — простые числа

$$n=4 \quad A=6 \times 4 - 1 = 23$$

$$B=6 \times 4 + 1 = 25$$

$$\begin{array}{r} 23 \overline{) 5} \\ 3 \overline{) 4} \end{array} \rightarrow 23 \text{ — простое число}$$

$$\begin{array}{r} 25 \overline{) 5} \\ 0 \overline{) 5} \end{array} \rightarrow 25 \text{ — не простое число}$$

$$n=5 \quad A=29$$

$$B=31$$

$$\begin{array}{r} 29 \overline{) 5} \\ 4 \overline{) 5} \end{array} \rightarrow 29 \text{ — простое число}$$

Задача

1. Проанализировать задачу с тем, чтобы уточнить алгоритм:
 - числа вида $6n \pm 1$ не могут быть ни четными, ни кратными 3, поэтому попытайтесь определить алгоритм, исключаяющий бесполезные операции деления;
 - посмотрите, принесет ли пользу оформление части алгоритма в виде подпрограммы.
2. Нарисовать одну или несколько блок-схем и написать соответствующие программы на Фортране.

Решение

1. Алгоритм

Вопрос использования ранее найденных простых чисел решается совсем легко, если они заносятся в массив T . В этот же массив нужно предварительно поместить числа 1, 2 и 3.

Так как числа вида $6n \pm 1$ не могут быть ни четными, ни кратными 3, то в качестве первого делителя можно взять сразу 5, а это означает, что число 5 также предварительно помещается в массив T , хотя затем оно повторно вычисляется.

Будем искать простые числа в диапазоне от 1 до $6n + 1$; значение n вводится в самом начале выполнения программы.

Таким образом, мы приходим к следующему алгоритму:

1. ввести N
2. занести начальные значения в массив T

$$T(1) = 1 \quad T(3) = 3$$

$$T(2) = 2 \quad T(4) = 5$$

установить $I = 3$ I указывает последнюю заполненную ячейку в массиве T ;

установить $L = 1$ повторно вычислится $5 = 6 \times 1 - 1$.

3. Установить $A = 6L - 1$

$$B = 6L + 1$$

3. а) проверить, является ли A простым числом
если да, то $I = I + 1$ ¹⁾

$$T(I) = A$$

перейти к 3b),

если нет, то перейти к 3b)

¹⁾ Следует убедиться, что I не выходит за пределы массива T . [Заметим, что в блок-схеме и программе такая проверка отсутствует. — Прим. ред.]

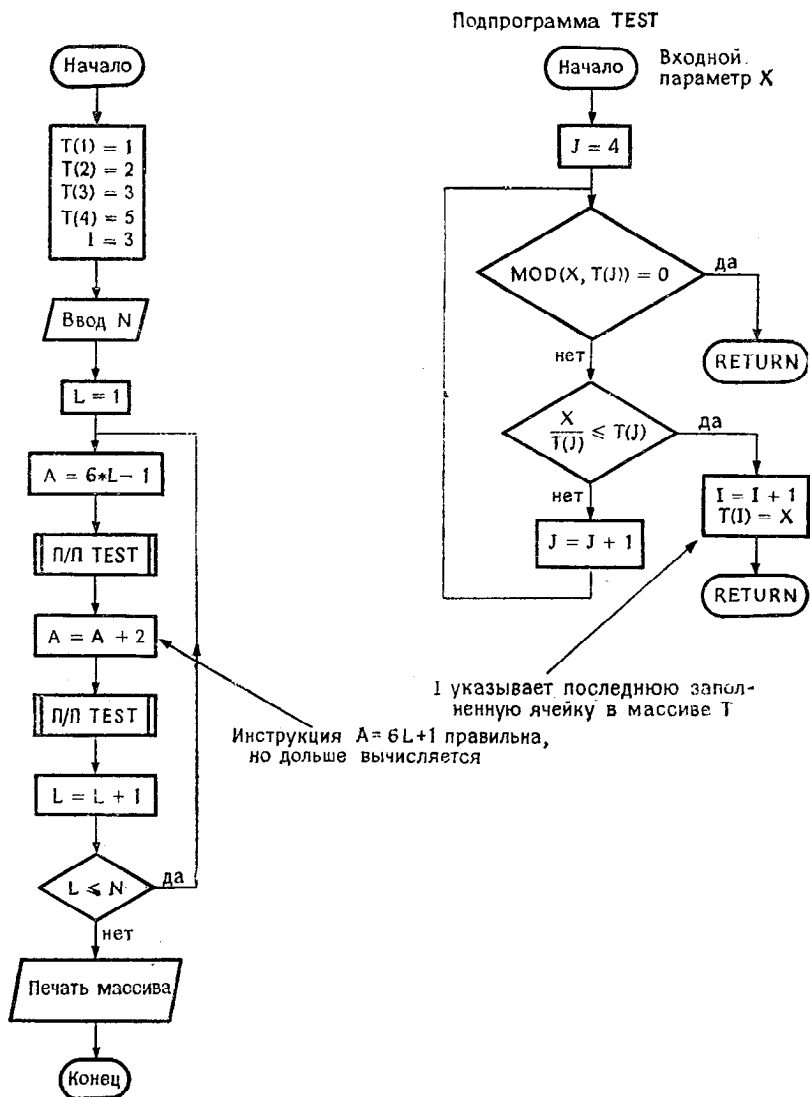


Рис. 2.5.

3b) проверить, является ли В простым числом

да: $I = I + 1$ ¹⁾ нет: ничего

$T(I) = B$

перейти к 4

4. $L = L + 1$

если $L \leq N$, то перейти к 3.

2. Блок-схема

Составление блок-схемы (рис. 2.5) не представляет никаких трудностей.

Перед программированием надо заметить, что массив Т и переменная I используются как в главной программе, так и в подпрограмме TEST. Следовательно, Т и I целесообразно разместить в общей (COMMON) области.

```

INTEGER T(10000), A, B
COMMON T, I
T(1)=1
T(2)=2
T(3)=3
T(4)=5
WRITE(1,50)
IMP=1
I=3
READ(2) N
DO 10 L=1, N
  A=6*L-1
  B=A+2
  CALL TEST(A)
  CALL TEST(B)
10  WRITE(IMP,20) I, (T(K), K=1, I)
20  FORMAT(18H СПИСОК СОДЕРЖИТ ,I5,17H ПРОСТЫХ ЧИСЕЛ
1  ///(10I7))
STOP
50  FORMAT(36H ЗАДАЙТЕ ЧИСЛО N, СИСТЕМА
1  /46H НАЙДЕТ ВСЕ ПРОСТЫЕ ЧИСЛА <ИЛИ = 6*N+1: )
END

```

Используется особенность системы
разделения времени, которая допус-
кает бесформатный ввод

```

SUBROUTINE TEST(M)
COMMON T, I
INTEGER T(10000), R
DO 20 J=4, I
  L=T(J)
  IF(L+L.GT.M) GO TO 25
  R=MOD(M,L)
  IF(R.EQ.0) GO TO 30
20  CONTINUE
25  I=I+1
  T(I)=M
30  RETURN
END

```

¹⁾ См. примечание на стр. 39.

Результат выполнения программы:

ЗАДАЙТЕ ЧИСЛО N, СИСТЕМА
НАЙДЕТ ВСЕ ПРОСТЫЕ ЧИСЛА <или = 6*N+1:11

СПИСОК СОДЕРЖИТ 20 ПРОСТЫХ ЧИСЕЛ

1	2	3	5	7	11	13	17	19	23
29	31	37	41	43	47	53	59	61	67

STOP NUMBER 00

2.5. Разложение числа на простые множители

Разложить целое число M на простые множители — это значит найти все его делители, которые являются простыми числами ¹⁾.

Пример

$$792 = 2 \times 2 \times 2 \times 3 \times 3 \times 11.$$

В действительности часто пытаются пойти дальше и определить, сколько раз каждый простой делитель делит число M . Тогда мы запишем

$$792 = 2^3 \times 3^2 \times 11.$$

Метод, позволяющий осуществить разложение

1. Ввести число M . Если $M \leq 0$, то остановиться.
2. Установить $I = 1$.
3. Установить $I = I + 1$.
4. Если $M \leq I$, то вернуться к 1 для обработки следующего числа ²⁾.

Если $M > I$, то проверить, является ли I делителем M .

если I — не делитель, то перейти к 3.

если I — делитель, то напечатать I ,

- установить $M = M/I$,
- перейти к 4.

2.5.1. Задача 1

- Нарисовать блок-схему, соответствующую изложенному методу.
- Написать программу на Фортране, соответствующую этой блок-схеме.

¹⁾ Если читателя заинтересует эта задача, то он может найти немало полезных подробностей в книге Д. Кнута «Искусство программирования для ЭВМ», т. 2, М., «Мир», 1977, п. 4.5.4. — *Прим. ред.*

²⁾ В этой строке ошибка. Вы легко в этом убедитесь, если «прокрутите» алгоритм для $M = 4$ или $M = 6$. — *Прим. ред.*

2.5.2. Задача 2

Теперь мы хотим сконструировать более совершенную программу, которая определяет еще и «частоту» каждого делителя.

- проанализируйте задачу и составьте блок-схему.
- затем напишите программу, которая *с помощью одного только формата вывода* (если число не простое) позволяет напечатать результат в таком виде:

```

7920    ДЕЛИТСЯ НА      2    4    РАЗ
        ДЕЛИТСЯ НА      3    2    РАЗ
        ДЕЛИТСЯ НА      5    1    РАЗ
        ДЕЛИТСЯ НА     11    1    РАЗ

5    ПРОСТОЕ ЧИСЛО

```

2.5.3. Решение задачи 1

- Чтобы узнать, делится ли M на I , как и в некоторых предыдущих упражнениях (НОД, НОК), достаточно проверить, получится ли нулевой результат при вычислении $M - (M/I) \times I$ или $\text{MOD}(M, I)$.
- Как только найден простой делитель, его значение печатается, а число M делится на этот делитель.

Таким образом, мы приходим к блок-схеме, представленной на рис. 2.6.

При прямолинейном переводе блок-схемы получается следующая программа на Фортране¹⁾, которая в довольно неудобном виде печатает результаты, полученные при ее выполнении (см. стр. 44):

```

10  READ(1,40)M
    IF(M.LE.0) STOP 77
    WRITE(1,40)M
    I=1
20  I=I+1
    IF(M.LE.2)GO TO 10

30  IF(MOD(M,I).NE.0)GO TO 20
    WRITE(1,50)I
    M=M/I
    GO TO 30
40  FORMAT(16)
50  FORMAT(5X,15)
    END

```

¹⁾ Обратите внимание, что есть отличие между программой и блок-схемой. В блок-схеме используется отношение $M \leq I$, а в программе $M \leq 2$. Разное положение занимают ссылка (A) в блок-схеме и метка 30 в программе (см. также предыдущее примечание). — Прим. ред.

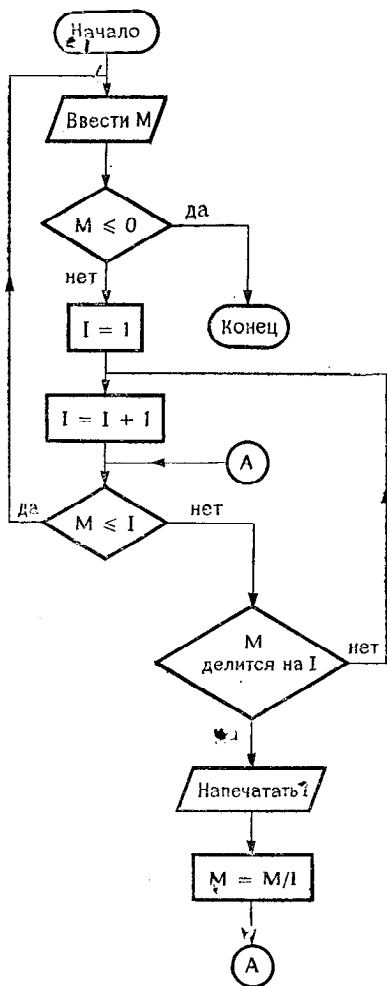


Рис. 2.6.

Результат выполнения программы:

7920

2
2
2
2
3
3
5
!!

2.5.4. Решение задачи 2

Предыдущая программа имеет следующие очевидные недостатки.

- плохо оформлена печать результатов,
- если исходное число простое, то оно вообще не печатается.

Искомый алгоритм должен определять простые числа и печатать их. Кроме того, необходимо иметь вспомогательную переменную, чтобы подсчитать, сколько раз делитель делит число.

Прочитанное число N делится, и если найдется делитель I , то как и прежде выполнится инструкция $N = N/I$. Поэтому необходимо либо напечатать значение N до того, как станет известно, простое ли число N , или нет, либо сохранить значение N в другой переменной. Мы выберем первое решение и воспользуемся управляющей литерой верстки $1H+$.

Среди возможных решений следующий алгоритм относится к довольно быстрым:

1. ввести N
если $N \leq 0$, остановка
если $N > 0$, напечатать значение N и перейти к 2;
2. установить $I = 1$

$L = .FALSE.$

$LIM = \sqrt{N}$ перейти к 3;

3. установить $I = I + 1$

$J = 0;$

- 3а) если $I \leq LIM$, перейти к 3с)

в противном случае перейти к 3б);

- 3б) если $L = .FALSE.$, напечатать „ПРОСТОЕ ЧИСЛО“ и перейти к 1,

если $L = .TRUE.$, перейти к 1;

- 3с) проверить, делится ли N на I

если да, то установить $N = N/I$

$J = J + 1$

$L = .TRUE.$

перейти к 3а)

если нет и если $J = 0$, то перейти к 3

если $J \neq 0$, напечатать

„ДЕЛИТСЯ НА“ I J „РАЗ“

и перейти к 3.

Этому алгоритму соответствует блок-схема на рис. 2.7.

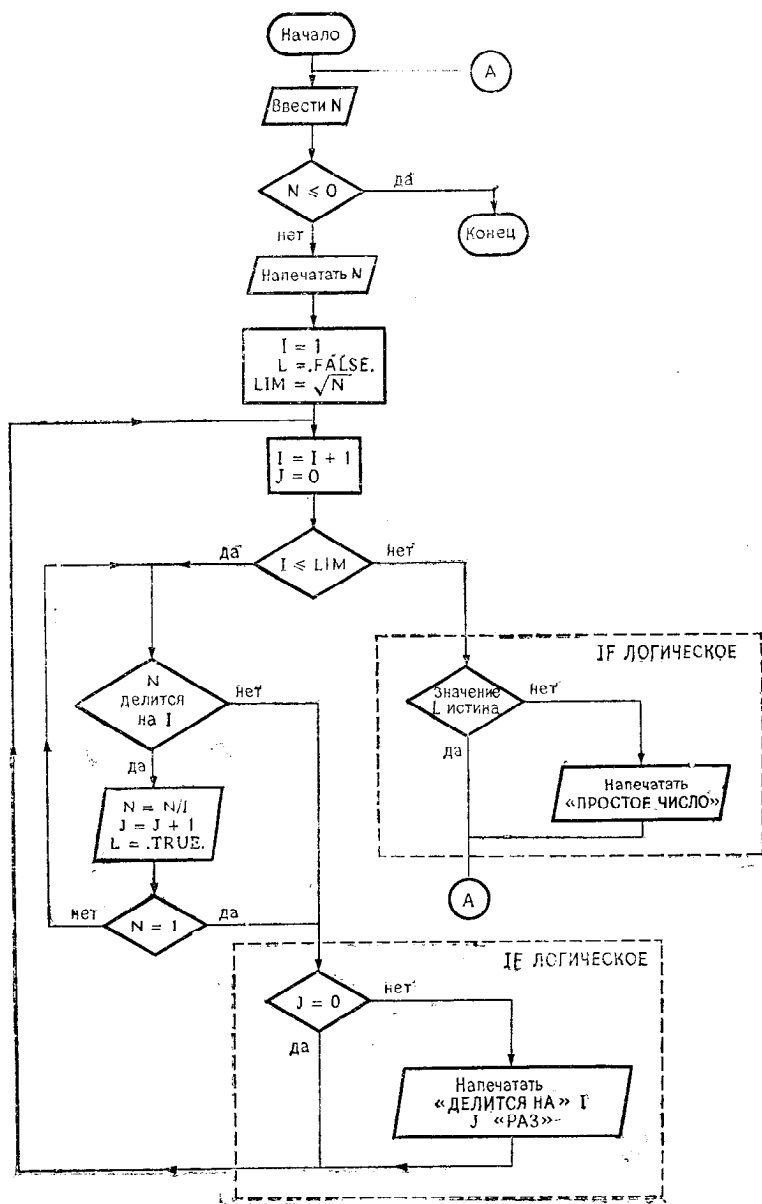


Рис. 2.7.

После того как блок-схема составлена, не представляет труда написать программу, но остается проблема редактирования и представления результатов:

- а) необходимо напечатать значение числа;
- б) на той же стороне нужно поместить текст

„ПРОСТОЕ ЧИСЛО“ или
„ДЕЛИТСЯ НА“ I J „РАЗ“.

Так как значение N редактируется прежде, чем известен текст, который за этим значением последует, этот текст должен быть напечатан в той же строке с помощью управляющей литеры верстки IH+.

Впрочем, если число имеет несколько делителей, то сведения о них надо напечатать в следующих строчках, располагая их соответствующим образом.

Все это учитывается в следующих форматах:

```
70 FORMAT(//I6)
80 FORMAT(IH+,7X,11HПРОСТОЕ ЧИСЛО/)
90 FORMAT(IH+,7X,17H ДЕЛИТСЯ НА ,2I4,5HРАЗ //)
```

Чтобы не печатать поверх ранее
напечатанного значения N

Описатель для переменных I и J

Для перехода на следующую строку

В конечном итоге получится такая программа:

```
LOGICAL L
10 READ(1,65)N
   IF(N.LE.0)STOP 77
   WRITE(1,70)N
   I=1
   L=.FALSE.
   LIM=SQRT(FLOAT(N))
(20 I=I+1
    J=0
    IF(I.LE.LIM)GO TO 40
    IF(.NOT. L)WRITE(1,80)
    GO TO 10
40 IF(MOD(N,I).NE.0)GO TO 50
   N=N/I
   L=.TRUE.
   J=J+1
   IF(N.NE.1)GO TO 40
50 IF(J.NE.0) WRITE(1,90)I,J
65 FORMAT(15)
70 FORMAT(//I6)
80 FORMAT(IH+,7X,11HПРОСТОЕ ЧИСЛО/)
90 FORMAT(IH+,7X,17H ДЕЛИТСЯ НА ,2I4,5HРАЗ //)
END
```

Напечатать значение числа

Напечатать на той же строке, начиная с 8 позиции ДЕЛИТСЯ НА и т.д., затем 2 раза перевести строку

ГЛАВА 3. ПРОСТЫЕ УПРАЖНЕНИЯ, ОРИЕНТИРОВАННЫЕ НА ЧИСЛЕННЫЙ АНАЛИЗ

3.1. Константа Эйлера, первая задача

Константа Эйлера определяется как

$$C = \lim_{n \rightarrow \infty} \left\{ \sum_{i=1}^n \frac{1}{i} - \ln n \right\}.$$

Можно получить приближенное значение C , вычисляя значение E по формуле

$$E = \sum_{i=1}^n \frac{1}{i} - \ln n$$

и полагая значение n равным нескольким тысячам.

Упражнение 1

Вычислить значение E , соответствующее заданному значению n ; сумма $\sum_{i=1}^n \left(\frac{1}{i}\right)$ вычисляется в обычном порядке.

Вычисления выполняются при

$$n = 1000, 2000, 10\,000, 32\,000.$$

Упражнение 2

Вычисление E выполняется при тех же значениях n , но слагаемые суммируются в обратном порядке.

Упражнение 3

То же задание, что и в упражнении 1, но вычисления выполняются с двойной точностью.

Упражнение 4

То же задание, что и в упражнении 2, но вычисления выполняются с двойной точностью.

Вопрос

Сравните полученные результаты. Почему они разные?

Решение к упражнениям 1 и 2

Оба метода легко выразить в виде блок-схем:

Упражнение 1

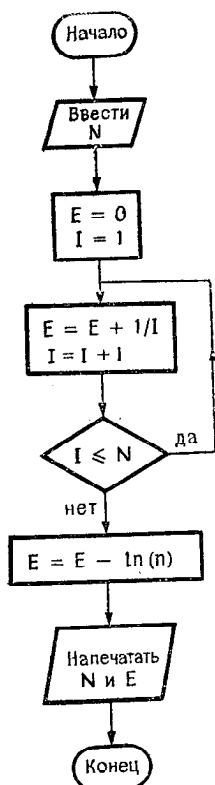


Рис. 3.1.

Упражнение 2

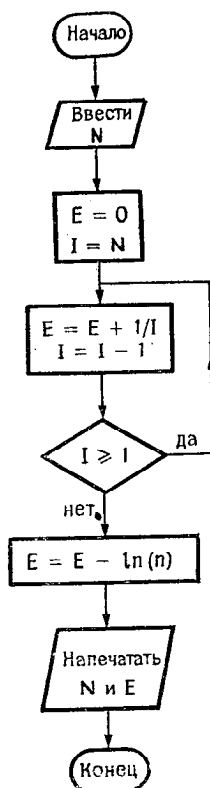


Рис. 3.2.

Без труда пишется программа, соответствующая упражнению 1:

```

READ(1,20)N
E=0.
DO 10 I=1,N
10  E=E+1./FLOAT(I)
E=E-ALOG(FLOAT(N))
WRITE(1,30)N,E
20  FORMAT(I5)
30  FORMAT(4H N=,I5,5X,2HE=,F13.10//)
STOP
END
  
```

Для большинства систем функция FLOAT в данном случае необходима.

Здесь функция FLOAT необходима

Следует обратить внимание на тот факт, что функция ALOG (натуральный логарифм) применяется только к вещественному аргументу.

Упражнение 2 легко программируется с помощью инструкций IF:

```

      E=0.
      READ(1,15) N
15    FORMAT(16)
      I=N
10    E=E+1./FLOAT(I)
      I=I-1
      IF(I.GE.1)GO TO 10
      E=E-ALOG(FLOAT(N))
      WRITE(1,20)N,E
20    FORMAT(3H N=,15,5X,'EULER = ',F12.8//)
      END

```

Результат выполнения программы:

```

      N=30000      EULER =      ,57721259

```

Можно также использовать DO-цикл, в котором шагращения должен быть положительным:

```

      READ(1,20)N
      E=0.
      DO 10 J=1,N
        I=N-J+1
10    E=E+1./FLOAT(I)
      E=E-ALOG(FLOAT(N))
      WRITE(1,30)N,E
20    FORMAT(16)
30    FORMAT(' N=',16,5X,'EULER = ',F10.8//)
      END

```

Эта, вообще говоря, элегантная программа при выполнении оказывается не более быстрой, чем предыдущая, но в ней показана ситуация, в которой также можно применить инструкцию DO.

Решение к упражнениям 2 и 3

Главное осложнение связано с тем, что приходится использовать переменные двойной точности (DOUBLE PRECISION). Функцию ALOG необходимо заменить функцией DLOG , которая применяется к аргументам двойной точности. Таким образом получается следующая программа:

```

DOUBLE PRECISION E
READ(1,20)N
E=0.D+0
DO 10 J=1,N
  I=N-J+1
10  E=E+1.D+0/DBLE(FL0AT(I))
  E=E-DLOG(DBLE(FL0AT(N)))
  WRITE(1,30)N,E
20  FORMAT(I6)
30  FORMAT(3H N=,I6,5X,7HEULER =,D20.12/)
  STOP
END

```

Замечание

Чтобы исключить обращения к функциям преобразования, нужно сделать так, чтобы в упражнениях 1 и 2 переменные I и N имели тип REAL, а в упражнениях 3 и 4 — тип DOUBLE PRECISION.

Пример

Упражнение 1

```

REAL I,N
DATA E,1/0.,1.0/
READ(1,20)N
10  E=E+1./I
  I=I+1.
  IF(I.LE.N)GO TO 10
  E=E-ALOG(N)
  WRITE(1,30)N,E
20  FORMAT(F6.0)
30  FORMAT(4H N =,F6.0,3X,7HEULER =,F13.10)
  STOP
END

```

Сравнение полученных результатов

Точное значение этой константы начинается с 0,5772156649...

Приведенная ниже таблица подготовлена с помощью вычислительной машины невысокой точности (в мантиссе 24 двоичных разряда при однократной точности и 39 при двойной точности).

Тип переменной	REAL		DOUBLE PRECISION	
	Упр. 1	Упр. 2	Упр. 3	Упр. 4
1000	0,57726765	0,57764530	0,5777155750	0,5777155805
2000	0,57640266	0,57731438	0,5774656283	0,5774656418
10000	0,56856918	0,57644272	0,5772655334	0,5772656512
32000		0,57452011	0,5772308427	0,5772312494

Мы констатируем, что обратное суммирование лучше, чем суммирование в прямом порядке (особенно для однократной точности). Это происходит по той причине, что ошибки округления в первом случае сказываются сильнее.

Кроме того, при однократной точности, начиная с некоторого значения N , наблюдается совершенно определенная тенденция отклонения от точного значения. Причина этого явления в явно недостаточной точности вычислений. Это подтверждается тем фактом, что результаты вычислений с двойной точностью постепенно приближаются к точному значению.

Низкая точность полученных результатов объясняется тем, что ряд $1/n$ расходящийся¹⁾ и последовательность

$$\sum_{i=1}^n \frac{1}{i} - \ln n$$

сходится медленно.

3.2. Константа Эйлера, вторая задача

Четыре предыдущих упражнения показали, что полученные результаты в значительной степени зависят от числа членов ряда, участвующих в вычислении. Теперь мы предлагаем написать программу, которая позволит напечатать полученные значения E для значений N , кратных 500 и при изменении N от 500 до 30 000 (суммирование производится в прямом порядке). Необходимо:

- нарисовать блок-схему;
- определить, как программа может обнаружить, что N кратно 500;
- если потребуется, то скорректировать блок-схему и затем написать соответствующую ей программу.

Решение

Обозначим через E_p и E_q приближенные значения константы Эйлера, полученные соответственно при $n = p$ и $n = q$.

Если $p \leq q$, то мы можем записать

$$E_p = S_p - \ln p,$$

$$E_q = S_q - \ln q = S_p + \sum_{i=p+1}^q \left(\frac{1}{i}\right) - \ln q.$$

¹⁾ См. также п. 1.2.7 в книге Д. Кнута «Искусство программирования для ЭВМ», т. 1, М., «Мир», 1976.

Разумеется, не следует повторно вычислять S_p и тратить на это время. Высказанное соображение учитывается в блок-схеме, изображенной на рис. 3.3.

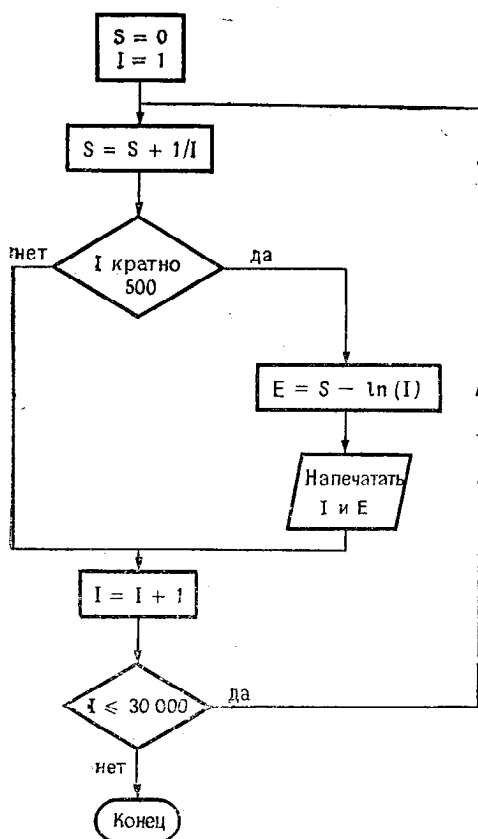


Рис. 3.3.

Чтобы обнаружить момент, когда I кратно 500, мы можем вычислять остаток от деления I на 500:

$R = I - I/500 * 500$, где R и I — целые числа.

В другом изящном решении используется функция MOD, которая вычисляет остаток от деления двух чисел: если I — целое число, то мы пишем

IF (MOD (I, 500).NE.0) GO TO ...

если I — вещественное число, то мы пишем

IF (AMOD(I, 500.).NE.0.) GO TO ...

Эти замечания позволяют написать следующую программу:

```
DATA IMP,S/4.0./
DO 10 I=1,30000
  S=S+1./FLOAT(I)
  IF(MOD(I,500.).NE.0)GO TO 10
  E=S-ALOG(FLOAT(I))
  WRITE(IMP,20)I,E
10 CONTINUE
20 FORMAT(' I=',I6,4X,'E=',F10.8)
END
```

3.3. Последовательность Фибоначчи и золотое сечение¹⁾

Последовательность чисел Фибоначчи определяется следующим рекуррентным соотношением:

$$\begin{cases} u_1 = 1, \\ u_2 = 2, \\ u_n = u_{n-1} + u_{n-2}. \end{cases}$$

Это значит, что

$$u_3 = u_2 + u_1 = 3, \quad u_4 = u_3 + u_2 = 5$$

и т. д.

Золотое сечение²⁾, используемое в архитектуре, можно получить как предел отношения

$$V_n = \frac{u_n}{u_{n-1}}.$$

¹⁾ Подробные сведения о числах Фибоначчи и золотом сечении читатель найдет в книге Д. Кнута «Искусство программирования для ЭВМ», т. 1, М., «Мир», 1976, п. 1.2.8. — *Прим. ред.*

²⁾ Точное значение равно $\frac{1}{2}(1 + \sqrt{5}) = 1,6180339887 \dots$ — *Прим. ред.*

Задача

Нарисовать блок-схему программы, которая вычисляет первые 20 членов последовательности U_n и печатает значения V_n при n , изменяющемся от 3 до 20.

Проанализируйте блок-схему и выясните, нельзя ли ее изменить так, чтобы более рационально использовалась оперативная память.

Написать соответствующую программу.

Решение

Если мы будем прямолинейно следовать рекуррентному соотношению, то получим блок-схему, изображенную на рис. 3.4.

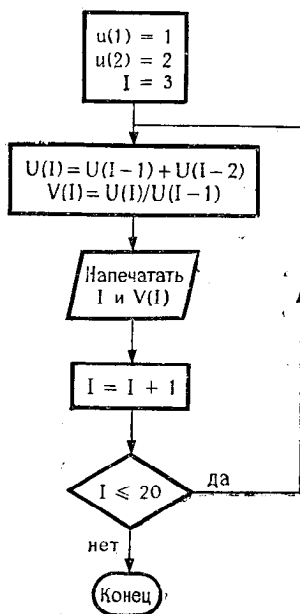


Рис. 3.4.

Теперь мы можем заметить, что бесполезно «запоминаться» последовательность значений, которые принимает отношение u_i/u_{i-1} , и, следовательно, можно использовать в качестве v переменную без индексов.

Кроме того, мы отмечаем, что для вычисления u_i нам необходимы значения u_{i-1} и u_{i-2} , но совсем не требуются другие величины: u_{i-3} , u_{i-4} и т. д. Поэтому мы вполне можем

обойтись тремя переменными A , B , C , которые будут содержать три последних значения из последовательности.

Итак, мы можем перейти к блок-схеме, изображенной на рис. 3.5.

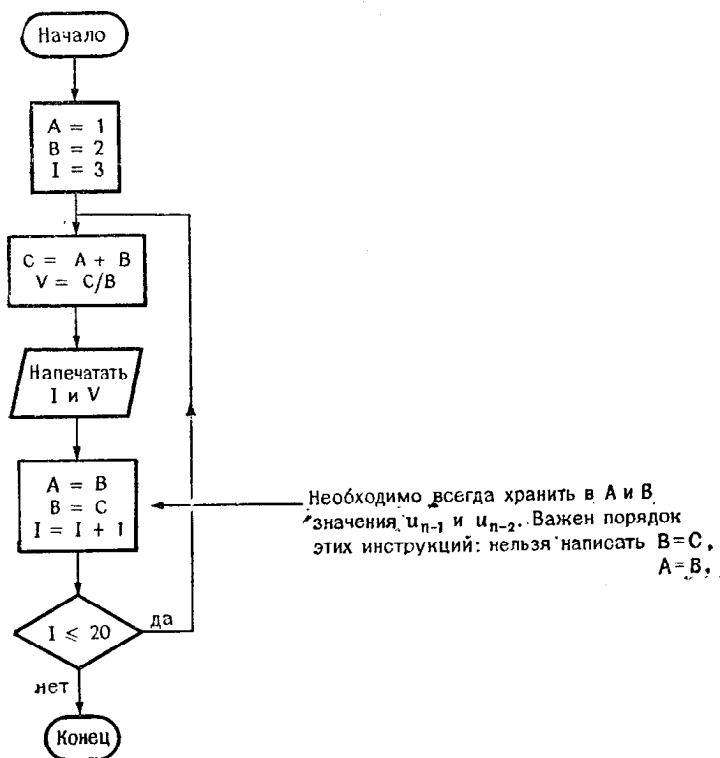


Рис. 3.5.

Исходя из этой блок-схемы, мы получим следующую программу:

```

A=1.
B=2.
DO 10 I=3,20
  C=A+B
  V=C/B
  WRITE(1,20)I,V
  A=B
  B=C
10  FORMAT(1X,I3,F13.8)
20  END
  
```

Результат выполнения программы:

3	1.500000000
4	1.666666667
5	1.600000000
6	1.625000000
7	1.61538461
8	1.61904762
9	1.61764706
10	1.61818182
11	1.61797753
12	1.61805555
13	1.61802575
14	1.61803713
15	1.61803279
16	1.61803445
17	1.61803381
18	1.61803406
19	1.61803396
20	1.61803400

3.4. След матрицы

След квадратной матрицы — это число, равное сумме диагональных элементов матрицы:

$$\text{След} = \sum_i A(i, i).$$

Написать функцию TRACE¹⁾, которая вычисляет эту величину в следующих случаях:

1-й случай. Используется самый очевидный метод.

2-й случай. Используется техника перехода к одному индексу.

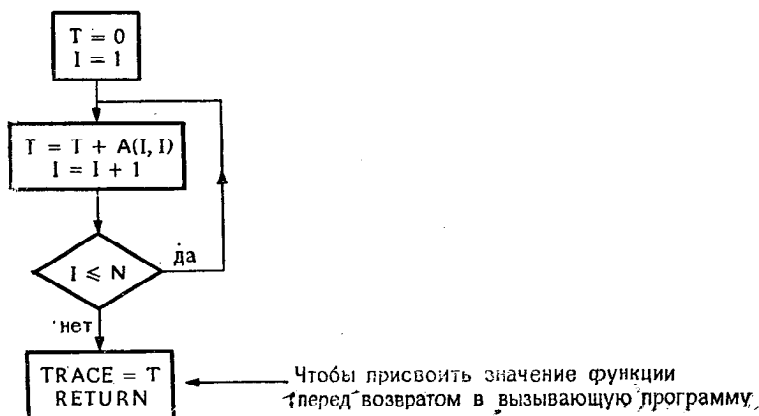


Рис. 3.6.

¹⁾ На русский язык переводится как «след». — Прим. ред.

Решение

1-й случай. Не представляет труда нарисовать блок-схему (рис. 3.6).

Теперь можно написать следующую функцию:

```

FUNCTION TRACE(A,N)
REAL A(N,N)
T=0.
DO 10 I=1,N
    T=T+A(I,I)
10  TRACE=T
RETURN
END

```

В этой программе можно заменить T на TRACE и, таким образом, исключить одну инструкцию:

```

FUNCTION TRACE(A,N)
REAL A(N,N)
TRACE=0.
DO 10 I=1,N
    TRACE=TRACE+A(I,I)
10  RETURN
END

```

Какая из этих программ окажется лучше, зависит от вычислительной машины и компилятора (чаще всего предпочтение отдается первому варианту).

2-й случай. В предыдущем методе сказывается неудобство обработки двумерного массива и в результате происходит потеря времени при выполнении программы. Можно попытаться обрабатывать матрицу так, как если бы это был одномерный массив, но вместе с тем учесть порядок, в котором элементы матрицы расположены в памяти.

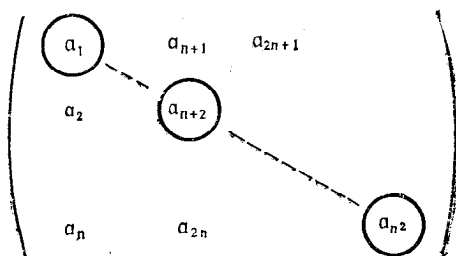


Рис. 3.7.

Диагональные элементы размещаются от a_1 к a_{n^2} с шагом $n+1$ (см. рис. 3.7). Таким образом, мы можем построить следующую блок-схему:

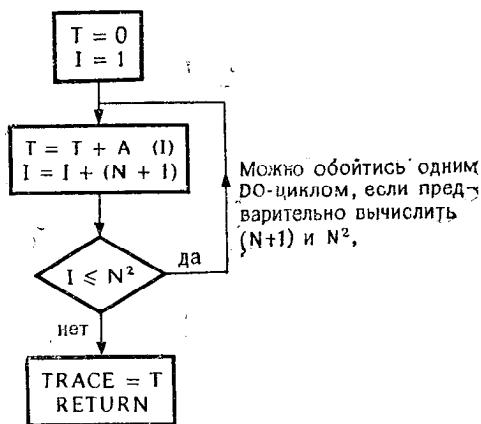


Рис. 3.8.

И хотя программа на Фортране оказывается длиннее, чем предыдущая, зато она быстрее выполняется.

```

FUNCTION TRACE(A,N)
REAL A(N2)
N2=N*N
T=0.
N1=N+1
DO 10 I=1,N2,N1
  T=T+A(I)
TRACE=T
RETURN
END
  
```

10

Можно также писать, REAL A(I), поскольку эта инструкция не требует резервировать память, а только указывает, что A-массив

Замечание

Механизм передачи параметров в некоторых ЭВМ, например в машине PHILIPS P880, не позволяет непосредственно использовать такую программу.

В главной программе нужно написать

```

REAL A (20, 20), B (400)
EQUVALENCE (A (1, 1), B (1))
.
.
.
X = TRACE (B, 20)
  
```

чтобы „усыпить бдительность“ компилятора

3.5. Определитель треугольной матрицы

Квадратная матрица называется верхней треугольной матрицей, если все элементы, расположенные под главной диагональю, нулевые.

Квадратная матрица называется нижней треугольной, если все элементы, расположенные над этой диагональю, нулевые.

Чтобы вычислить определитель треугольной матрицы, достаточно найти произведение элементов главной диагонали.

Задача 1

Нарисовать блок-схему программы, позволяющей вычислить определитель треугольной матрицы $T_{n,n}$. Затем написать программу на Фортране, соответствующую блок-схеме.

Задача 2

Проделать то же упражнение, используя технику перехода к одному индексу с целью повышения скорости выполнения программы.

Решение задачи 1

Переменной присваивается начальное значение, равное 1, и затем она последовательно умножается на все элементы главной диагонали. Таким образом, получается блок-схема, изображенная на рис. 3.9.

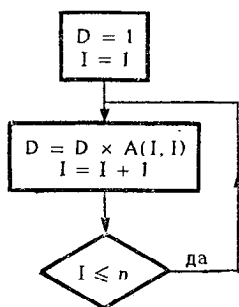


Рис. 3.9.

Соответствующая программа на Фортране очевидна.

```

10 |
   | D = 1.
   | DO 10 I = 1, N
   | D = D * A(I, I)
  
```

Решение задачи 2

Как и при вычислении следа матрицы, важно установить, что достаточно одного индекса, изменяющегося от 1 до n^2 с

шагом $n + 1$. Тогда можно написать следующую программу:

```

10 | D = 1
    | N1 = N + 1
    | N2 = N * N
    | DO 10 I = 1, N2, N1
    | D = D * A(I)

```

Предостережение

Не все компиляторы позволяют использовать массив с одним индексом, если он декларирован как двумерный массив. В этом случае можно обмануть бдительность компилятора, используя инструкцию EQUIVALENCE (см. замечание в разд. 3.4).

Пример

Чтобы вычислить определитель треугольной матрицы $B(10, 10)$, мы можем использовать вспомогательный массив A с одним индексом, сделав его эквивалентным массиву B .

```

10 | REAL B(10, 10), A(100)
    | EQUIVALENCE (A(1), B(1, 1))
    |
    |
    |
    | D = 1
    | DO 10 I = 1, 100, 11
    | D = D * A(I)

```

3.6. Транспонирование квадратной матрицы

Транспонировать матрицу это значит получить «симметричную» ей матрицу по отношению к главной диагонали. Таким образом, если B получена из A как результат транспонирования, то

$$a_{i,j} = b_{j,i}.$$

Задача 1

1. Нарисовать блок-схему подпрограммы TRANS1(A, B, N), в которой A — заданная матрица, N указывает размер матрицы и B — транспонированная матрица.

2. Написать программу, позволяющую проверить эту подпрограмму.

Задача 2

1. Нарисовать блок-схему подпрограммы TRANS2(A, N), в которой параметр A задает как исходную матрицу, так и выходную, транспонированную матрицу.

Решение задачи 1

Если принять во внимание предыдущие упражнения, то не составляет труда нарисовать блок-схему (рис. 3.10).

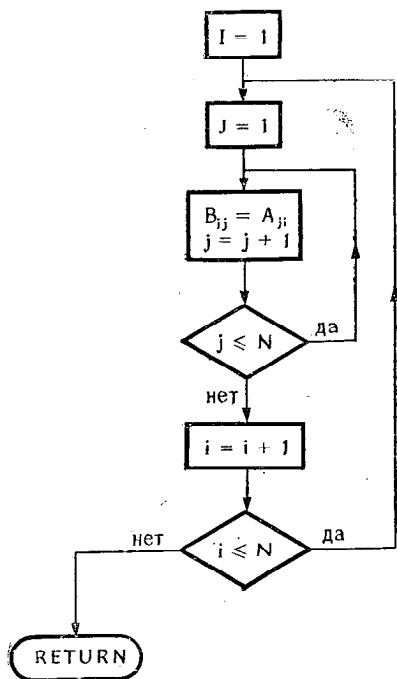


Рис. 3.10.

Теперь совершенно ясно, как написать соответствующую программу на Фортране:

```

SUBROUTINE TRANS1(A,B,N)
REAL A(N,N),B(N,N)
DO 10 I=1,N
  DO 10 J=1,N
    10  B(J,I)=A(I,J)
  RETURN
END
  
```

Решение задачи 2

Следует обратить внимание на то, что элементы нужно менять местами только один раз, а пересылки элементов главной диагонали вообще бессмысленны.

Следовательно, индекс i должен изменяться от 2 до n , а j — от 1 до $i-1$ (рис. 3.11). Таким образом, мы приходим к блок-схеме, изображенной на рис. 3.12.

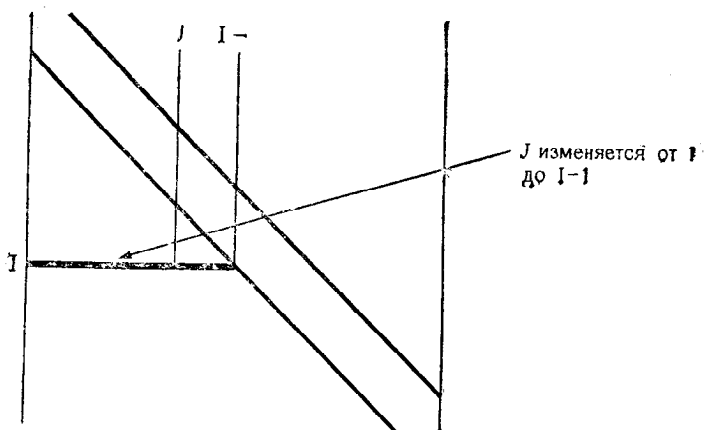


Рис. 3.11.

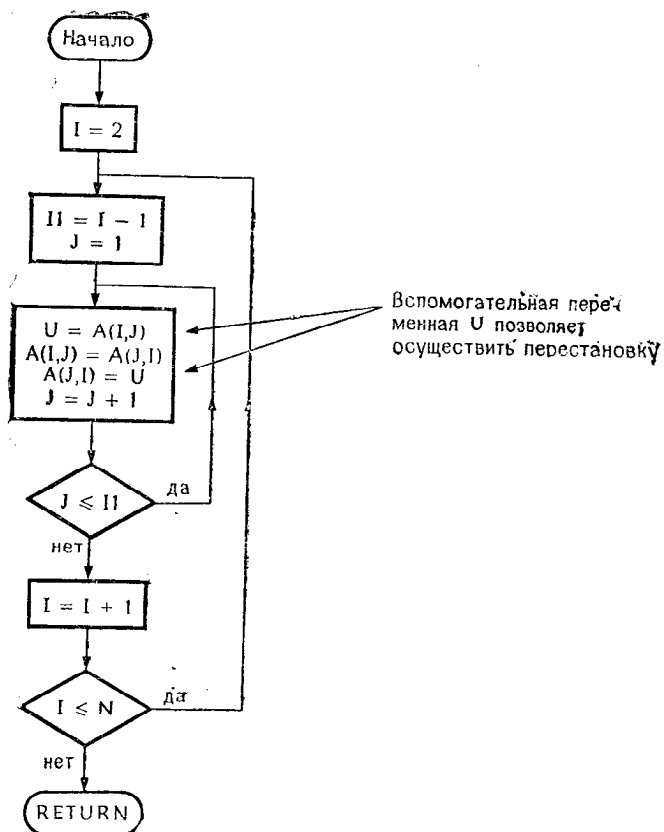


Рис. 3.12.

Теперь не представляет никакого труда написать программу

```

SUBROUTINE TRANS2(A,N)
REAL A(N,N)
DO 10 I=2,N
  I1=I-1
  DO 10 J=1,I1
    U=A(I,J)
    A(I,J)=A(J,I)
10  A(J,I)=U
RETURN
END

```

3.7. Решение уравнения $f(x) = x$ методом итерации

Существует простой метод, позволяющий решить уравнение вида $f(x) = x$. Он состоит в выборе начального приближенного значения x_0 и последовательном вычислении:

$$x_1 = f(x_0),$$

$$x_2 = f(x_1)$$

и т. д.

Если производная $f'(x)$ удовлетворяет условию $|f'(x)| < 1$, то последовательность x_i сходится к искомому корню.

Исходя из предположения, что метод может не сойтись, необходимо предусмотреть в программе следующие условия окончания работы:

- остановка, как только $|f(x) - x| \leq \varepsilon$,
- остановка, как только число итераций достигнет N . Величины ε и N задаются.

Задача 1

Нарисовать блок-схему программы, которая позволяет найти корень уравнения

$$e^{-x/10} = x.$$

Заданы следующие условия:

$$x_0 = 1,$$

$$\varepsilon = 0,00005,$$

$$N = 30.$$

Необходимо напечатать:

- число выполненных итераций,
- найденное значение x ,
- соответствующее значение $f(x)$.

В программе использовать DO-цикл.

Задача 2

Написать подпрограмму с названием SFEX, которая позволяет решить уравнение $f(x) = x$, если заданы следующие входные параметры:

- имя функции f ,
- начальное значение x_0 ,
- максимальное число итераций N ,
- требуемая точность ϵ .

Выходные параметры:

- найденное значение x ,
- соответствующее значение $f(x)$,
- количество выполненных итераций I .

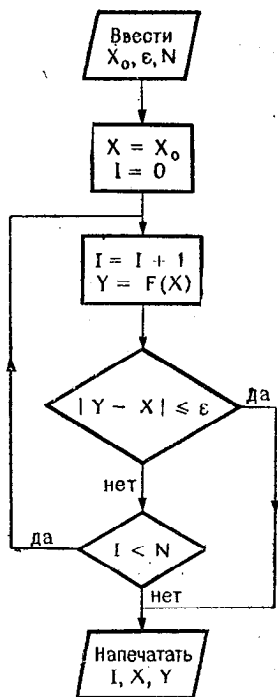


Рис. 3.13.

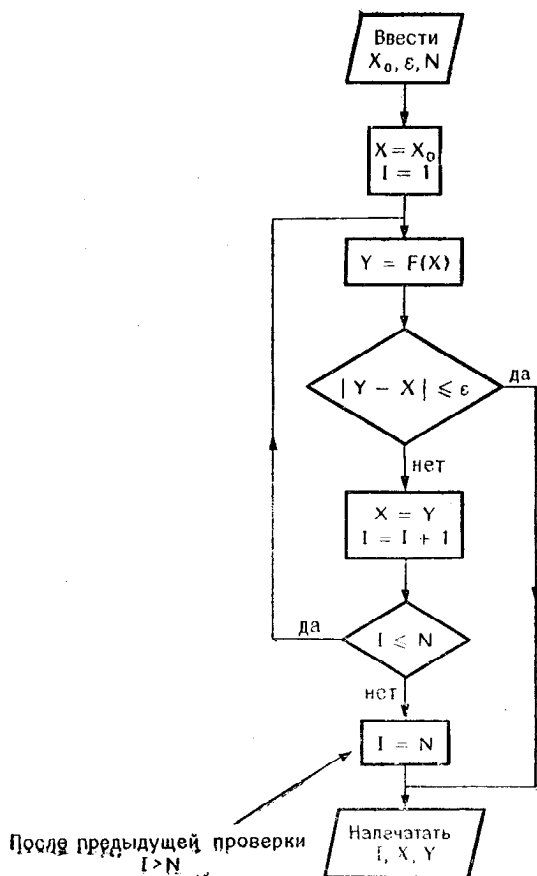


Рис. 3.14.

Форма обращения к этой подпрограмме такова:

CALL SFEX(F, X0, N, EPSI, X, Y, I).

Решение задачи 1

Решение, изображенное на рис. 3.13, представляется вполне естественным¹⁾. Но если используется ДО-цикл, то требуется несколько специфическая блок-схема (рис. 3.14), в которой учитывается тот факт, что при нормальном выходе из ДО-цикла значение управляющей переменной не определено²⁾.

Листинг программы соответствует блок-схеме, изображенной на рис. 3.14.

```

F(X)=EXP(-X/10.)
EPSI=0.00005
X0=1.
N=30
X=X0
DO 10 I=1,N
  Y=F(X)
  IF(ABS(Y-X).LE.EPSI)GO TO 20
10  X=Y
  I=I+1
20  WRITE(1,30)I,X,Y
30  FORMAT(3H I=,I3,4X,'X=',F10.6,4X,'Y=',F10.6)
END

```

Решение задачи 2

Подпрограмма SFEX частично повторяет предыдущую программу и поэтому программируется легко. Не следует забывать о декларации EXTERNAL в главной программе.

```

EXTERNAL F
EPSI=0.00005
X0=1.
CALL SFEX(F,1.,30,EPSI,X,Y,I)
WRITE(1,20)I,X,Y
20  FORMAT(3H I=,I3,4X,2HX=,F10.6,4X,2HY=,F10.6)
STOP
END

```

¹⁾ Но не совсем верным, поскольку значение X внутри цикла не изменяется. Можно предложить такое исправление: заменить инструкцию $X = X_0$ на $Y = X_0$ и вставить инструкцию $X = Y$ перед $I = I + 1$. — Прим. ред.

²⁾ В этой блок-схеме также есть ошибка. В том случае, когда заданная точность не достигнута и происходит выход из цикла по условию $I > N$, значения X и Y оказываются равными. Возможность исправить блок-схему и соответствующую программу предоставляется читателю в качестве упражнения (обратите внимание на предыдущее примечание). — Прим. ред.

```

SUBROUTINE SFEX(F,X0,N,EPSI,X,Y,J)
X=X0
DO 10 J=1,N
  Y=F(X)
  IF (ABS(Y-X).LE.EPSI) RETURN
10  X=Y
  J=N
RETURN
END

FUNCTION F(X)
F=EXP(-X/10.)
RETURN
END

```

Результат выполнения программы:

I= 5 X= 0.912771 Y= 0.912765

3.8. Решение уравнения методом хорд и методом деления пополам

Существуют многочисленные методы решения уравнения $f(x) = 0$. Из простых методов назовем метод Ньютона, рассмотренный ранее в другой моей книге, а также метод хорд и метод деления пополам, которые характеризуются медленной сходимостью, но реализация которых зато проста.

Отправные гипотезы

Для этих двух методов существенно, чтобы функция была непрерывна и ограничена в заданном интервале (a, b) , внутри которого ищется корень. Предполагается также, что $f(a)$ и $f(b)$ имеют разные знаки.

Метод хорд

Этот метод состоит в выполнении итераций, начиная с хорды, соединяющей точки А и В, между которыми заключен искомый корень. Хорда проводится так, как показано на рис. 3.15.

Алгоритм будет таким:

1. Провести прямую, соединяющую точки $(a, f(a))$ и $b, f(b)$.

Эта прямая пересекает ось x в точке x_1 :

- если $f(x_1)$ и $f(a)$ имеют разные знаки, то положить $b = x_1$ и перейти к 2,
- если $f(x_1)$ и $f(b)$ имеют разные знаки, то положить $a = x_1$ и перейти к 2.

2. Если условия остановки не удовлетворяются, то перейти к 1. Условия могут быть такими:

- $|f(x)| \leq \varepsilon$ (ε задается),
- интервал $(a - b) \leq \eta$ (η задается),
- количество итераций превысило заданную величину.

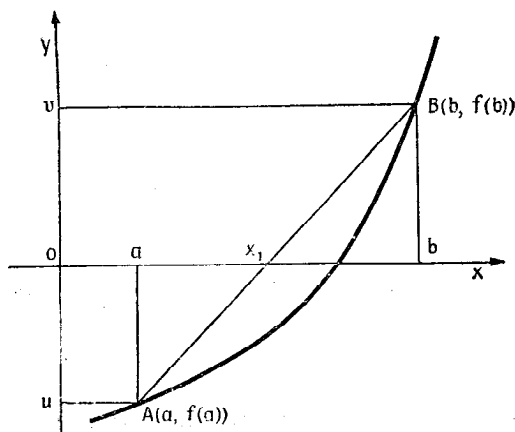


Рис. 3 15.

Метод деления пополам (или дихотомия)

Этот метод очень напоминает предыдущий, но отличается от него способом перехода от одного интервала к другому:

1. если $f(a)$ и $f(b)$ имеют разные знаки, то
2. вычислить $x = (a + b)/2$,
3. если $f(x)$ и $f(b)$ имеют разные знаки, то установить $a = x$ и перейти к 2,
4. если $f(x)$ и $f(a)$ имеют разные знаки, то установить $b = x$ и перейти к 2.

Обычно используются прежние условия остановки.

Опыт показывает, что этот метод, как правило, характеризуется более быстрой сходимостью, чем предыдущий.

Задача

1. Нарисовать блок-схему подпрограммы CORDE¹⁾, следуя изложенному алгоритму и используя два первых условия остановки (остановка при выполнении одного из условий).

¹⁾ В переводе на русский язык — «хорда». — Прим. ред.

Эта подпрограмма должна принимать входные аргументы:

- имя функции,
- границы интервала,
- EPSI } для условий остановки
- ETA }

и выдавать следующие выходные аргументы:

- найденное значение x ,
- найденное значение $f(x)$,
- значение целой переменной, указывающее на результат операции

$L = -1$ корень найден,

$= 0$ остановка по условию $|f(x)| \leq \epsilon$,

$= 1$ остановка по условию $|b - a| \leq \eta$.

2. Из подпрограммы CORDE получить подпрограмму DICHON¹⁾, в которой используется второй алгоритм (дихотомия).

Замечание

Чтобы упростить вычисление, можно положить

$$u = f(a),$$

$$v = f(b).$$

Тогда уравнение прямой запишется как

$$y - u = \frac{v - u}{b - a} (x - a)$$

и абсцисса точки пересечения прямой с осью X будет равна

$$x = a - u \frac{b - a}{v - u} = \frac{av - ub}{v - u}.$$

Написать подпрограмму на Фортране, соответствующую блок-схеме.

Найти корень уравнения $e^x - 2x - 10 = 0$ в интервале $(0, 20)$, используя подпрограмму CORDE.

3. Написать программу на Фортране, которая с помощью предыдущих подпрограмм позволяет решить уравнение

$$x^2 - 15x - 250 = 0.$$

¹⁾ Начальная часть слова *dichotomie* (дихотомия). — Прим. ред.

Решение

1. Возникает трудность: каким простым способом можно проверить, что $f(a)$ и $f(b)$ имеют разные знаки? Это можно сделать так:

- Если произведение $f(a) \times f(b)$ отрицательно, то $f(a)$ и $f(b)$ имеют разные знаки.
- Если произведение положительно, то $f(a)$ и $f(b)$ имеют одинаковые знаки.

Вообще говоря, не исключается обращение к подпрограмме, когда границы интервала (a, b) такие, что $f(a)$ и $f(b)$ имеют одинаковые знаки. Поэтому необходимо сразу же предусмотреть проверку, выясняющую, что $f(a)$ и $f(b)$ имеют одинаковые знаки, присвоить в этом случае целой переменной L значение -1 и вернуться в вызывающую компоненту программы.

Условия остановки проверяются, как показано на рис. 3.16.

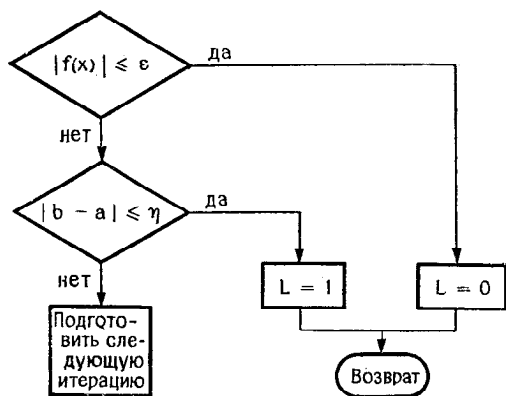


Рис. 3.16.

Теперь мы можем нарисовать предварительную блок-схему (рис. 3.17).

В этой блок-схеме:

- если при возврате $L = 0$, то интервал $[a, b]$ остается нескорректированным;
- однако проверку $|b - a| \leq \eta$ можно выполнять только после коррекции этого интервала. Поэтому следует отдать предпочтение другой блок-схеме, изображенной на рис. 3.18.

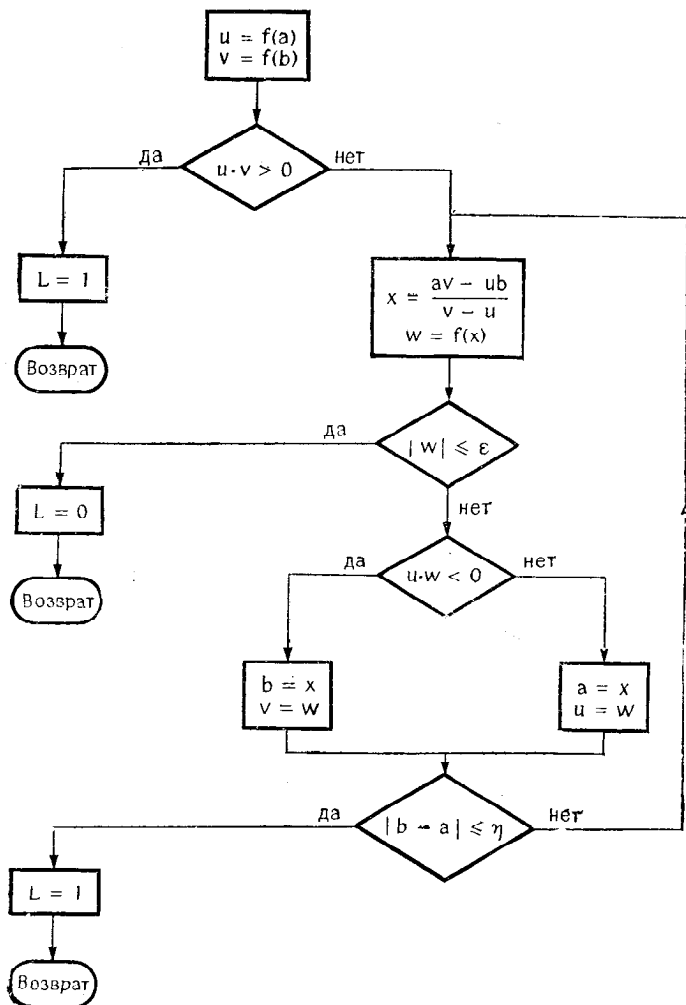


Рис. 3.17.

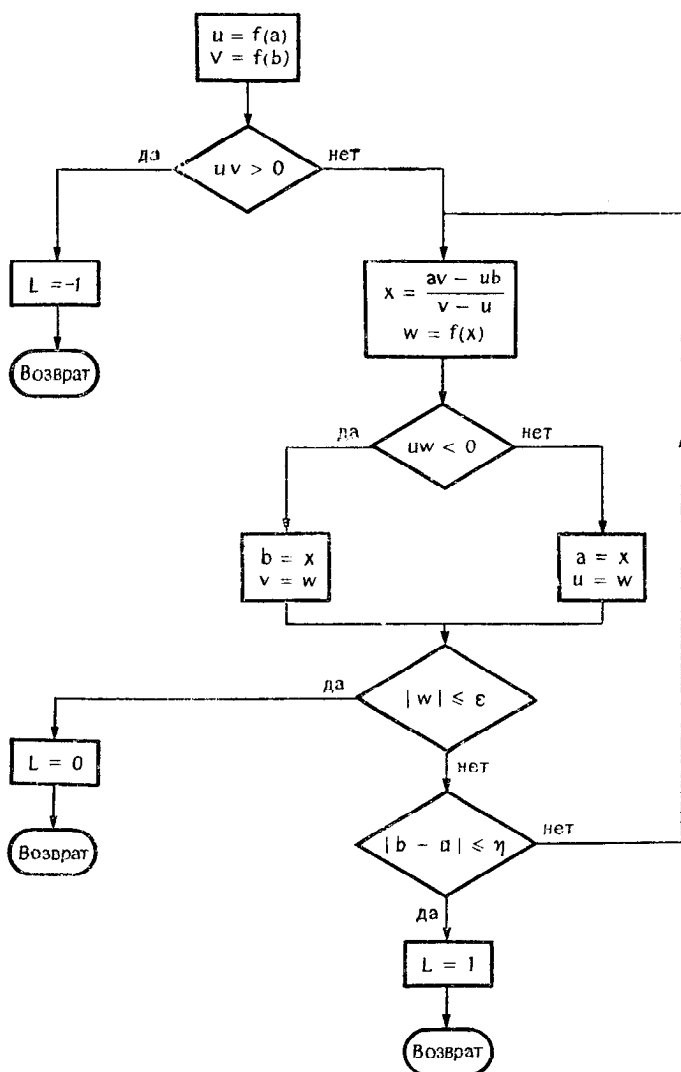


Рис. 3.18.

Программирование этой подпрограммы не представляет никаких трудностей, что и видно из следующего листинга:

```

SUBROUTINE CORDE(F,C,D,EPSI,ETA,X,L)
  A=C }
  B=D } ← Чтобы не изменять значения C и D в программе,
        }   которая вызывает подпрограмму CORDE
  U=F(A)
  V=F(B)
  IF(U*V) 20, 20, 10
10  L=-1
   RETURN
20  X=(A*V-U*B)/(V-U)
   W=F(X)
   IF(U*W) 30, 30, 40
30  B=X
   V=W
   GO TO 50
40  A=X
   U=W
50  IF(ABS(W).LE.EPSI) GO TO 60
   IF(ABS(B-A).GT.ETA) GO TO 20
   L=1
   RETURN
60  L=0
   RETURN
END

```

2. Подпрограмму DICH0 легко получить из предыдущей подпрограммы, изменяя инструкцию вычисления x (инструкция с меткой 20). Подпрограмма выглядит так:

```

SUBROUTINE DICH0(F,C,D,EPSI,ETA,X,L)
  A=C
  B=D
  U=F(A)
  V=F(B)
  IF(U*V) 20, 20, 10
10  L=-1
   RETURN
20  X=(A+B)/2. ← Обычно умножение на 0.5
                выполняется быстрее, чем
                деление на 2
   W=F(X)
   IF(U*W) 30, 30, 40
30  B=X
   V=W
   GO TO 50
40  A=X
   U=W
50  IF(ABS(W).LE.EPSI) GO TO 60
   IF(ABS(B-A).GT.ETA) GO TO 20
   L=1
   RETURN
60  L=0
   RETURN
END

```

3. Чтобы решить уравнение $f(x) = x^2 - 15x - 250$, надо обратить внимание на тот факт, что подпрограмме можно

передать имя «внешней процедуры» (FUNCTION, SUBROUTINE или внешней стандартной функции), но, как правило, нельзя передать имя функции-формулы.

Мы, следовательно, должны написать функцию

```

      FUNCTION F0NC(X)
      F0NC=(X-15.)*X-250.
      RETURN
      END
  
```

В главной программе мы напомним

```

      EXTERNAL F0NC
      EPSI=0.001
      ETA=0.05
      CALL DICH0(F0NC,0.,100.,EPSI,ETA,X,L)
      IF(L) 10,20,30
10  STOP 77
20  CONTINUE
30  WRITE(1,50)X
50  FORMAT(3H X=,1PE17.8//)
      STOP 00
      END
  
```

Решения не существует

Замечание

Можно было бы в подпрограмме присвоить L значения 0, 1 или 2 в зависимости от полученных результатов. Но в этом случае использование арифметического IF в вызывающей программе становится менее удобным, потому что тогда надо писать

IF (L — 1) 10, 20, 30.

ГЛАВА 4. УПРАЖНЕНИЯ, ОРИЕНТИРОВАННЫЕ НА ЧИСЛЕННЫЙ АНАЛИЗ

4.1. Вычисление длины кривой

Чтобы вычислить длину кривой, являющейся графиком функции $y = f(x)$, можно аппроксимировать кривую ломаной линией и вычислить длину этой линии.

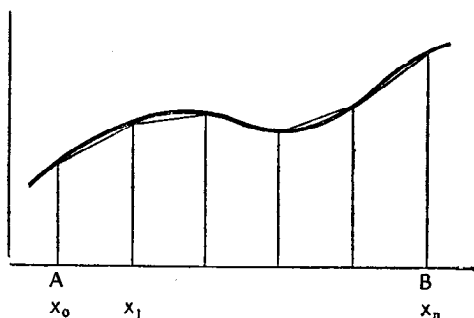


Рис. 4.1.

Ошибка метода при этом окажется в допустимых пределах, если выбраны достаточно малые интервалы.

Необходимо задать:

- имя функции $y = f(x)$,
- границы (A, B) интервала, в котором изменяется x ,
- число n интервалов, используемых в вычислении.

Задача 1

Написать программу, которая вычисляет длину кривой, соответствующей функции $y = \sqrt{X * (D - X)}$ при $D = 20$ и X , изменяющемся от 0 до 20, полагая $n = 50$.

Задача 2

- Написать подпрограмму-функцию с именем LONG, которая тем же методом, что и прежде, позволяет вычислить длину кривой $y = f(x)$. Имя функции передается как один из параметров.
- Написать главную программу, которая с помощью функции LONG вычислит длину той же кривой.

Замечание

Длина отрезка прямой, соединяющего две точки (x_i, x_{i+1}) , определяется по формуле

Длина отрезка $= \sqrt{(f(x_{i+1}) - f(x_i))^2 + h^2}$, где $h = x_{i+1} - x_i$.

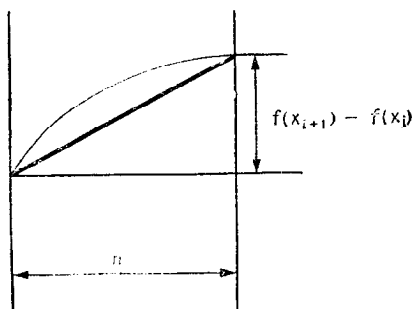


Рис. 4.2.

Решение задачи 1

Поскольку величина h^2 требуется при каждом повторении цикла, ее желательно вычислить один раз и присвоить вспомогательной переменной H2. Эта идея реализована в блок-схеме, изображенной на рис. 4.3.

В данном случае разумно использовать функцию-формулу. Таким образом мы приходим к следующей программе:

```

F(X)=SQRT(X*(D-X))
D=20.
N=50
H=D/FL0AT(N)
H2=H*H
S=0.
X=0.
U=F(X)
D0 10 I=1,N
X=X+H
V=F(X)
S=S+SQRT((V-U)**2+H2)
10 U=V
WRITE(1,20)S
20 F0RMAT(6H LONG=,1PE18.7//)
STOP
END

```

Решение задачи 2

Задача 2 является простым обобщением предыдущей задачи. Параметрами функции LONG будут:

- имя функции,

- величины A и B , характеризующие пределы изменения X ,
- число N .

Чтобы функция **LONG** была вещественной, ее нужно описать как таковую в программной компоненте, которая к ней

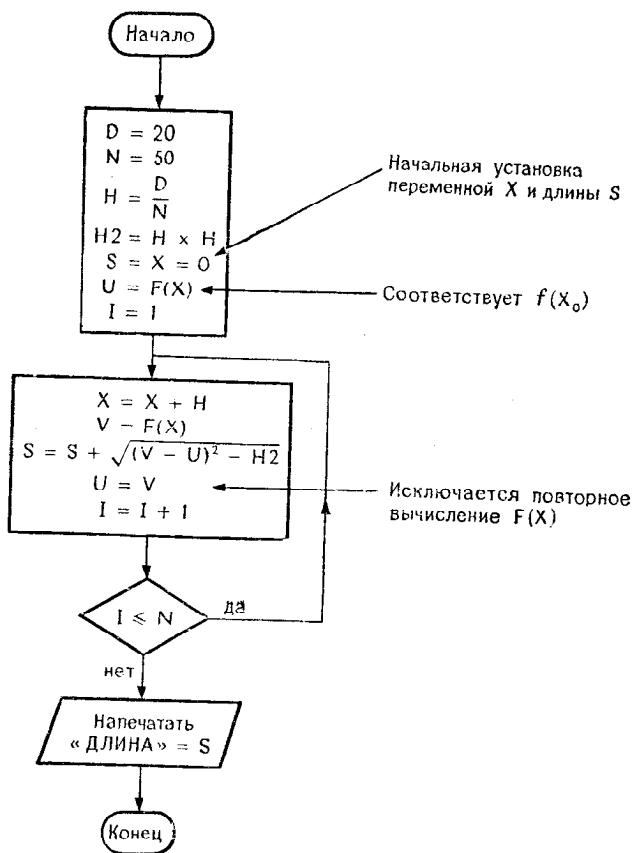


Рис 4.3.

обращается (в нашем случае в главной программе), и в декларации **FUNCTION**, поскольку главная программа и подпрограмма компилируются независимо.

Главная программа обращается к функции **LONG** с параметром, который является именем внешней функции, и, следовательно, это имя должно быть указано в декларации **EXTERNAL** в главной программе.

Итак, мы приходим к следующим листингам:

```

REAL LONG ← Эта декларация
EXTERNAL CIRC0N      необходима
DATA D,N/20.,50/
Y=L0NG(CIRC0N,0.,D,N)
WRITE(1,50)Y
STOP
50  F0RMAT(3H Y=,F10.6)
    END

REAL FUNCTION L0NG(F,A,B,N)
H=(B-A)/FLOAT(N)
H2=H*H
S=0.
X=A
U=F(X)
D0 10 I=1,N
    X=X+H
    V=F(X)
    S=S+SQRT((V-U)**2+H2)
10  U=V
    L0NG=S
RETURN
END

FUNCTION CIRC0N(X)
DATA  D/ 20./
CIRC0N=SQRT(X*(D-X))
RETURN
END

```

Результат выполнения программы:

Y = 31.364986

4.2. Вычисление интеграла методом трапеций

Условие задачи: заштрихованный участок (на рис. 4.4) можно аппроксимировать трапецией, площадь которой равна

$$S_1 = \frac{1}{2} [f(x_{i+1}) + f(x_i)] [x_{i+1} - x_i].$$

Таким образом, чтобы вычислить $\int_a^b f(x) dx$ методом трапеций, можно применить следующую формулу:

$h = (b - a)/n$, если используется n трапеций,

$$S = \left[\frac{1}{2} (f(a) + f(b)) + f(a+h) + f(a+2h) + \dots \right. \\ \left. \dots + f(a+(n-1)h) \right] h.$$

- Написать функцию с именем INTEGR, которая вычисляет определенный интеграл другой функции, заданной

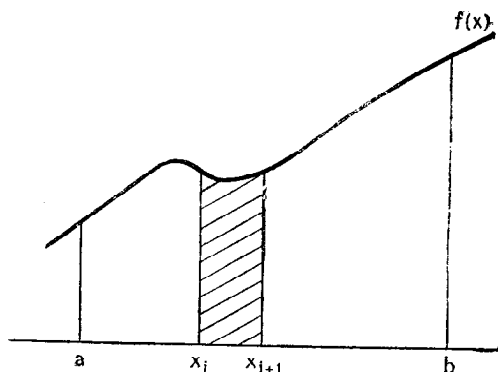


Рис. 4.4.

как параметр. Другие параметры указывают пределы интегрирования a и b , а также число шагов. Форма обращения может быть такой:

$$Y = \text{INTEGR}(Q, A, B, N),$$

где Q — имя интегрируемой функции, A, B — пределы интегрирования, N — число интервалов.

Применить эту функцию для вычисления интеграла

$$\int_{-2}^4 QUA(x) dx, \quad \text{где} \quad QUA(x) = |x^2 + 8x + 1|.$$

Результат получается как сумма площадей 20 трапеций. Нарисовать блок-схему и написать соответствующие программные компоненты.

Решение

Подпрограмму-функцию с именем INTEGR нужно описать как вещественную в самой подпрограмме и во всех программных компонентах, в которых она используется.

Вычисление S целесообразно осуществлять в 2 приема, чтобы использовать простой цикл. Эти соображения учтены в блок-схеме, изображенной на рис. 4.5.

Чтобы использовать инструкцию DO, нам придется предварительно вычислить значение $N - 1$ и присвоить его переменной $N1$.

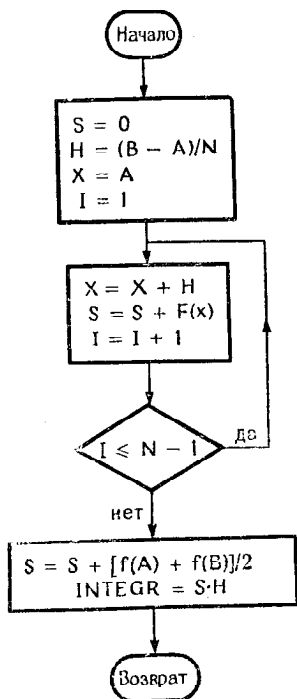


Рис 4.5.

Теперь можно подготовить листинги:

```

REAL INTEGR
EXTERNAL GAUSS
N=50
Y=INTEGR(GAUSS,-1.95,1.95,N)
WRITE(1,10)Y
10 FORMAT(3H Y=,G15.7/)
STOP
END
  
```

Пределы интегрирования. Если интеграл берется в пределах от -2 до +2, то нужно обязательно писать -2.,2., чтобы обеспечить полное соответствие между формальными и фактическими параметрами.

```

REAL FUNCTION INTEGR(F,A,B,N)
N1=N-1
H=(B-A)/FLOAT(N)
S=0.
X=A
DO 10 I=1,N1
  X=X+H
  S=S+F(X)
10 INTEGR=H*(S+0.5*(F(A)+F(B)))
RETURN
END
  
```

Запись $S=0$ также допустима

Обычно умножение на 0.5, выполняется быстрее, чем деление на 2

```

FUNCTION GAUSS(T)
DATA PI/3.1415927/
COEF=1./SQRT(2.*PI)
GAUSS=COEF*EXP(-T*T*0.5)
RETURN
END

```

Результат выполнения программы:

Y= 0.9467044

Программа, вычисляющая функцию GAUSS(T), совершенно корректна с точки зрения синтаксиса. Однако к ней можно предъявить претензию, поскольку величина COEF должна вычисляться при каждом вызове функции, хотя вычисление можно было бы выполнить только один раз в главной программе, если бы переменная COEF находилась в общей (COMMON) области. Это замечание вынуждает нас внести небольшие изменения в программы:

```

COMMON COEF
REAL INTEGR
EXTERNAL GAUSS
N=50
COEF=1./SQRT(2.*3.1415927)
Y=INTEGR(GAUSS,-1.95,1.95,N)
WRITE(1,10)Y
10 FORMAT(3H Y=,G15.7//)
STOP
END

REAL FUNCTION INTEGR(F,A,B,N)
N1=N-1
H=(B-A)/FLOAT(N)
S=0.
X=A
DO 100 I=1,N1
  X=X+H
10  S=S+F(X)
INTEGR=H*(S+0.5*(F(A)+F(B)))
RETURN
END

FUNCTION GAUSS(T)
COMMON COEF
GAUSS=COEF*EXP(-T*T*0.5)
RETURN
END

```

4.3. Вычисление определенного интеграла методом Симпсона

Для вычисления определенного интеграла методом Симпсона используется следующая формула:

$$S = \frac{h}{3} (y_0 - 4y_1 + 2y_2 + 4y_3 + \dots + 4y_{n-1} + y_n),$$

где n — четное число.

Задача

Написать функцию SIMPSO, которая вычисляет определенный интеграл методом Симпсона и параметрами которой являются:

- имя интегрируемой функции,
- пределы интегрирования,
- число интервалов N .

Эта функция должна проверять, что N — четное число, и, если проверка не удовлетворяется, вызывать остановку по инструкции STOP 111.

Решение

Объяснения, приведенные в связи с методом трапеций, в целом приемлемы и здесь. В деталях же есть отличия, потому что не удается организовать столь простое суммирование, как прежде.

Предположим, что

$$S_1 = \sum_{p=0}^{\frac{n}{2}-1} y_{2p+1} \quad \text{соответствует } n, \text{ изменяющемуся от } 1 \text{ до } n-3, \text{ с шагом } 2,$$

$$S_2 = \sum_{p=1}^{\frac{n}{2}-1} y_{2p} \quad \text{соответствует } n, \text{ изменяющемуся от } 2 \text{ до } n-2, \text{ с шагом } 2.$$

Следовательно, S_1 содержит все члены с коэффициентом 4 от y_1 до y_{n-3} , за исключением y_{n-1} .

Суммы S_1 и S_2 можно вычислить в одном DO-цикле. Эти соображения учтены в блок-схеме (рис. 4.6).

Остается проверить четность N , что легко делается с помощью логического IF:

IF (MOD (N, 2).NE.0.) STOP 111

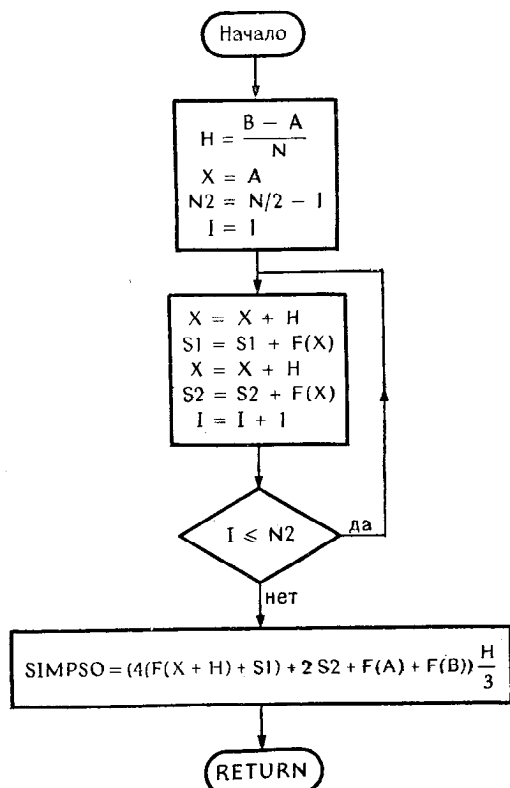


Рис. 4.6.

Получаем следующий листинг:

```

FUNCTION SIMP0(F,A,B,N)
  IF(MOD(N,2).NE.0)STOP 111
  N2=N/2 -1
  H=(B-A)/FLOAT(N)
  S1=0.
  S2=0.
  X=A
  DO 10 I=1,N2
    X=X+H
    S1=S1+F(X)
    X=X+H
    S2=S2+F(X)
10  SIMP0=(4.*(S1+F(X+H)) +2.*S2+F(A)+F(B))*H/3.
  RETURN
END

GAUSS(T)=COEF*EXP(-T*T/2.) ← Будьте внимательны!
N=50                               Как правило, нельзя использовать
PI=3.1415926                       идентификатор функции-формулы
COEF=1./SQRT(2.*PI)                в качестве параметра подпрограммы
Y=SIMP0(GAUSS,-1.95,1.95,N)
WRITE(1,10)Y
10  FORMAT(3H Y=,G15.7//)
STOP
END

```

Результат выполнения программы:

Y= 0.9488231 ← Точный результат равен 0.95,
т.е. ошибка составляет 0.0012

4.4. Вычисление определенного интеграла методом Ветдлля

В этом методе, обладающем более высокой точностью, чем предыдущие, используется следующая формула (число интервалов обязательно кратно 6):

$$\int_a^b f(x) dx = \frac{3h}{10} [f(a) + 5f(a+h) + f(a+2h) + 6f(a+3h) + f(a+4h) + 5f(a+5h) + 2f(a+6h) + \dots + f(b)]$$

Коэффициент=1 при достижении f(b)

Этот метод, очевидно, применим и в частном случае, когда $n = 6$.

Задача

Написать функцию WEDDLE, которая вычисляет определенный интеграл, используя приведенную выше формулу.

Решение

Параметры, очевидно, те же, что и параметры, используемые для предыдущих функций. Исходя из предыдущих блок-схем, можно легко получить следующую программу:

```

FUNCTION WEDDLE(F,A,B,N)
INTEGER P
IF (MOD(N,6).NE.0) STOP 111
P=N/6
X=A
H=(B-A)/FLOAT(N)
S1=0.
S2=0.
S3=0.
S4=0.
S5=0.
S6=0.
DO 10 I=1,P
  S1=S1+F(X+H)
  S2=S2+F(X+2.*H)
  S3=S3+F(X+3.*H)
  S4=S4+F(X+4.*H)
  S5=S5+F(X+5.*H)
  S6=S6+F(X+6.*H)
10  X=X+6.*H
WEDDLE=.3*H*(F(A)+5.*S1+S2+6.*S3+S4+5.*S5+2.*S6-F(B))
RETURN
END

```

↑
Поскольку F(B) уже вошло в S6

```

GAUSS(T)=COEF*EXP(-T*T/2.)
N=6
PI=3.1415926
COEF=1./SQRT(2.*PI)
Y=WEDDLE(GAUSS,-1.95,1.95,N)
WRITE(1,10)Y
10  FORMAT(3H Y=,G15.7//)
STOP
END

```

Результаты выполнения программы:

$Y = 0.9501319$ при $N = 6$

$Y = 0.9488267$ при $N = 12$

Эти результаты демонстрируют как достоинство формулы Веддля (высокая точность метода), так и ее чувствительность к ошибкам округления.

4.5. Решение системы линейных уравнений методом Гаусса

Этот метод распадается на две фазы. В первой фазе матрица линейной системы приводится к треугольному виду (выполняется триангуляция), затем во второй фазе решается треугольная линейная система.

Чтобы перейти от начальной системы к треугольной, необходимо исключить (сделать нулевыми) все элементы под главной диагональю, действуя следующим образом:

- 1) установить $i = 1$;
- 2) если $a_{ii} \neq 0$, то с помощью линейной комбинации строки i и строк $i + 1, i + 2, \dots, n$ можно исключить элементы $a_{i+1,i}, a_{i+2,i}, \dots, a_{n,i}$; затем установить $i = i + 1$;
если $i \leq n - 1$, перейти к 2;
если $i > n - 1$, перейти к 4;
- 3) если $a_{ii} = 0$, нужно найти среди следующих строк элемент $a_{hi} \neq 0$ и поменять местами строки i и h , затем вернуться к 2;
- 4) если $a_{nn} = 0$, то матрица вырожденная и необходимо прекратить решение;
если $a_{nn} \neq 0$, то триангуляция закончена и можно переходить к следующей фазе.

Пример

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 4 & 3 \\ -1 & 2 & 4 & -6 \\ 2 & -3 & -2 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -3 \\ 1 \\ 17 \\ -9 \end{pmatrix}.$$

Комбинируя первую строку со всеми последующими, исключим элементы первого столбца:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 0 & 1 & -1 \\ 0 & 4 & 7 & -2 \\ 0 & -7 & -8 & -3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -3 \\ 4 \\ 14 \\ -3 \end{pmatrix}.$$

Теперь, поскольку $a_{22} = 0$ и $a_{32} \neq 0$, меняем местами строки 2 и 3:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 4 & 7 & -2 \\ 0 & 0 & 1 & -1 \\ 0 & -7 & -8 & -3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -3 \\ 14 \\ 4 \\ -3 \end{pmatrix}.$$

Теперь можно использовать строку 2 для исключения элемента в строке 4 (строка 3 в этом случае бесполезна, так как

$a_{32} = 0$):

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 4 & 7 & -2 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & \frac{17}{4} & -\frac{26}{4} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -3 \\ 14 \\ 4 \\ \frac{86}{4} \end{pmatrix}.$$

И наконец, получаем

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 4 & 7 & -2 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & -\frac{9}{4} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -3 \\ 14 \\ 4 \\ \frac{18}{4} \end{pmatrix}.$$

Остается решить эту линейную систему.

Решение треугольной линейной системы начинается снизу: решая последнее уравнение $\vec{T}\vec{X} = \vec{C}$, получим $x_n = C_n/t_{nn}$; в нашем примере

$$x_4 = -\frac{18/4}{9/4} = -2.$$

Подставляя x_n в уравнение $n-1$, получим значение x_{n-1} . Процесс продолжается до тех пор, пока не будут получены все искомые величины.

Задача

1. Нарисовать блок-схему подпрограммы, позволяющей решить линейную систему методом Гаусса. Форма обращения к подпрограмме такова:

CALL GAUSS (A, B, X, N, D, RES)

A = массив размером $N \times N$;

B = массив размером N;

X = массив размером N;

N = целая переменная, указывающая размер массивов;

D = вещественная переменная, через которую передается значение определителя матрицы;

RES = логическая переменная, характеризующая результат операции при возврате в главную программу:

RES : FALSE — матрица вырожденная,

RES : TRUE — решение получено.

Рисовать блок-схему нужно в несколько этапов, опуская на первых порах некоторые детали.

2. Написать подпрограмму на Фортране, соответствующую блок-схеме.

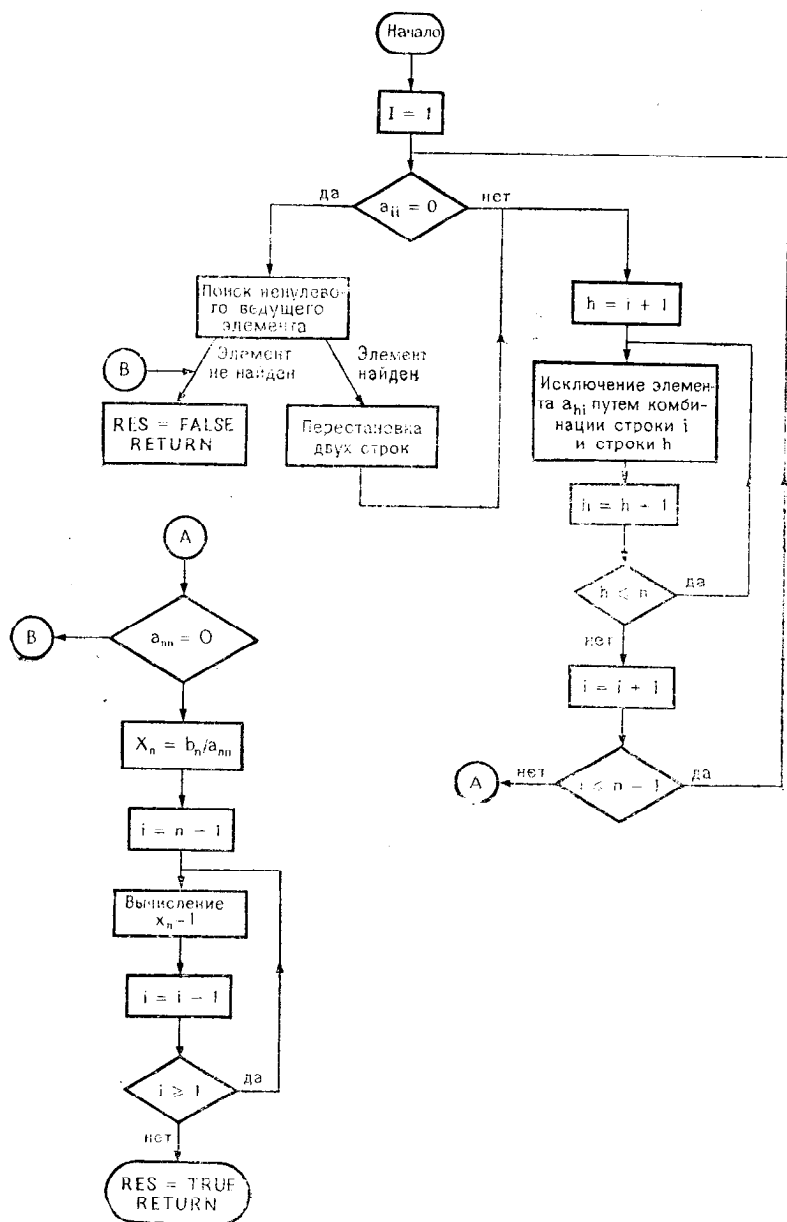


Рис. 4.

Решение

1. Вначале желательно нарисовать не очень подробную блок-схему, примерно такую, как на рис. 4.7.

Теперь нужно уточнить, как строится линейная комбинация строки i и строки h . Для этого вычисляется $R = a_{hi}/a_{ii}$, затем при j , меняющемся от $i+1$ до h , выполняются присваивания

$$a_{hj} = a_{hj} - R \cdot a_{ij}$$

и, наконец, $b_h = b_h - Rb_i$.

Чтобы вычислить x_i , в уравнение надо подставить значения x_{i+1} , x_{i+2} , ..., x_i . Это соображение можно выразить с помощью блок-схемы (рис. 4.8).

```

SUBROUTINE GAUSS(A,B,X,N,DET,RES)
REAL A(N,N),B(N),X(N)
LOGICAL RES
INTEGER N
RES=.FALSE.
N1=N-1
DO 100 I=1,N1
  H=I+1
  IF(A(I,I).NE.0.)GO TO 50
C ПОИСК НЕНУЛЕВОГО ВЕДУЩЕГО ЭЛЕМЕНТА
  DO 10 K=H,N
    IF(A(K,I).NE.0.)GO TO 30
  10 CONTINUE
C ЭЛЕМЕНТ НЕ НАЙДЕН
  GO TO 135
C ПЕРЕСТАНОВКА ДВУХ СТРОК
  30 DO 40 J=I,N
    V=A(I,J)
    A(I,J)=A(H,J)
    A(H,J)=V
  40 V=B(I)
    B(I)=B(H)
    B(H)=V
C ИСКЛЮЧЕНИЕ ЭЛЕМЕНТА A(H,I)
  50 IF(A(H,I).EQ.0.) GO TO 80
    R=-A(H,I)/A(I,I)
    DO 60 J=I+1,N
      A(H,J)=A(H,J)+R*A(I,J)
    60 B(H)=B(H)+R*B(I)
  80 H=H+1
  IF(H.LE.N)GO TO 50
  100 CONTINUE
  IF(A(N,N).EQ.0.)GO TO 135*
C РЕШЕНИЕ ТРЕУГОЛЬНОЙ СИСТЕМЫ
  X(N)=B(N)/A(N,N)
  I=N1
  110 V=0.
    I1=I+1
    DO 120 H=I1,N
      V=V+A(I,H)*X(H)
    120 X(I)=(B(I)-V)/A(I,I)
    I=I-1
    IF(I.GE.1)GO TO 110*
  130 D=1
    DO 130 I=1,N
      D=D*A(I,I)
  RES=.TRUE.
  RETURN
END

```

```

REAL A(5,5),AT(5,5),B(5),X(5),Y(5)
LOGICAL RES
DATA N/5/
DO 20 I=1,N
  DO 10 J=1,N
    A(I,J)=1./FLOAT(I+J)
  10 A(I,J)=A(I,J)*I.
  20 X(I)=I
    DO 30 J=1,N
      B(J)=0.
    30 B(J)=B(I)+A(I,J)*X(J)
  CALL GAUSS(A,B,Y,N,DET,RES)
  IF(.NOT. RES) STOP 77
  WRITE(1,100)A,B,X,Y
  FORMAT(5(1SF13.7))
  STOP 00
END

```

Порождение A и X

Вычисление B

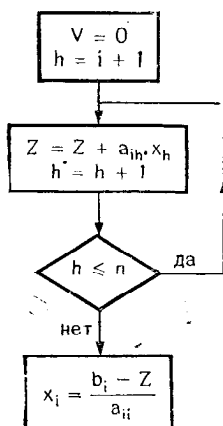


Рис. 4.8.

Теперь можно сделать подробную блок-схему (рис. 4.9 и рис. 4.9 bis).

Результат выполнения программы:

Матрица транспонированная	1.5000000	0.0000000	0.0000000	0.0000000	0.0000000
	0.3333333	1.1759257	0.0000000	0.0000000	0.0000000
	0.2500000	0.1444444	1.1072569	0.0000000	0.0000000
	0.2000000	0.1222222	0.0945107	1.0775626	0.0000000
	0.1666667	0.1058201	0.0842238	0.0707013	1.0609131
B	4.5499992	3.8031731	4.1209316	4.6637564	5.3045645
x	1.0000000	2.0000000	3.0000000	4.0000000	5.0000000
y	1.0000000	1.9999995	2.9999990	3.9999990	4.9999990
STOP NUMBER 00					

4.6. Интегрирование дифференциального уравнения методом Рунге — Кутты

При интегрировании дифференциального уравнения, представленного в каноническом виде $y' = f(x, y)$, при помощи классического метода Рунге — Кутты четвертого порядка выполняется переход от точки (x_i, y_i) к точке (x_{i+1}, y_{i+1}) с применением следующих формул:

$$T_1 = h \cdot f(x_i, y_i),$$

$$T_2 = h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{T_1}{2}\right),$$

$$T_3 = h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{T_2}{2}\right),$$

$$T_4 = h \cdot f(x_i + h, y_i + T_3),$$

$$y_{i+1} = y_i + (T_1 + 2T_2 + 2T_3 + T_4)/6,$$

где

$$h = x_{i+1} - x_i.$$

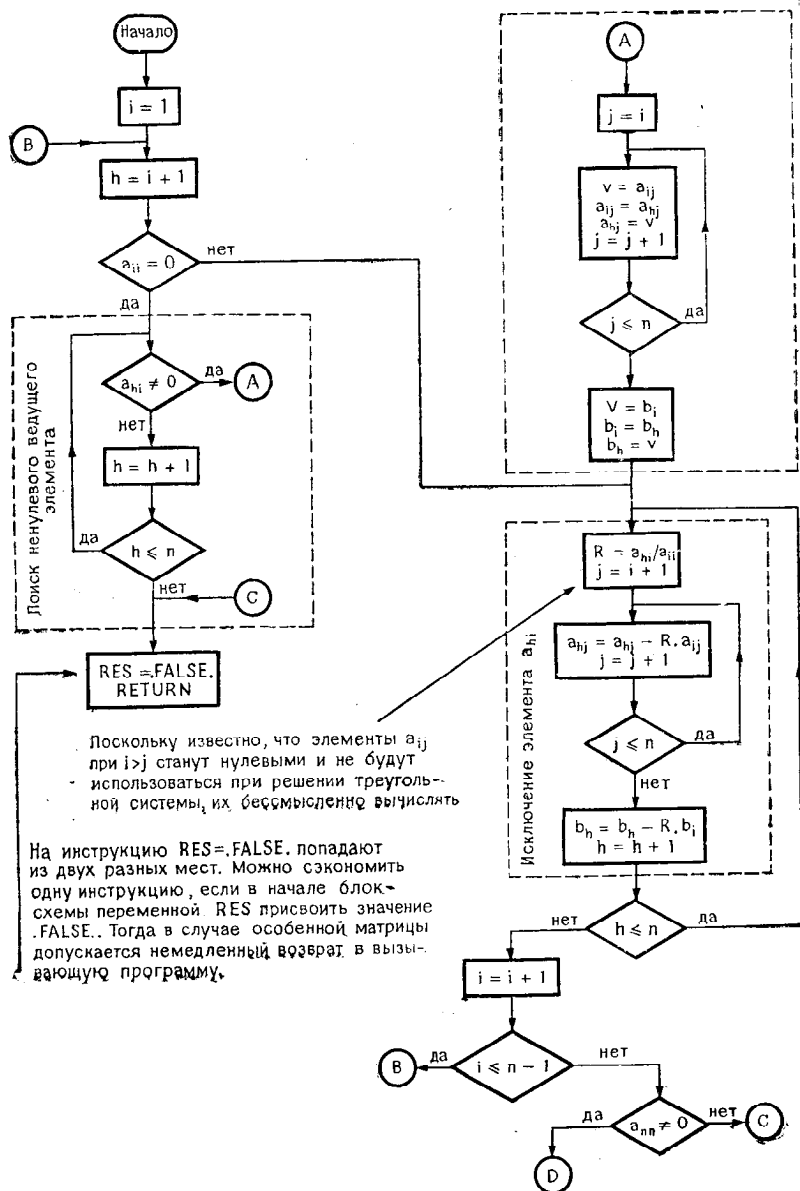


Рис. 4.9.

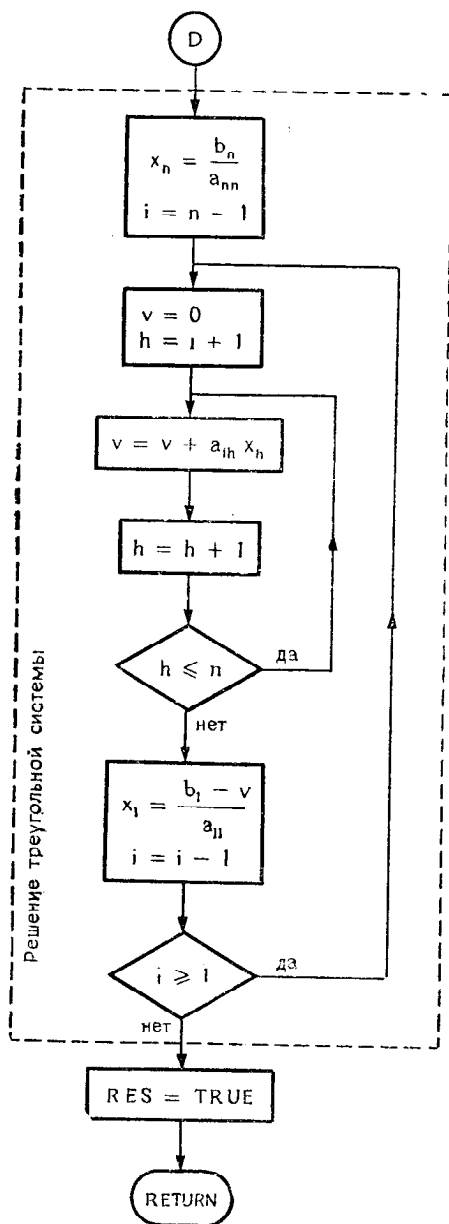


Рис. 4.9 bis.

В таком пошаговом методе предполагается, что известна начальная точка (x_0, y_0) , соответствующая «начальному условию».

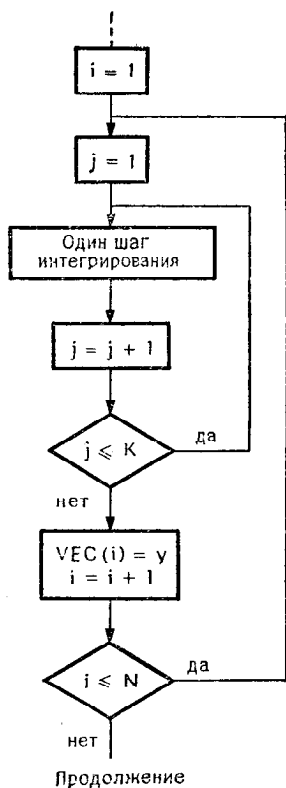


Рис. 4.10.

Задача

1. Написать подпрограмму RK4, которая осуществляет такое интегрирование и которая вызывается следующим образом:

CALL RK4 (F0NC, H, XI, YI, K, N, VEC),

где

F0NC = имя функции $f(x, y)$;

H = шаг интегрирования;

$\left. \begin{matrix} XI \\ YI \end{matrix} \right\}$ = координаты начальной точки (x_0, y_0) ;

K = число вычислительных шагов между двумя величинами, сохраняемыми в памяти;

N = общее число сохраняемых величин (выполняется в сумме $N \times K$ шагов);

VEC = массив, в котором подпрограмма RK4 должна сохранить вычисленные результаты $y(i)$.

2. Использовать эту подпрограмму для решения дифференциального уравнения $y' = 2e^{-x} \cos 2x - y$ с начальным условием ($x_0 = 0, y_0 = 0$). Это уравнение имеет своим решением функцию $y = e^{-x} \sin 2x$. Следовательно, можно сравнить результаты, полученные с помощью подпрограммы RK4, с фактическими значениями. Выберем шаг $h = 0.1$, $K = 10$ и $N = 10$.

Решение 1

Подпрограмма RK4 должна быть приемлема для решения любого дифференциального уравнения при условии, что функция $f(x, y)$ непрерывна и ограничена. Программирование формул не представляет никакого труда. Остается еще вопрос организации вычислений, который можно решить так, как показано на фрагменте блок-схемы (рис. 4.10).

И тогда мы можем полностью написать программу:

```

REAL VEC(100)
EXTERNAL DERIV
DATA IMP,N,K,XI,YI/I,10,10,0.,0./
READ(2)H
CALL RK4(DERIV,H,XI,YI,K,N,VEC)
DO 10 I=1,N
  J=K*I
  X=XI+FLOAT(J)*H
  Z=EXP(-X)*SIN(2.*X)
  E=Z-VEC(I)
10  WRITE(IMP,20)X,VEC(I),Z,E
20  FORMAT(F7.3,3(1PE18.8))
STOP
END

```

```

SUBROUTINE RK4(F0NC,H,XI,YI,K,N,VEC)
REAL VEC(N)
H2=0.5*H
Y=YI
X=XI
DO 2 I=1,N
  DO 1 J=1,K
    T1=H*F0NC(X,Y)
    T2=H*F0NC(X+H2,Y+0.5*T1)
    T3=H*F0NC(X+H2,Y+0.5*T2)
    T4=H*F0NC(X+H,Y+T3)
    Y=Y+(T1+2.*(T2+T3)+T4)/6.
  1 X=X+H
  2 VEC(I)=Y
RETURN
END

```

```

FUNCTION DERIV(T,Y)
DERIV=2.*EXP(-T)*COS(2.*T)-Y
RETURN
END

```

10.1

1.000	3.34510803E-01	3.34511936E-01	1.13248825E-06
2.000	-1.02422580E-01	-1.02422014E-01	5.66244125E-07
3.000	-1.39118433E-02	-1.39113776E-02	4.65661287E-07
4.000	1.81207284E-02	1.81207508E-02	2.23517418E-08
5.000	-3.66561208E-03	-3.66556505E-03	4.70317900E-08
6.000	-1.33014959E-03	-1.33004109E-03	1.08499080E-07
7.000	9.03325970E-04	9.03318170E-04	-7.79982656E-09
8.000	-9.65705549E-05	-9.65793297E-05	-8.77480488E-09
9.000	-9.26914508E-05	-9.26801440E-05	1.13068381E-08
10.000	4.14484166E-05	4.14476235E-05	-7.93079380E-10

ГЛАВА 5. УПРАЖНЕНИЯ ПО СТАТИСТИКЕ

5.1. Среднее, дисперсия, среднее квадратичное отклонение

Если имеется n измерений X_i , то можно сделать их оценку, вычислив среднее (математическое ожидание) по формуле

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i,$$

дисперсию X_i по формуле

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2,$$

среднее квадратичное отклонение по формуле

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2}.$$

Задача

Написать подпрограмму, входными аргументами которой будут массив X и его размер n , а выходными аргументами будут среднее значение, дисперсия и среднее квадратичное отклонение. Следовательно, можно принять такую форму вызова этой подпрограммы:

CALL MVET(X, n, M, V, E),

где X — массив,

n — размер массива,

M — среднее (тип вещественный),

V — дисперсия,

E — среднее квадратичное отклонение.

Решение

Построим сначала блок-схему (рис. 5.1) так, как подсказывают приведенные выше формулы.

Не приходится сомневаться, что можно написать подпрограмму, которая полностью соответствует этой блок-схеме и которая даст правильные результаты. Между тем существует

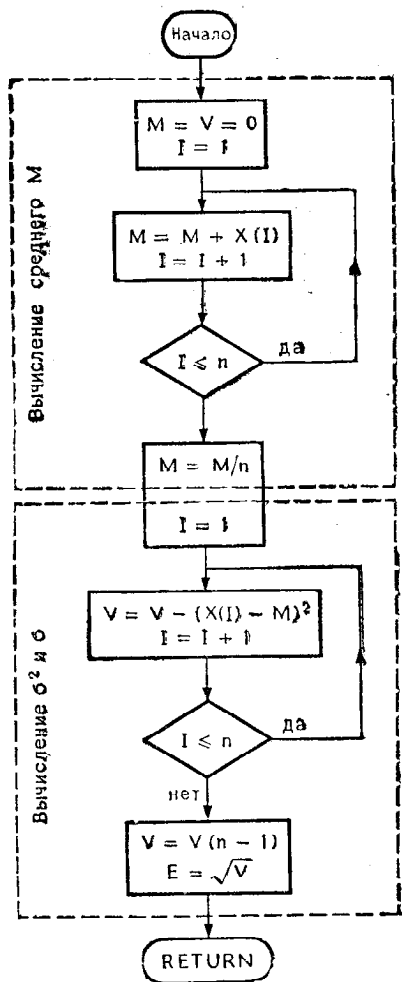


Рис. 5.1.

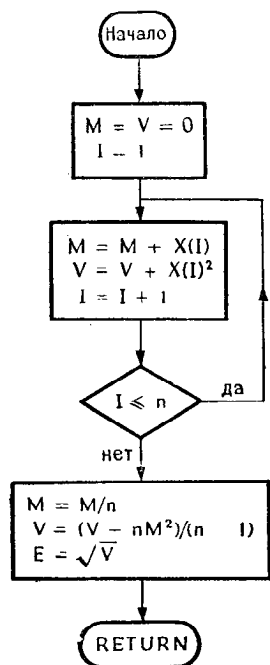


Рис. 5.2.

более быстрый метод, который основывается на ином способе вычисления σ^2 :

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i^2 - 2X_i\bar{X} + \bar{X}^2),$$

$$\sigma^2 = \frac{1}{n-1} \left[\sum_{i=1}^n X_i^2 - \underbrace{2\bar{X} \sum_{i=1}^n X_i}_{-2n\bar{X}^2} + \underbrace{\sum_{i=1}^n \bar{X}^2}_{n\bar{X}^2} \right],$$

$$\sigma^2 = \frac{1}{n-1} \left[\sum_{i=1}^n X_i^2 - n\bar{X}^2 \right].$$

Последняя формула позволяет исключить один цикл, поскольку $\sum X_i^2$ и $\sum X_i$ можно вычислить одновременно (блок-схема на рис. 5.2).

Теперь мы можем написать такую подпрограмму:

```

SUBROUTINE MVET(X,N,M,V,E)
REAL X(N),M ← Декларация необходима, чтобы
M=0.          переменная M не рассматривалась
V=0.          как целочисленная
DO 10 I=1,N
  M=M+X(I)
10  V=V+X(I)**2
  R=FLOAT(N) ← Чтобы исключить многократные
               преобразования
  M=M/R        Можно также написать R=N
  V=(V-R*M*M)/(R-1.)
  E=SQRT(V)
RETURN
END

```

Замечание

Программа будет выполняться быстрее, если в нее внести небольшие изменения:

```

SUBROUTINE MVET(X,N,M,V,E)
REAL X(N),M
A=0.
R=0.
DO 10 I=1,N
  C=X(I) ← Благодаря переменной C уменьшаются
           расходы на вычисление индекса
  A=A+C
  B=B+C*C } ← Для доступа к M и V требуется
              косвенная адресация. Поэтому благодаря
              переменным B и C уменьшается время
              работы программы
R=FLOAT(N)
M=A/R
V=(B-R*M*M)/(R-1.)
E=SQRT(V)
RETURN
END

```

Таким образом сокращается время на вычисление индекса, а также уменьшается количество обращений к аргументам M и V , значения которых вызываются по косвенному адресу с некоторой потерей времени.

5.2. Простая линейная регрессия

В результате эксперимента получены величины (X_i, Y_i) , которые, как предполагается, связаны отношением $Y = aX + b$.

Учитывая ошибки измерения, определить a и b таким образом, чтобы прямая $y = ax + b$ была самой близкой к точкам (X_i, Y_i) в смысле «наименьших квадратов».

В соответствии с критерием наименьших квадратов a и b выбираются так, чтобы выражение

$$\sum_{i=1}^n (y_i - ax_i - b)^2$$

было минимальным. Дифференцируя это выражение по a и по b , получим следующую систему уравнений:

$$\begin{cases} \sum_{i=1}^n (y_i - ax_i - b) = 0 & \text{производная по } b, \\ \sum_{i=1}^n x_i (y_i - ax_i - b) = 0 & \text{производная по } a. \end{cases}$$

Эту систему можно записать иначе:

$$\begin{cases} (\sum x_i) a + nb = \sum y_i, \\ (\sum x_i^2) a + (\sum x_i) b = \sum x_i y_i. \end{cases}$$

Чтобы решить эту систему, можно воспользоваться подпрограммой GAUSS, рассмотренной ранее, или же можно непосредственно решить эту систему. Тогда найдем

$$a = \frac{n(\sum x_i y_i) - (\sum x_i)(\sum y_i)}{n(\sum x_i^2) - (\sum x_i)^2},$$

$$b = \frac{n(\sum x_i y_i) - a \sum x_i}{n(\sum x_i^2) - (\sum x_i)^2}.$$

Известно также, что решение относительно b можно представить в виде

$$b = \bar{Y} - a\bar{X},$$

где через $\bar{Y}$ и $\bar{X}$ обозначены средние значения Y_i и X_i :

$$\bar{Y} = \frac{\sum y_i}{n} \quad \text{и} \quad \bar{X} = \frac{\sum \bar{x}_i}{n}.$$

5.2.1. Задача 1

Нарисовать блок-схему и написать подпрограмму REGLIN, позволяющую осуществить линейную регрессию.

Входными аргументами являются массивы X и Y и размер массивов N.

Выходными аргументами являются значения A и B

5.2.2. Задача 2

С точки зрения статистики недостаточно определить a и b, используя метод наименьших квадратов. Необходимо также убедиться в статистической достоверности результатов. С этой целью вычисляется коэффициент корреляции r таким образом¹⁾:

$$r = \text{sign } a \cdot \sqrt{\frac{\sum (Y_{\text{вычисленное}} - \bar{Y})^2}{\sum (Y_i - \bar{Y})^2}}.$$

Написать подпрограмму REGLIN с дополнительным параметром, соответствующим r.

5.2.3. Решение задачи 1

Прежде чем решать систему уравнений необходимо вычислить ее коэффициенты. Совершенно ясно, что в этом простом примере не следует использовать подпрограмму GAUSS. Поэтому блок-схема будет такой, как на рис. 5.3.

Соответствующую программу легко получить из приведенного ниже листинга, если исключить инструкции, относящиеся к вычислению g.

5.2.4. Решение задачи 2

Нельзя вычислить $\sum (i_{\text{вычисл}} - \bar{Y})^2$, пока не найдены коэффициенты A и B. Поэтому в предыдущую блок-схему необходимо добавить еще один цикл (рис. 5.4).

Если для вычисления $\sum (Y_i - YB)^2$ воспользоваться предыдущим циклом, то получится окончательная блок-схема (рис. 5.5).

¹⁾ Эта формула годится для частного случая простой линейной регрессии. Если интерес представляет корреляция двух из n переменных, то необходимо использовать другой метод.

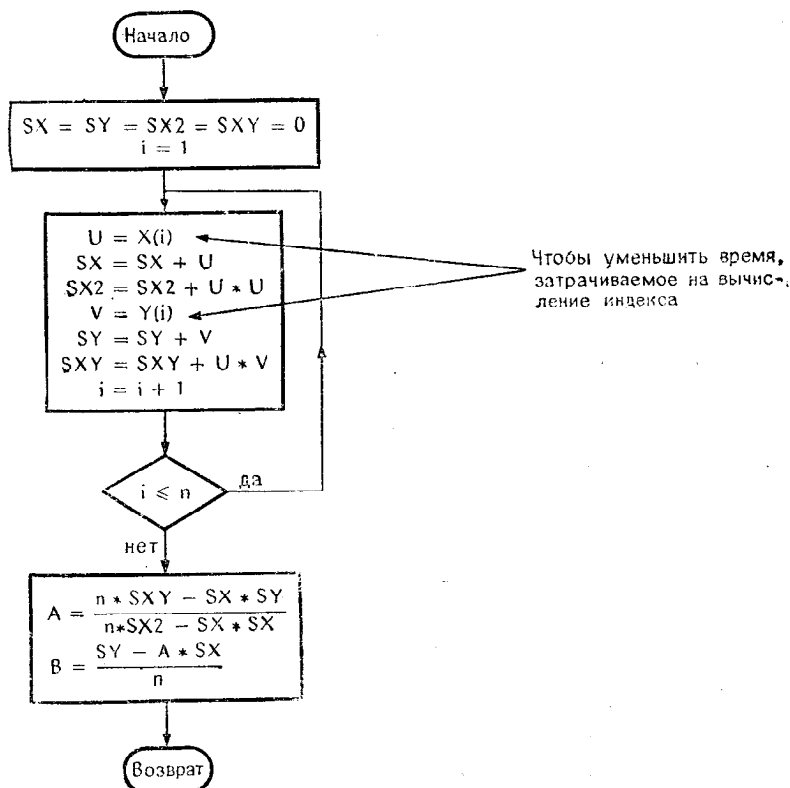


Рис. 5.3.

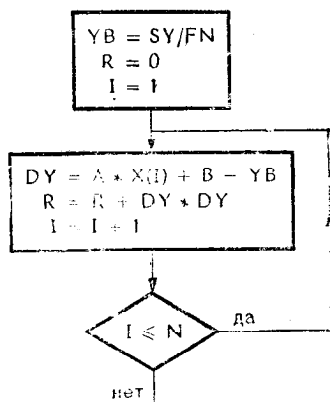


Рис. 5.4.

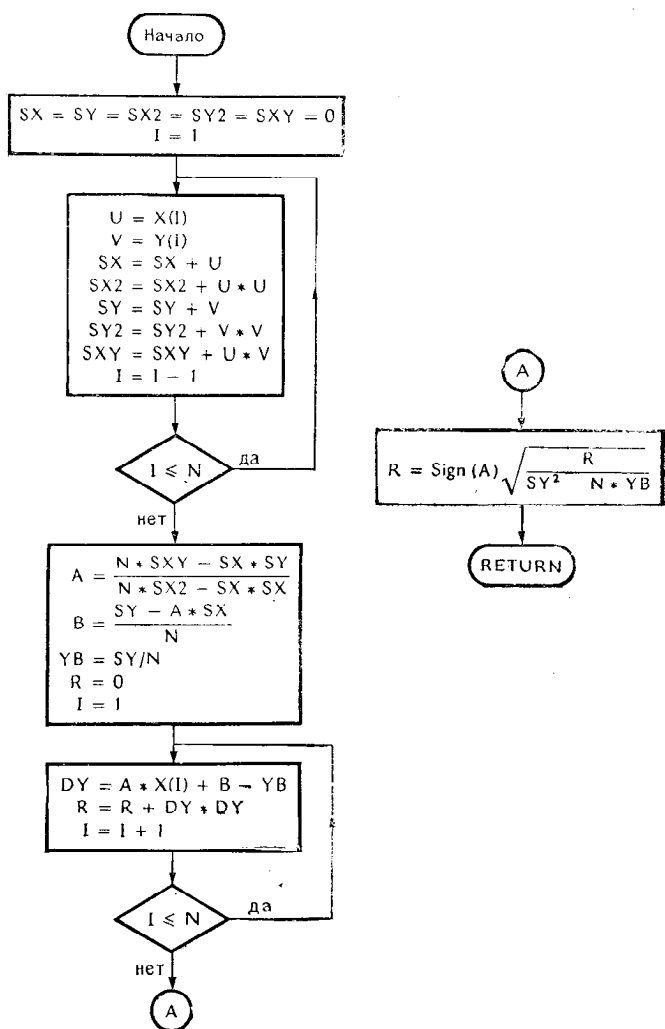


Рис. 5.5.

После этого не представляет никакого труда написать следующую программу:

	SUBROUTINE REGLIN(X,Y,N,A,B,R)	
	REAL X(N),Y(N)	
	SX=0.	
	SY=0.	
	SX2=0.	
	SXY=0.	
	SY2=0.	
	DO 10 I=1,N.	
	U=X(I)	
	SX=SX+U	
	SX2=SX2+U*U.	
	V=Y(I)	
	SY=SY+V	
	SY2=SY2+V*V	
10	SXY=SXY+V*U	
	FN=FLOAT(N)	
	YB=SY/FN	
	A=(SXY*FN-SX*SY)/(FN*SX2-SX*SX)	
	B=(SY-A*SX)/FN	
	R=0.	
	T=0.	
	DO 20 I=1,N	
	YC=A*(I)+B	
	R=R+(YC-YB)**2	
20	T=T+(Y(I)-YC)**2	
	R=SIGN(SQRT(R/(R+T)),A)	
	RETURN	
	END	
	REAL A(20),B(20)	
	READ(4,N)	
	READ(4)((A(I),B(I)),I=1,N)	
	CALL REGLIN(A,B,N,P,Q,COR)	
	WRITE(1,30)P,Q,COR	
30	FORMAT(3(1PE13.10)/)	
	STOP	
	END	

Данные

14	← N
10, 21.2	} Пары A _i , B _i
10, 19.9	
11, 22.5	
11, 23.7	
12, 25.0	
15, 30.3	
17, 36.1	
19, 38.6	
20, 41.3	
20, 42.7	
23, 45.0	
25, 50.0	
27, 50.2	
30, 62.1	

} Специфический ввод в систему,
где доп. указан свободный формат

Результаты выполнения программы:

```
1.9946246147E 00  9.8833056541E-01  9.9692369186E-01
STOP NUMBER 00
```

5.3. Анализ генератора случайных чисел

В системе имеется функция HASARD(A, B), которая дает случайное число в интервале [A, B] в соответствии с заданным законом распределения. Можно разделить этот интервал [A, B] на p меньших интервалов одинаковой длины и найти, как случайные числа распределяются в этих p интервалах.

Существуют более совершенные статистические тесты для проверки качества генератора¹⁾.

¹⁾ Информацию о них можно найти в книге Д. Кнута «Искусство программирования для ЭВМ», т. 2, М., «Мир», 1977 г., § 3.3. — Прим. ред.

Рассмотрим конкретный пример: $\begin{cases} A = 0, \\ B = 1, p = 20. \end{cases}$
 Генерируется $N = 10\,000$ чисел.

Задача 1

Нарисовать блок-схему.

Написать соответствующую программу на Фортране.

Задача 2

В дополнение к предыдущей задаче требуется начертить гистограмму в предположении, что в одной строке печатающего устройства помещаются 72 литеры.

Решение задачи 1

Нам предстоит преодолеть всего лишь одно затруднение, а именно определить, к какому классу относится число.

Первый метод состоит в выполнении последовательности проверок, которые позволяют определить, в какой интервал попадает число. Этот метод легко программируется, но программа получается довольно длинной и медленной.

Самый быстрый метод заключается в определении значения целой переменной J , соответствующей номеру класса, к которому относится число. Например, для интервала $[0, 1]$, разделенного на 20 классов длиной 0.05, мы можем получить значение J следующим образом:

$$J = \text{целое } (20x) + 1,$$

$$x = 0.03 \rightarrow \text{целое } (0.6) + 1 = 1,$$

$$x = 0.06 \rightarrow \text{целое } (1.2) + 1 = 2.$$

Чтобы подсчитать число элементов в каждом классе, можно действовать следующим образом:

- 1) получить случайное число x ,
- 2) вычислить J ,
- 3) установить $T(J) = T(J) + 1$, где T — массив из p элементов и каждый элемент $T(J)$ служит счетчиком для класса J .

Замечания

1. Вместо $J = \text{INT}(20. * x) + 1$ можно написать $J = \text{INT}(20. * x + 1.)$, что приведет к замедлению при выполнении, или

$J = \text{IFIX}(20. * X) + 1$ с использованием функции IFIX , которая, вообще говоря, эквивалентна INT , но реализуется не во всех компиляторах; кроме того, сокращение INT более понятно.

2. Инструкция $J = 20. * x + 1$ ошибочна, так как в этом случае нет уверенности, что будет отброшена дробная часть (в некоторых системах может произойти округление).

Если используется интервал, отличный от $[0, 1]$, например $[a, b]$, то задача слегка усложняется. Тогда придется разделить интервал на p классов длиной $h = (b - a)/p$ и вычислять

$$J = \text{целое} \left[(x - a) \cdot \frac{1}{h} + 1 \right].$$

Пример

$$a = 3 \quad p = 20 \Rightarrow h = 0.5$$

$$b = 13$$

$$x = 4.99 \Rightarrow J = \text{целое} \left(4.99 \cdot \frac{1}{0.5} \right) + 1 = 9 + 1 = 10.$$

Мы можем теперь написать следующую программу:

```

INTEGER T(20),P
READ(2)A,B,P,N
DO 10 I=1,P
10  T(I)=0
    HI=FLOAT(P)/(B-A)
    DO 20 I=1,N
        X=HASARD(A,B)
        J=INT((X-A)*HI)+1
20  T(J)=T(J)+1
    WRITE(1,50)(I,T(I),I=1,P)
    STOP
45  FORMAT(2F10.1,I2,I6)
50  FORMAT(2X,IHI,3X,4HT(I)/(IX,I2,3X,I4))
END

```

Результат выполнения программы:

0.,20.,20,10000

I	T(I)
1	514
2	536
3	521
4	506
5	501
6	518
7	499
8	512
9	520
10	508
11	525
12	476
13	456
14	459
15	510
16	510
17	464
18	436
19	503
20	476

Решение задачи 2

Пользуясь массивом T , полученным ранее, нужно определить длину каждого столбика гистограммы.

Обычно ищется самое большое значение $T(J)$ и затем оно служит для выбора идеального масштаба, чтобы наилучшим образом использовать ширину бумаги. Этот метод не представляет трудности.

Можно, однако, поступить иначе: *заранее* выбрать масштаб L и вычислять последовательно длину каждого столбика. Если одна позиция в строке соответствует L числам, то $T(J)$ числам в классе J соответствует длина K , которая вычисляется по следующей формуле:

$$K = \frac{T(J)}{L}.$$

Если вычисления производятся с округлением, то возможны два варианта записи:

$$K = T(J)/\text{FLOAT}(K)$$

или лучше

$$K = \text{INT}(T(J)/RL + 0.5),$$

где $RL = \text{FLOAT}(K)$ вычисляется только один раз перед входом в цикл, а константа 0.5 используется для округления.

$$K = \text{INT}(T(J)/RL + 0.5)$$

↑ Чтобы округлить

$$RL = \text{FLOAT}(K)$$

↑ вычисляется один раз перед началом цикла

```

INTEG ER T(20),A(72),P
INTEG ER AST
DATA AST,P,N/1H*,20,10000/
RP=FLØAT(P)
DØ 10 I=1,P
10   T(I)=0
    DØ 20 I=1,N
        Y=HASARD(0.,1.)
        J=INT(RP*Y)+1
        T(J)=T(J)+1
20   WRITE(1,50)(I,T(I),I=1,P)
    DØ 30 I=1,72
30   A(I)=AST
    DØ 40 I=1,P
        K=(T(I)+5)/10
40   WRITE(1,60)(A(J),J=1,K)
    STØP
50   FØRMAT(2X,1H1,3X,4HT(I))/(1X,I2,3X,I4)
60   FØRMAT(72A1)
    ENP

```

↑ Чтобы K было меньше 72.
Для округления прибавляется 5.

ГЛАВА 6. ФИНАНСОВЫЕ РАСЧЕТЫ

6.1. Платежи по займу

Если сделан заем на сумму 10 000 франков сроком на несколько лет (например, сроком на 15 лет) с известным годовым процентом (например, 9%) и если выплаты производятся каждый месяц, то какой будет сумма ежемесячного платежа, какую долю в каждом платеже составит выплата собственно по займу и выплата по проценту? Решите ту же задачу, когда выплата производится раз в квартал и раз в полгода.

Этот вопрос имеет непосредственное отношение к упражнению, для которого необходимы следующие данные:

A = сумма займа;

taux = годового процент в %;

m = число выплат за год:

$m = 12$ — ежемесячные выплаты;

$m = 4$ — ежеквартальные выплаты;

$m = 2$ — выплаты раз в полгода;

$m = 1$ — выплаты один раз в год;

n = общее число выплат;

L = виды выплат:

$L = 0$ — постоянная выплата;

$L = 1$ — уменьшающаяся выплата.

6.1.1. Вычислительные формулы

1. *Расчет «эквивалентного процента».* Величина t соответствует месячному проценту, если платежи производятся ежемесячно; квартальному проценту, если платежи производятся ежеквартально, и т. д. Имея годового процент (в %), можно вычислить t по формуле

$$t = \left(1 + \frac{\text{taux}}{100}\right)^{1/m} - 1,$$

где t представлено в классическом виде: 0.03 соответствует 3%.

2. *Расчет суммы платежа.* При $L = 0$ выплаты производятся равными суммами. Тогда сумма каждой выплаты равна

$$R = A \frac{t(1+t)^n}{(1+t)^n - 1}.$$

Затем определяется часть платежа, идущая на погашение долга, и часть, которая зависит от процента:

	процент	погашение
1-й платеж: $I_1 = At$		$C_1 = R - I_1$
2-й платеж: $I_2 = (A - C_1)t$		$C_2 = R - I_2$
и т. д.		

При $L = 1$ сумма выплат уменьшается, но в каждом платеже часть, идущая на погашение долга, остается неизменной и равна A/p .

	долг
Сумма 1-го платежа $R_1 = \frac{A}{n} + At$	$A - \frac{A}{n}$

Сумма 2-го платежа $R_2 = \frac{A}{n} + A \left(1 - \frac{1}{n}\right)t$	$A - \frac{2A}{n}$
--	--------------------

и т. д.

6.1.2. Задача

Нарисовать принципиальную блок-схему.

Написать программу; все вычисления должны выполняться с двойной точностью.

6.1.3. Решение

Расчет эквивалентного процента не зависит от способа выплаты займа. Напротив, расчет остатка производится

по-разному. Это означает, что начало блок-схемы будет таким, как на рис. 6.1.

В практических случаях необходим контроль за правильностью исходных данных, чтобы обнаружить ошибки перфорации и другие ошибки.

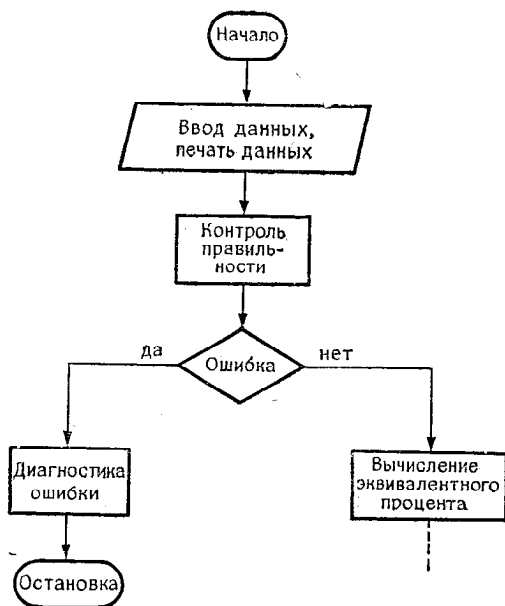


Рис. 6.1.

Можно контролировать:

- величину процента, который заведомо должен быть положительным и заключаться, например, между 0 и 20%;
- сумму займа, которая не должна превышать 10 000 франков;
- значения M и N , которые должны быть целыми (на практике значениями M могут быть только 1, 2, 4 и 12).

Здесь мы не будем заниматься этим контролем. Опустим также необходимый в реальных случаях расчет вымат на дополнительные расходы, включая начальные расходы по составлению бумаг, обязательные расходы на производство платежей, на страхование и т. д. На практике различают, кроме того, досрочные и отложенные платежи и т. д.

В блок-схеме, изображенной на рис. 6.2, не учитываются все эти осложняющие дело подробности.

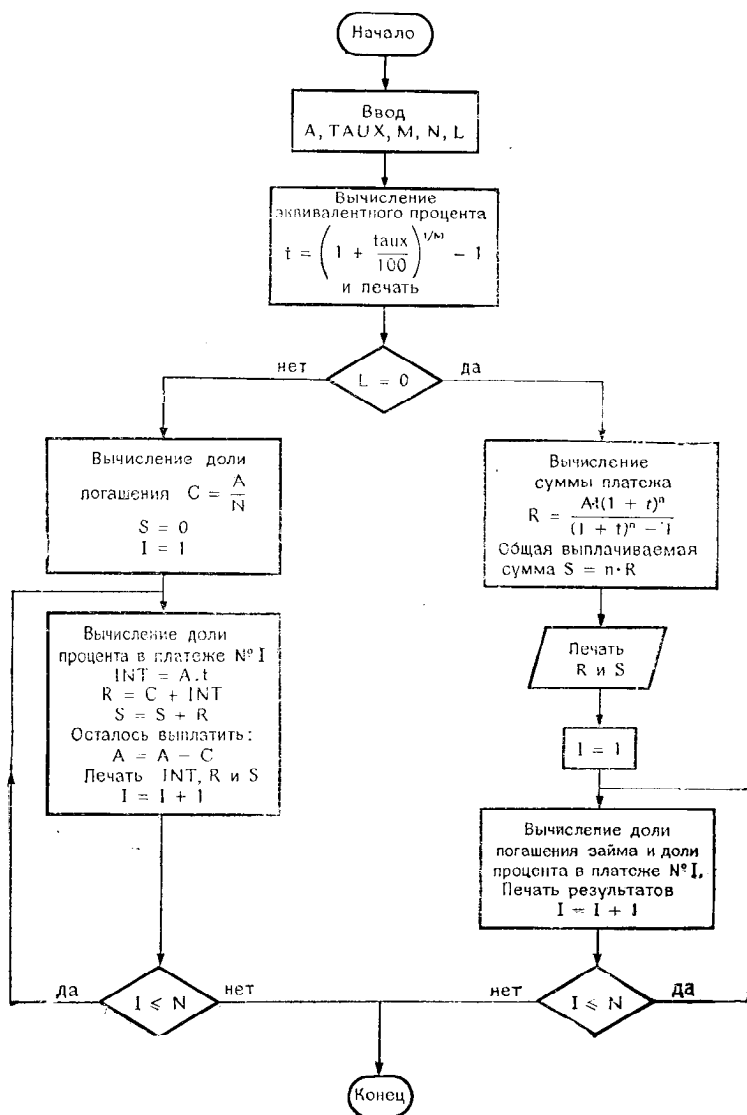


Рис. 6.2.

Блок-схема переводится в следующую программу:

```

DOUBLE PRECISION A,TAUX,T,X,R,INT,C,S,UN
DATA UN/1.0D+0/
READ(2)A,TAUX,M,N,L
T=(UN+0.01D+0 *TAUX)**(1./FLOAT(M))-UN
WRITE(1,50)T
IF(L.EQ.0)GO TO 20
C=A/DBLE(FLOAT(N))
S=0.
DO 10 I=1,N
INT=A*T
R=C+INT
S=S+R
WRITE(1,60)INT,R,S
10 A=A-C
STOP
20 X=(1.D+0+T)**N
R=A*T*X/(X-1.D+0)
S=DBLE(FLOAT(N))*R
WRITE(1,70)R,S
DO 30 I=1,N
INT=A*T
C=R-INT
WRITE(1,80)INT,C
30 A=A-C
STOP
50 FORMAT(18H TAUX EQUIVALENT = ,D17.10//)
60 FORMAT(3D18.8)
70 FORMAT(3H R=,D18.12/3H S=,D18.12//)
80 FORMAT(2D15.6)
STOP
END

```

6.2. Расчет процента

Пусть сделан заем на сумму A и процент неизвестен. На погашение займа требуется N платежей равными суммами R по M платежей в год.

Определить годовой процент.

Чтобы произвести этот расчет, надо предварительно определить эквивалентный процент t . Формулы, которые связывают A , M , t , R и $TAUX$, будут такими:

$$R = A \frac{t(1+t)^n}{(1+t)^n - 1},$$

$$1 + \frac{TAUX}{100} = (1+t)^m.$$

Можно использовать подпрограмму `DICHO`, рассмотренную ранее¹⁾, внося в нее необходимые изменения для вычислений с двойной точностью.

¹⁾ См. разд. 3.8. — *Прим. ред.*

Решение

Основная трудность состоит в том, чтобы найти t как корень уравнения

$$\frac{At(1+t)^n}{(1+t)^n - 1} - R = 0.$$

Это уравнение можно решить с помощью подпрограммы DICHO, если:

- внести в нее изменения для вычислений с двойной точностью;
- правильно задать исходные параметры.

При вычислении годового процента можно взять начальный интервал $(0., 0.2)$, а при вычислении месячного эквивалентного процента $(0., 0.016)$. Так как программа должна одинаково правильно функционировать при месячных, квартальных, полугодовых и годовых платежах, то мы вынуждены выбрать начальный интервал $(0., 0.2)$, что приводит к такому вызову подпрограммы:

CALL DICHO (F, 0., 0.2 и т. д.).

Можно уточнить верхнюю границу интервала, используя деление на M , и тогда вызов подпрограммы будет выглядеть так:

CALL DICHO (F, 0., 0.2/ M и т. д.).

Впрочем, заметим, что функция $F = A \frac{t(1+t)^n}{(1+t)^n - 1} - R$ не определена при $t = 0$, поскольку знаменатель равен нулю. Поэтому необходимо слегка сместить нижнюю границу интервала, выбрав ее равной, например, 0.005.

Кроме того, значения переменных A , R , T , $TAUX$ и функции F будут вычисляться с двойной точностью. Следовательно, некоторые параметры DICHO будут не вещественными переменными, а переменными с двойной точностью. Речь идет об идентификаторах F (имя функции), X , а также переменных U , V , W , A и B , которые используются в вычислениях.

Для удобства пользователя параметры EPSI и ETA задаются с обычной точностью, что слегка усложняет условные инструкции и требует перевода в REAL или в DOUBLE PRECISION. Проверки делаются быстрее при простой точности. Принимая во внимание все предыдущие замечания, мы напомним такие программы:

```

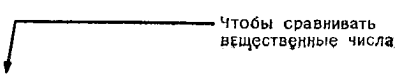
DOUBLE PRECISION T,A,R,F,TAUX
COMMON N,A,R
EXTERNAL F
READ(2)A,R,M,N
WRITE(1,20)A,R,M,N
CALL DICH0(F,0.005,0.2/FL0AT(M),0.000005,0.00002,T,L)
IF(L.LT.0) STOP 77
TAUX=100.D+0+((1.D+0+T)**M-1.D+0)
WRITE(1,10)TAUX
10  FORMAT(6H TAUX=,D20.12)
20  FORMAT(2D15.7,2I6)
STOP
END

```

```

SUBROUTINE DICH0(F,C,D,EPSI,ETA,X,L)
DOUBLE PRECISION F,X,A,B,U,V,W
A=C
B=D
U=F(A)
V=F(B)
IF(U*V)20,20,10
10  L=-1
RETURN
20  X=(A+B)*0.5D+0
W=F(X)
IF(U*W)30,30,40
30  B=X
V=W
GO TO 50
40  A=X
U=W
50  IF(ABS(SNGL(W)).LE.EPSI)GO TO 60
IF(ABS(SNGL(B-A)).GT.ETA)GO TO 20
L=1
RETURN
60  L=0
RETURN
END

```



```

DOUBLE PRECISION FUNCTION F(T)
DOUBLE PRECISION T,Q,A,R
COMMON N,A,R
Q=(1.D+0+T)**N
F=A*T*Q/(Q-1.D+0) -R
RETURN
END

```

ГЛАВА 7. РАЗЛИЧНЫЕ ПРОСТЫЕ УПРАЖНЕНИЯ

7.1. Сортировка в оперативной памяти

В этом параграфе мы рассмотрим два упражнения, посвященные сортировке с использованием метода пузырька (BUBBLE) и метода Шелла (SHELL), которые, очевидно, можно применить и при сортировке файлов, находящихся во вспомогательной памяти. Однако существуют другие методы, которые в данном случае более приемлемы¹⁾.

7.1.1. Сортировка методом пузырька

Имеется массив A , содержащий n элементов. Надо разместить элементы массива таким образом, чтобы они расположились, например, в возрастающем порядке. Метод пузырька состоит в организации упорядоченного списка элементов, в который на соответствующие им места добавляются один за другим неотсортированные элементы.

Этому методу соответствует следующий алгоритм:

1. Сравнить $A(1)$ и $A(2)$:

- если $A(1) > A(2)$, сделать перестановку;
- если $A(1) \leq A(2)$, перейти к 2.

2. Сравнить $A(2)$ и $A(3)$:

- если $A(2) \leq A(3)$, перейти к 3;
- если $A(2) > A(3)$, поменять местами $A(2)$ и $A(3)$, сравнить $A(2)$ и $A(1)$ и сделать перестановку, если она нужна.

3. И вообще, если $A(I)$ — последний элемент в упорядоченном списке, то сравнить $A(I)$ и $A(I+1)$.

4. Если $A(I) \leq A(I+1)$, установить $I = I+1$ и, если $I \leq n-1$, перейти к 3, в противном случае сортировка закончена.

Если $A(I) > A(I+1)$, то

- поменять местами $A(I)$ и $A(I+1)$;
- убедиться, что $A(I)$ находится на своем месте в упорядоченном списке;
- установить $k = I$.

5. Сравнить $A(k-1)$ и $A(k)$:

¹⁾ См. статью Вильяма А. Мартина «Sorting», Computing Survey, vol. 3, № 4, декабрь 1971 г., а также книгу Дональда Е. Кнута «Искусство программирования для ЭВМ», том 3, гл. 5, М., «Мир», 1978 г.

- если $A(k)$ меньше, то сделать перестановку; установить $k = k - 1$ и, если $k > 1$, перейти к 5;
- если $A(k) \geq A(k - 1)$, то перейти к 4.

Очевидно, что этот алгоритм может быть представлен в более лаконичной форме, которую читателю предлагается найти самостоятельно.

Задача

1. Нарисуйте блок-схему, соответствующую методу пузырька.
2. Рассмотрите алгоритм и преобразуйте блок-схему так, чтобы перестановка элементов происходила только в одном месте.
3. Напишите подпрограмму BUBBLE с двумя параметрами: A — сортируемый массив и n — размер массива. Предполагается, что массив A содержит вещественные числа.

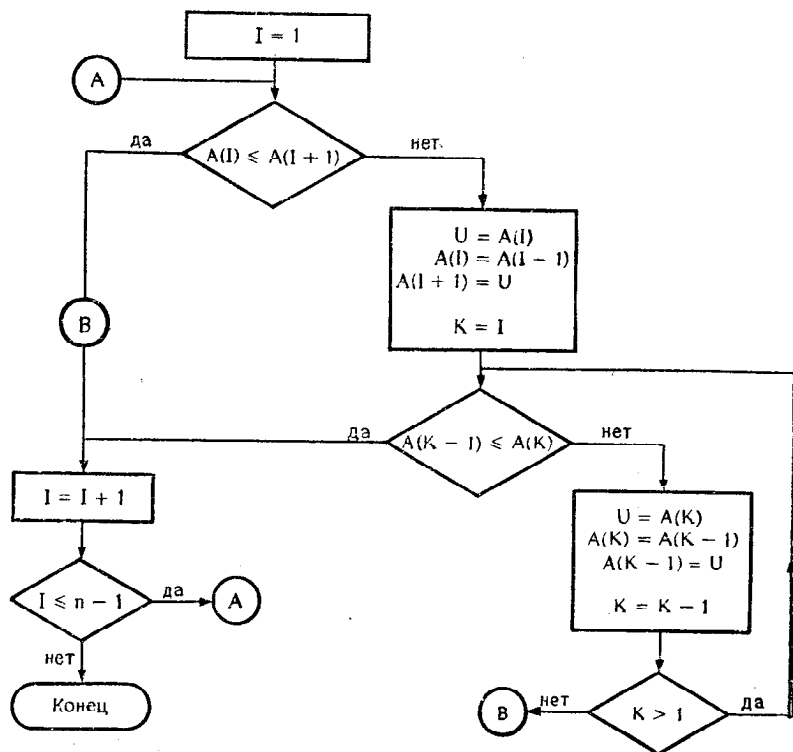


Рис. 7.1.

Решение

Первая блок-схема (рис. 7.1) получается как простое переложение рассмотренного метода.

40



```

INTEGER A(12)
DATA N/12/
READ(3,40)A
WRITE(1,40)A
FORMAT(12I3)
CALL BUBBLE(A,N)
WRITE(1,40)A
STOP
END

```

```

SUBROUTINE BUBBLE(A,N)
INTEGER A(N),U
N1=N-1
DO 20 I=1,N1
    K=I
    J=K+1
    IF(A(K).LE.A(J))GO TO 20
    U=A(K)
    A(K)=A(J)
    A(J)=U
    WRITE(1,30)(A(L),L=1,J)
30    FORMAT(15I3)
    K=K-1
    IF(K.GE.1)GO TO 10
20    CONTINUE
RETURN
END

```

нительной целой переменной J, которая позволяет сократить затраты на вычисление индекса.

Результат выполнения программы:

```

19 13 5 27 1 26 31 16 2 9 11 24
13 19
13 5 19
5 13
5 13 19 1 27
5 13 1 19
5 1 13
1 5
1 5 13 19 26 27
1 5 13 19 26 27 16 31
1 5 13 19 26 16 27
1 5 13 19 16 26
1 5 13 16 19
1 5 13 16 19 26 27 2 31
1 5 13 16 19 26 2 27
1 5 13 16 19 2 26
1 5 13 16 2 19
1 5 13 2 16
1 5 2 13
1 2 53
1 2 5 13 16 19 26 27 9 31
1 2 5 13 16 19 26 9 27
1 2 5 13 16 19 9 26
1 2 5 13 16 9 19
1 2 5 13 9 16
1 2 5 9 13
1 2 5 9 13 16 19 26 27 11 31
1 2 5 9 13 16 19 26 11 27
1 2 5 9 13 16 19 11 26
1 2 5 9 13 16 11 19
1 2 5 9 13 11 16
1 2 5 9 11 13
1 2 5 9 11 13 16 19 26 27 21 31
1 2 5 9 11 13 16 19 26 21 27
1 2 5 9 11 13 16 19 21 26
1 2 5 9 11 13 16 19 21 26 27 31
STOP NUMBER 00

```

7.1.2. Сортировка методом Шелла

При сортировке n чисел вначале определяется k , такое, что

$$2^k < n \leq 2^{k+1},$$

затем вычисляем $D = 2^k - 1$.

На первом этапе сортировки производятся следующие действия (переменная I изменяется от 1 до $N - D$):

1. Проверить, выполняется ли неравенство $A(I) \leq A(I + D)$.

1. а) если да, то перейти к 2;
1. б) если нет, то
 - осуществить перестановку;
 - установить $k = I$;
1. с) проверить, выполняется ли неравенство $A(k - D) \leq A(k)$
 если да, то перейти к 2
 если нет, то осуществить перестановку, установить $k = k - D$;
 перейти к 1. с).

2. Продолжить проверки, увеличивая I каждый раз на 1. Когда этот этап закончен, но $D > 0$, установить $D = (D - 1)/2$.

Новый этап сортировки провести так же, как и предыдущий. Когда значение D станет нулевым, сортировка закончена.

Задача

Нарисовать блок-схему.

Написать подпрограмму SHELL(A, N), где A — представляет массив, подлежащий сортировке, а N — число элементов массива. Предполагается, что массив A типа INTEGER.

Решение

Чтобы определить D , достаточно найти самое малое число, являющееся степенью 2, большее, чем n . Затем это число нужно разделить на 2, чтобы получить $D = 2^k$.

Заметим, кроме того, что вычисления $\frac{2^k}{2} - 1$ и $\frac{2^k - 1}{2}$, осуществленные над целыми величинами, дают в Фортране одинаковый результат.

Теперь мы можем составить блок-схему (рис. 7.3), которая позволит затем написать соответствующую программу на Фортране.

Блок-схема ни в коей мере не зависит от типа сортируемого массива. Однако при программировании необходимо использовать инструкцию, в которой декларируется тип массива.

В приведенных ниже листингах мы работаем с массивом A целого типа. Достаточно изменить две инструкции, специфицирующие тип (INTEGER на REAL или DOUBLE PRECISION), чтобы перейти к вещественному массиву или к массиву, содержащему числа с двойной точностью.

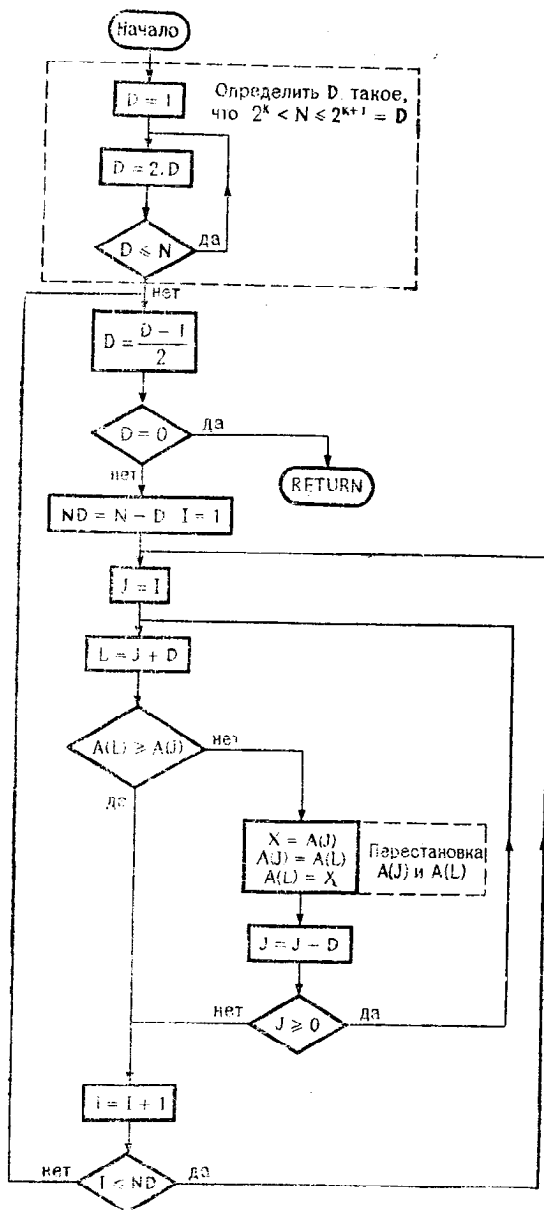


Рис. 7.3.

```

SUBROUTINE SHELL(A,N)
  INTEGER A(N),D,X
  D=1
10  D=2*D
    IF(D.LE.N)GO TO 10
20  D=(D-1)/2
    IF(D.EQ.0)RETURN
    ND=N-D
    DO 40 I=1,ND
      J=I
30  L=1+D
      IF(A(L).GE.A(J))GO TO 40
      X=A(J)
      A(J)=A(L)
      A(L)=X
      K=MAX0(J,L)
      WRITE(1,70)(A(M),M=1,K)
70  FORMAT(15I3)
      J=J-D
      IF(J.GT.0)GO TO 30
40  CONTINUE
    GO TO 20
  END

```

Эти 3 инструкции предназна-
чены только для того, чтобы
показать порядок выполняемых
перестановок

```

  INTEGER A(12)
  DATA N/12/
  READ(3,40)A
  WRITE(1,40)A
40  FORMAT(12I3)
  CALL SHELL(A,N)
  WRITE(1,40)A
  STOP
  END

```

Результат выполнения программы:

```

19 13 5 27 1 26 31 16 2 9 11 21 ← Начальное
16 13 5 27 1 26 31 19
16 2 5 27 1 26 31 19 13
16 2 5 11 1 26 31 19 13 9 27
11 2 5 16
11 1 5 16 2
11 1 5 16 2 13 31 19 26
11 1 5 16 2 13 9 19 26 31
11 1 5 9 2 13 16
9 1 5 11
9 1 5 11 2 13 16 19 21 31 27 26
1 9
1 5 9
1 5 9 2 11
1 5 2 9
1 2 5
1 2 5 9 11 13 16 19 21 27 31
1 2 5 9 11 13 16 19 21 27 26 31
1 2 5 9 11 13 16 19 21 26 27
1 2 5 9 11 13 16 19 21 26 27 31 ← Конечное
STOP NUMBER 00

```

18 перестановок

расположение

расположение

7.2. Редактирование таблицы преобразования

При редактировании таблицы преобразования не возникает обычно никаких затруднений в том, что касается вычислений. Но при этом по крайней мере начинающие программисты иногда сталкиваются с целым рядом деликатных вопросов, касающихся представления таблицы, так как необходимо извлечь максимум из возможностей инструкции FORMAT.

Задача

Напишите программу, которая позволит печатать соответствующие величины для температурных шкал ЦЕЛЬСИЯ, ФАРЕНГЕЙТА и РЕОМЮРА в виде такой таблицы:

+-----+-----+-----+			
I	ЦЕЛЬСИЯ	I	ФАРЕНГЕЙТА
+-----+-----+-----+			
I	0.0	I	32.00
I	1.0	I	33.80
I	2.0	I	35.60
I	3.0	I	37.40
I	4.0	I	39.20
I	5.0	I	41.00
I	6.0	I	42.80
I	7.0	I	44.60
I	8.0	I	46.40
I	9.0	I	48.20
I	10.0	I	50.00
I	11.0	I	51.80
I	12.0	I	53.60
I	13.0	I	55.40

Переход от градусов Цельсия к градусам Фаренгейта осуществляется по формуле

$$F = 1,8 \times C + 32.$$

Переход от градусов Цельсия к градусам Реомюра осуществляется по формуле

$$R = 0,8 \times C.$$

Решение

Трудность этого упражнения заключается, очевидно, в манипуляциях с форматами и в инструкциях печати. Мы можем выделить следующие этапы:

Печать заголовка:

- 1 строка знаков —, 4 знака +, формат с меткой 20;
- 1 строка, содержащая заголовки, и 4 буквы I, формат с меткой 30;
- 1 строка знаков —, 4 знака +: формат с меткой 20.

Далее нужно напечатать 100 строк таблицы по формату с меткой 40, который можно вывести частично из формата с меткой 30.

В последней строке печатаются знаки — и 4 знака $\dagger$ и используется тот же формат, что и для первой строки.

Изучим сначала формат, содержащий заголовки столбцов:

1 ЦЕЛЬСИЯ 1 ФАРЕНГЕЙТА 1 РЕОМЮРА 1

Можно использовать следующий формат:

30 FORMAT(2H I,9H ЦЕЛЬСИЯ ,1H I,12H, ФАРЕНГЕЙТА ,1H I,
1 9H РЕОМЮРА ,1H I)

Первая литера (здесь пробел) не печатается, но она уточняет расположение (пробел говорит о переходе к следующей строке).

Исходя из этого формата, легко построить формат с меткой 20:

20 FORMAT (2H +,9(1H-),1H+,12(1H-),1H+,9(1H-),
1 1H+)

Таким образом, повторяется литера. Это
проще, чем писать: **ЭН**-----

Чтобы печатать градусы Цельсия с одним десятичным знаком после запятой, надо использовать формат вида Fp. 1, где p — величина, которую предстоит определить. Учитывая, что число должно быть с каждой стороны заключено по меньшей мере в 2 пробела, 9 литер в столбце распределяются следующим образом:

(2H I, F7.1, 2X, 1HI ...
9 литер

Это же рассуждение применимо к двум другим столбцам. В конечном итоге мы получаем

40 FORMAT (2H 1,F7.1, 2X,1H1, F9.2, 3X,1H1,F8.2,2H 1)

```

WRITE(1,20)
WRITE(1,30)
WRITE(1,20)
C=0.
DO 10 I=0,100
    F=4.5*C+32.
    R=1.25*C
    WRITE(1,40)C,F,R
10  C=C+1.
20  FORMAT(2H +,9(1H-),1H+,12(1H-),1H+,9(1H-),1H+)
30  FORMAT(2H 1,9H ЦЕЛЬСИЯ ,1H1,12H ФАРЕНГЕЙТА ,1H1,
1    9H РЕОМЮРА ,1H1)
40  FORMAT(2H 1,F7.1,1X,2H 1,F9.2,3X,1H1,F8.2,2H 1)
STOP
END

```

7.3. Слияние двух массивов

Пусть даны два массива А и В, содержащие соответственно N и M элементов. Требуется объединить эти два массива так, чтобы получился массив С, в котором элементы располагаются в возрастающем порядке. Массив С, очевидно, должен содержать по крайней мере $N + M$ элементов.

7.3.1. Задача 1

Предположим, что массивы А и В содержат вещественные числа и что элементы в массивах А и В расположены в возрастающем порядке. Требуется:

- нарисовать блок-схему,
- написать подпрограмму FUSION(A, B, C, N, NPM).

Используемый метод:

а) Пересылать в С элементы a_i до тех пор, пока элементы b_j еще не перенесенные в С, превосходят пересылаемые элементы.

б) Если предыдущее условие не выполняется, то пересылать в С элементы b_j до тех пор, пока b_j не станет больше первого a_i , еще не перенесенного в С. После этого вернуться к а).

с) Когда один из массивов исчерпан, достаточно переслать в С оставшиеся элементы другого массива.

7.3.2. Задача 2

Предположим, что массивы А и В не упорядочены.

Вопрос

Как свести эту задачу к предыдущей?

7.3.3. Решение задачи 1

Если при некоторых i и j окажется, что $A(I) = B(J)$, то в последовательные позиции массива С помещаются $A(I)$ и $B(J)$. Относительный порядок $A(I)$, $B(J)$ или $B(J)$, $A(I)$ не имеет никакого значения. Трудность этого упражнения заключается в манипуляциях с индексами. Мы возьмем три индекса, по одному для каждого массива:

- индекс I для массива А;
- индекс J для массива В;
- индекс К для массива С.

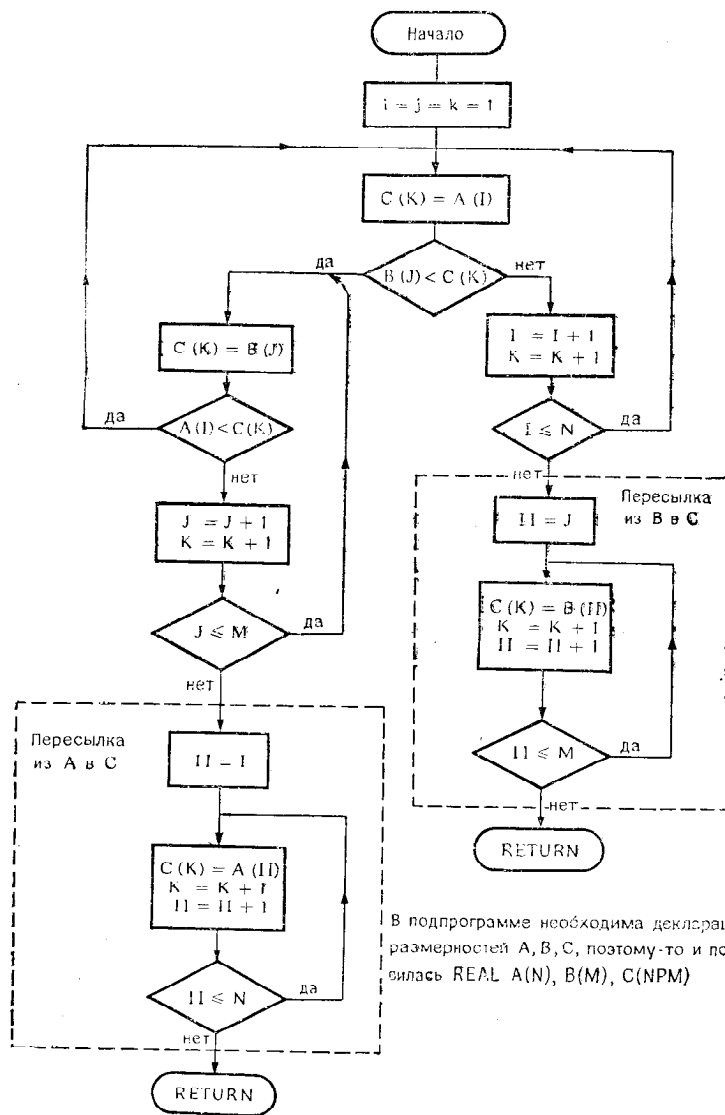
Индексы I и J указывают первые элементы соответственно в массивах А и В, еще не перенесенные в С. Индекс К указывает на элемент массива С, в котором разместится следующий элемент из А или В.

Блок-схема на рис. 7.4 начинается с присваивания $C(1) = A(1)$. Можно было бы начать с $C(1) = B(1)$. Мы получим следующий листинг:

```

100.      SUBROUTINE FUSION(A,B,C,N,N,NPM)
200.      REAL A(N),B(M),C(NPM)
400.      I=1
500.      J=1
600.      K=1
700.      10 C(K)=A(I)
800.      IF (B(J).LT.C(K))GO TO 30
900.      I=I+1
1000.     K=K+1
1100.     IF (I.LE.N)GO TO 10
1200.     DO 20 II=J,M
1300.       C(K)=B(II)
1400.     20 K=K+1
1500.     RETURN
1600.     30 C(K)=B(J)
1700.     IF (A(I).LT.C(K))GO TO 10
1800.     J=J+1
1900.     K=K+1
2000.     IF (J.LE.M)GO TO 30
2100.     DO 40 II=I,N
2200.       C(K)=A(II)
2300.     40 K=K+1
2400.     RETURN
2500.     END

```



7.3.4. Решение задачи 2

Следует, очевидно, попытаться свести задачу к предыдущему случаю. Для этого можно:

- отсортировать массив A;
- отсортировать массив B;
- затем слить эти массивы, как в предыдущем случае.

Чтобы упорядочить массивы, можно воспользоваться подпрограммой сортировки SHELL (см. упр. 7.1.2).

7.4. Поиск в массиве

Имеется целочисленный массив A, содержащий N элементов, расположенных в возрастающем порядке, и задано значение ключа. Переменной S необходимо присвоить индекс того элемента массива A, который равен ключу. Если такой элемент не найден, то переменной S присваивается 0.

7.4.1. Используемый метод

Назовем II, IM, IS соответственно нижний, средний и верхний индексы. При поиске используется метод дихотомии (называемый также бинарным поиском).

1. Вначале установить

$$II = 1,$$

$$IS = N,$$

$$RECH = 0 \text{ и перейти к 2.}$$

2. Если $II = IS$, то деление пополам закончено. Если ключ найден, то установить $RECH = II$ и возвратиться.

Если ключ не найден, то возвратиться.

Если $II \neq IS$, то перейти к 3.

3. Установить $IM = (II + IS)/2$.

Если ключ $\begin{cases} < A(IM), \text{ установить } II = IM \text{ и перейти к 2,} \\ = A(IM), \text{ установить } RECH = IM \text{ и возвратиться,} \\ > A(IM), \text{ установить } IS = IM \text{ и перейти к 2.} \end{cases}$

7.4.2. Задача

- Нарисовать блок-схему.
- Написать целочисленную функцию $RECH(\text{ключ}, A, N)$, которая дает результат

$RECH = \text{индекс, если найден } A(\text{индекс}) = \text{ключ,}$

$RECH = 0$, если элемент, равный ключу, не найден.

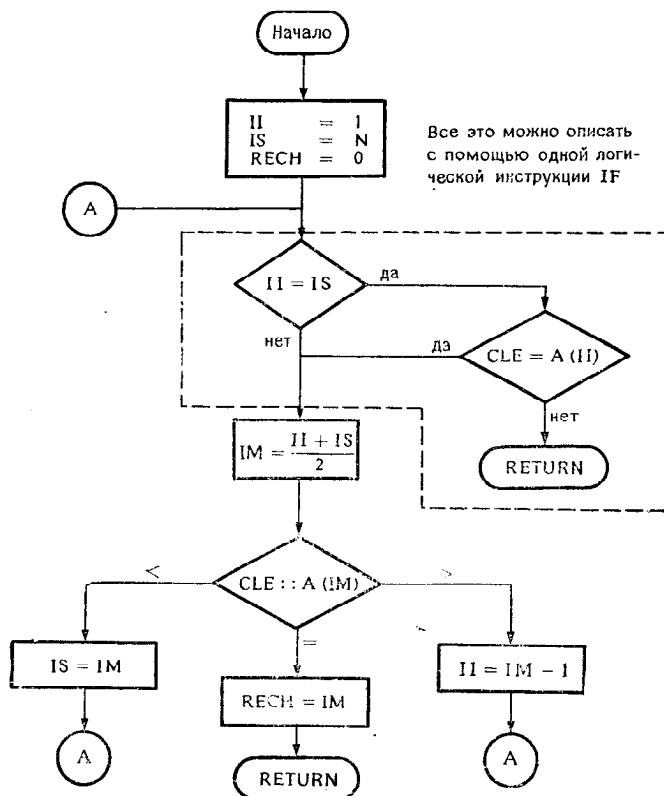


Рис. 7.5.

```

INTEGER FUNCTION RECH(CLE,A,N)
  INTEGER A(N),CLE
  II=1
  IS=N
  RECH=0
10 IF (II.EQ. IS .AND. CLE.NE. A(II)) RETURN
  IM=(II+IS)/2
  IF (CLE-A(IM))20,40,30
20 IS=IM
  GO TO 10
30 IS=IM
  GO TO 10
40 RECH=IM
  RETURN
END

INTEGER RECH,TAB(100)
J=RECH(K,TAB,100)

```

- Написать программу, которая обращается к этой функции.

7.4.3. Решение

Не представляет труда нарисовать блок-схему. Заметим, что две последовательные проверки $II = IS$ и $CLE = A(II)$ можно объединить в одном логическом IF. Таким образом, получается блок-схема, изображенная на рис. 7.5, и соответствующий ей листинг.

ГЛАВА 8. РАЗЛИЧНЫЕ БОЛЕЕ СЛОЖНЫЕ УПРАЖНЕНИЯ

8.1. Определение дня недели:

список пятниц, приходящихся на 13-е число

Имея дату, т. е. ЧИСЛО, МЕСЯЦ, ГОД, нужно определить соответствующий день недели

Пример: требуется найти, что день 3/9/1973 — *понедельник*.

Предлагается следующий метод ¹⁾:

1. Вычислить величину N

Если месяц — январь или февраль високосного года, то установить $N = 1$.

Если месяц — январь или февраль обычного года, то установить $N = 2$, в других случаях установить $N = 0$.

Чтобы узнать, является ли год високосным, можно действовать следующим образом:

- разложить год на две части A_1 и A_2 :

A_1 содержит 2 старшие цифры года,

A_2 содержит 2 младшие цифры года.

Пример: $1973 \rightarrow A_1 = 19, A_2 = 73$:

- если $A_2 = 0$ и A_1 делится на 4, то год високосный;
- если $A_2 \neq 0$, то год високосный, если A_2 делится на 4.

2. Вычислить «код дня» C по формуле

$$C = \text{целое}(365.25 * A_2) + \text{целое}(30.56 * M) + J + N.$$

3. Вычислить остаток S от деления C на 7:

если $S = 0$, то день — среда,

если $S = 1$, то день — четверг,

если $S = 2$, то день — пятница,

...

если $S = 6$, то день — вторник.

Задача

1. Нарисовать блок-схему вычисления, стремясь свести к минимуму количество проверок. Написать соответствующую

¹⁾ Этот метод был опубликован М. Ленуаром в журнале фирмы IBM.

программу на Фортране, пытаясь разумно использовать логические и арифметические инструкции IF.

2. Изменить метод так, чтобы.

$S = 1$ соответствовало понедельнику,

$S = 2$ соответствовало вторнику,

.....

$S = 7$ соответствовало воскресенью.

3. Основываясь на предыдущих результатах, написать программу, которая составляет список всех пятниц, приходящих на 13-е число в период с 1970 по 2000 г.

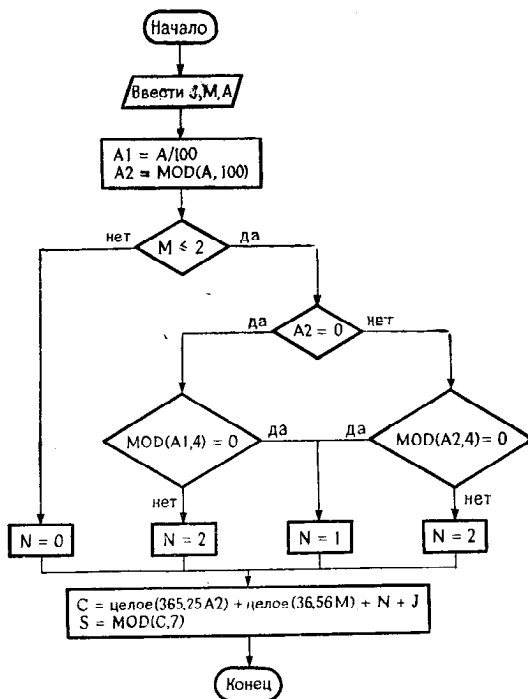


Рис. 8 1

Решение

1. Если $M > 2$, то нас совершенно не интересует тот факт, окажется год високосным или нет. Следовательно, такую проверку нужно отнести в начало.

Если $M \leq 2$, то необходима проверка $A2 = 0$ и в зависимости от результата этой проверки нас будет интересовать делимость $A1$ или $A2$ на 4.

Таким образом, получается блок-схема, изображенная на рис. 8.1.

Прежде чем перейти к программированию, необходимо сделать следующие замечания:

- проверки $M \leq 2$ и $A2 \neq 0$ можно сгруппировать, используя логическую инструкцию IF с последующей арифметической инструкцией IF:

IF (M.LE.2) IF (A2) 70, 10, 20



этот случай вообще невозможен, но он предусмотрен синтаксисом арифметической инструкции;

- операции умножения $365.25 * A2$ и $30.56 * M$ выполняются над числами с плавающей точкой (вещественными или с двойной точностью), хотя все переменные в программе относятся к целому типу. Поэтому следует обратить внимание на преобразования типов: целый $\rightarrow$ вещественный, вещественный $\rightarrow$ целый.

Теперь мы можем написать на Фортране такую программу:

```

      INTEGER A,A1,A2,C,S
      READ(2)J,M,A
      A1=A/100
      A2=M*100
      IF(M.LE.2)IF(A2)70,10,20
      N=0
      GOT0 50
10    IF(MOD(A1,4))70,40,30
20    IF(MOD(A2,4))70,40,30
30    N=2
      GOT0 50
40    N=1
50    I=INT(365.25*FLOAT(A2))
      K=INT(30.56*FLOAT(M))
      C=I+K+J+N
      S=1+MOD(C+2,7)
      WRITE(1,60)S
60    FORMAT(3H S=,I3)
70    STOP
      END
  
```

2. Если мы прибавим 2 к C, то получим следующее соответствие:

$S = 0$ — понедельник,

$S = 1$ — вторник,

...

$S = 6$ — воскресенье.

Остается прибавить 1 к S, чтобы получить требуемое соответствие, т. е.

$$S = 1 + \text{MOD}(C + 2, 7).$$

3. Список пятниц, приходящихся на 13-е число: для каждого месяца надо проверить, попадает ли 13-е число на пятницу. Следовательно, потребуется цикл по годам, в который будет вложен цикл по месяцам. Мы можем написать хотя бы такую программу:

```

      INTEGER A,A1,A2,C,S
      DATA J/13/
      WRITE(1,65)
      DO 55 A=1970,2000
      DO 55 M=1,12
      A1=A/100
      A2=MOD(A,100)
      IF(M.LE.2)IF(A2)70,10,20
      N=0
      GO TO 50
10    IF(MOD(A1,4))70,40,30
20    IF(MOD(A2,4))70,40,30
30    N=N+1
      GO TO 50
40    N=1
50    I=INT(365.25*FLOAT(A2))
      K=INT(30.56*FLOAT(M))3
      C=I+K+J+N
      S=MOD(C,7)
55    IF(S.EQ.2)WRITE(1,60)J,M,A
60    FORMAT(2X,13,4X,12,3X,15)
65    FORMAT(6H число,6H месяц,7H    год ///)
70    STOP
      END

```

Часть полученных результатов:

число	месяц	год	число	месяц	год
13	2	1970	13	6	1980
13	3	1970	13	2	1981
13	11	1970	13	3	1981
13	8	1971	13	11	1981
13	10	1972	13	8	1982
13	4	1973	13	5	1983
13	7	1973	13	1	1984
13	9	1974	13	4	1984
13	12	1974	13	7	1984
13	6	1975	13	9	1985
13	2	1976	13	12	1985
13	8	1976	13	6	1986
13	5	1977	13	2	1987
13	1	1978	13	3	1987
13	10	1978	13	11	1987
13	4	1979	13	5	1988
13	7	1979			

8.2. Автоматический расчет сдачи

Магазин оборудован автоматическими кассовыми аппаратами, которые работают следующим образом. Кассир набирает на клавиатуре кассового аппарата стоимость каждого купленного предмета, аппарат же постепенно накапливает суммарную стоимость. Затем кассир набирает на клавиатуре сумму денег, данную покупателем.

Аппарат должен подсчитать сумму сдачи и определить, какие монеты (купюры) образуют эту сумму. Предпочтение отдается монетам с большей ценностью, что обычно приводит к уменьшению числа монет, входящих в сдачу.

1-й случай: имеются монеты достоинством (в копейках ¹⁾) 1, 2, 5, 10, 20, 50, 100, 500.

2-й случай: имеются монеты со следующим достоинством: 1, 2, 5, 10, 25, 50, 100, 250.

Кроме того, в кассовом аппарате имеются отделения для каждого типа монет. Следовательно, нужно при сдаче учитывать наличие монет в том или ином отделении.

Используемые массивы:

$P(I)$ — достоинство монет типа I ,

$Q(I)$ — число монет типа I в сдаче,

$S(I)$ — число монет типа I в отделении.

Значением переменной E является общая сумма сдачи.

Задача 1

Предположим, что в каждом отделении бесконечно много монет, и, таким образом, этот вопрос нас не беспокоит.

• Нарисовать блок-схему в соответствии с методом, который используется в описанном кассовом аппарате, т. е. имея значения E и $P(I)$, нужно вычислить $Q(I)$ и напечатать все $Q(I)$, не равные нулю.

• Написать соответствующую программу на Фортране.

Задача 2

Примем во внимание конечный запас монет.

1-й случай: запаса монет хватает, чтобы дать сдачу.

2-й случай: запас монет не позволяет набрать сдачу и тогда она не может быть выдана.

При наличии достаточного запаса монет можно выделить два случая:

¹⁾ В оригинале «в сантимах». — Прим. ред.

• если отсутствуют монеты какого-либо достоинства, то необходимо использовать запас монет, низших по достоинству; в этом случае надо напечатать предупредительное сообщение;

• если запас каждого типа монет достаточен, то сообщение печатать не надо.

Требуется:

- проанализировать задачу,
- нарисовать блок-схему,
- написать соответствующую программу на Фортране.

Решение задачи 1

Метод заключается в том, что вначале в сдачу включается максимальное количество монет самого высокого достоинства, после чего образуется новая сумма для сдачи. Далее берется максимальное число монет достоинством непосредственно меньшим, чем оставшаяся сумма, и так до тех пор, пока не будет выдана вся сдача.

Очевидно, появится цикл по индексу монет, и, чтобы воспользоваться инструкцией DO, желательно расположить монеты в порядке уменьшения достоинства.

1-й случай

I	1	2	3	4	5	6	7	8
p(I)	500	100	50	20	10	5	2	1

2-й случай

I	1	2	3	4	5	6	7	8
p(I)	250	100	50	25	10	5	2	1

Таким образом, мы приходим к блок-схеме, изображенной на рис. 8.2.

Примеры выполнения программы:

4138 ← E

```
8 PIECE 500 CENTIMES
1 PIECE 100 CENTIMES
1 PIECE 20 CENTIMES
1 PIECE 10 CENTIMES
1 PIECE 5 CENTIMES
1 PIECE 2 CENTIMES
1 PIECE 1 CENTIMES
```

STOP NUMBER 00

4139 ← E

```
8 PIECE 500 CENTIMES
1 PIECE 100 CENTIMES
1 PIECE 20 CENTIMES
1 PIECE 10 CENTIMES
1 PIECE 5 CENTIMES
2 PIECE 2 CENTIMES
```

STOP NUMBER 03

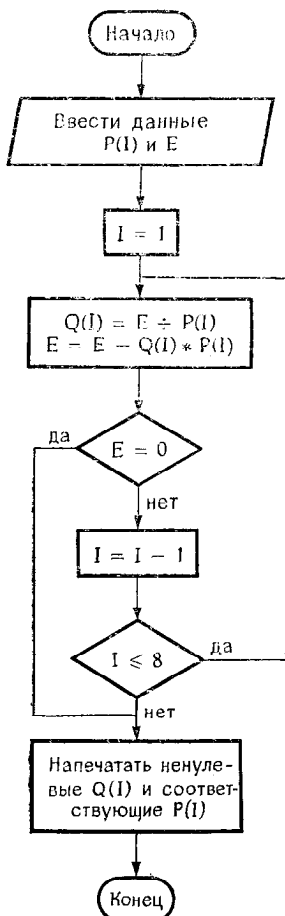


Рис. 8.2.

```

INTEGER E,P(8),Q(8)
READ(3,100)(P(I),I=1,8)
READ(2)E
DO 10 I=1,8
  Q(I)=E/P(I)
  E=MOD(E,P(I))
  IF(E.EQ.0)GO TO 20
10 CONTINUE
20 DO 30 I=1,8
30 IF(Q(I).NE.0)WRITE(1,110)Q(I),P(I)
STOP
100 FORMAT(8I4)
110 FORMAT(1X,I2,6H PIECE,I5,9H CENTIMES
END

```

Замечание

Некоторые компиляторы не допускают, чтобы конечной инструкцией цикла DO была логическая инструкция IF. В этом случае надо добавить инструкцию CONTINUE.

Пример

```
DO 30 I=1,8
  IF (Q(I).NE.0) и т. д.
30 CONTINUE
```

Решение задачи 2

• Если общий запас монет меньше суммы сдачи, то очевидно, сдачу дать нельзя. Может случиться, что запас монет превышает сумму сдачи, и тем не менее операция невозможна. Это обычно происходит только после нескольких последовательных сдач.

• Поэтому желательно вычитать $Q(I)$ из $S(I)$ только в том случае, когда есть полная уверенность в возможности выплаты сдачи (см. блок-схему на рис. 8.3 и соответствующие листинги).

```

INTEGER P(8),Q(8),S(8),E
COMMON P,Q,S
READ(3,45)(P(I),S(I),I=1,8)
5 READ(2)E ← Чтение по формату,
CALL RENDU(E) ← определяемому
WRITE(1,10)(I,P(I),Q(I),S(I),I=1,8) ← системой
GO TO 5
10 FORMAT(3H N0,2X,4H TИП,3X,3H КОЛ,2X,5H ЗАПАС
1 // (13,316))
45 FORMAT(14,13)
END

SUBROUTINE RENDU(E)
INTEGER P(8),Q(8),S(8),A,E
COMMON P,Q,S
L=0
DO 10 I=1,8 } ← Занесение нулей в Q(I) необходимо,
Q(I)=0 } ← поскольку подпрограмма может быть
DO 20 I=1,8 } ← вызвана несколько раз.
A=E/P(I) } ← Кроме того, не все Q(I) изменяются
Q(I)=MINO(A,S(I)) } ← в подпрограмме
E=E-Q(I)*P(I)
IF(A.NE.Q(I))L=L+1
IF(E.EQ.0) GO TO 30
20 CONTINUE
WRITE(1,50) } ← Не хватает денег, нельзя
STOP 8888 } ← дать сдачу
30 IF(L.NE.0)WRITE(1,60)L ← Можно дать сдачу, но не
DO 40 I=1,8 } ← оптимальным образом
40 S(I)=S(I)-Q(I)
RETURN
50 FORMAT(21H НЕ ХВАТАЕТ ДЕНЕГ )
60 FORMAT(15H ИССЯКАЕТ ЗАПАС , 3H L=,12//)
END
```

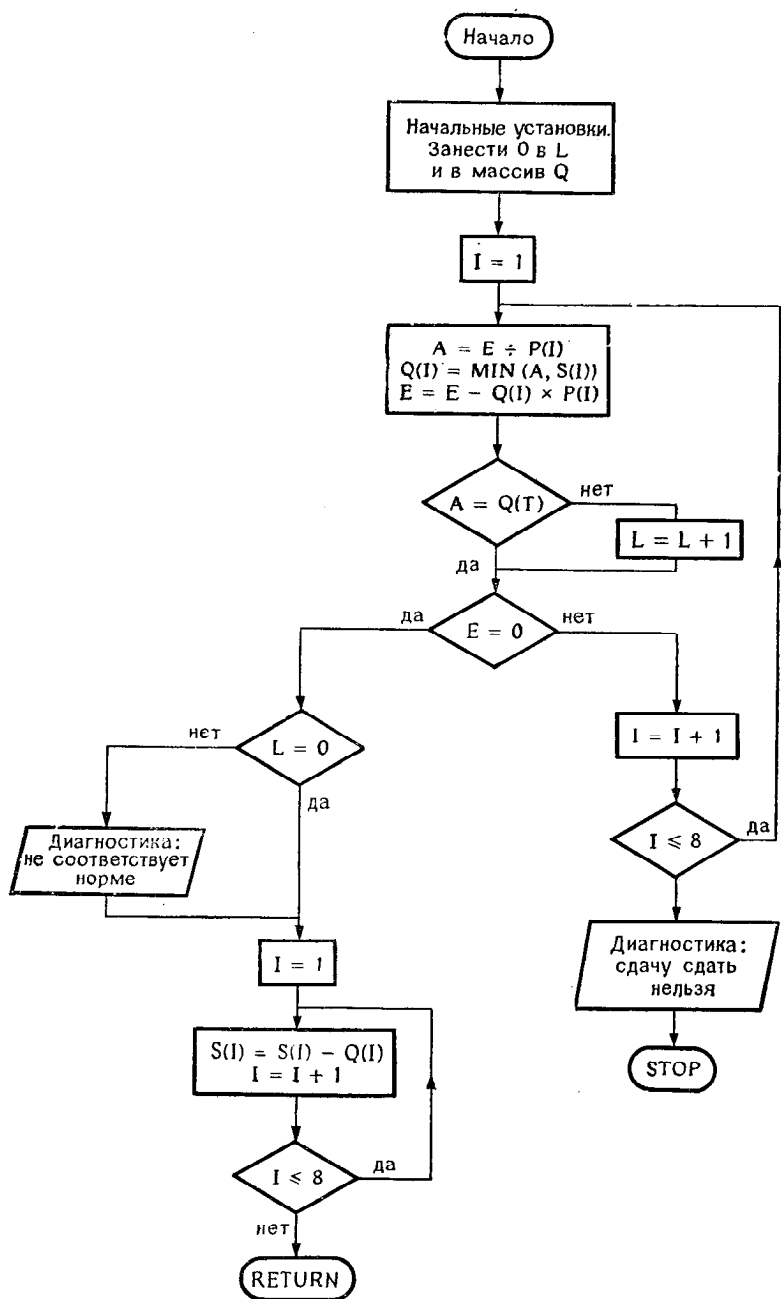


Рис. 8.3.

Пример выполнения программы:
Данные в формате (14, 13)

500 10
100 20
50 5
20 10
10 20
5 15
2 10
1 10

Результат выполнения программы:

3499 ← Сумма сдачи

№	ТИП	КОЛ	ЗАПАС
1	500	6	4
2	100	4	16
3	50	1	4
4	20	2	8
5	10	0	20
6	5	1	14
7	2	2	8
8	1	0	10

2699 ← Сумма сдачи

ИССЯКАЕТ ЗАПАС, $L = 1$ ← Поскольку есть только 4 монеты достоинством 500, а желательно иметь 5 таких монет

№	ТИП	КОЛ	ЗАПАС
1	500	4	0
2	100	6	10
3	50	1	3
4	20	2	6
5	10	0	20
6	5	1	13
7	2	2	6
8	1	0	10

1549 ← Сумма сдачи

ИССЯКАЕТ ЗАПАС, $L = 7$ ← Не хватает монет достоинством 500, 100, 50, 20, 10, 5 и 2, поэтому приходится давать сдачу более мелкими монетами

№	ТИП	КОЛ	ЗАПАС
1	500	0	0
2	100	10	0
3	50	3	0
4	20	6	0
5	10	20	0
6	5	13	0
7	2	6	0
8	1	2	8

8.3. Задача о козе

Коза привязана к столбику, расположенному в точке окружности радиуса R , охватывающей луг. Веревка, с помощью которой коза привязана к столбику, имеет длину D .

Какой должна быть длина веревки, чтобы для выпаса использовалась половина луга?

Численный пример: $R = 10$ м.

Чтобы решить эту задачу, надо табулировать функцию $S(D)$, которая выражает площадь выпаса в зависимости от D — длины веревки (рис. 8.4).

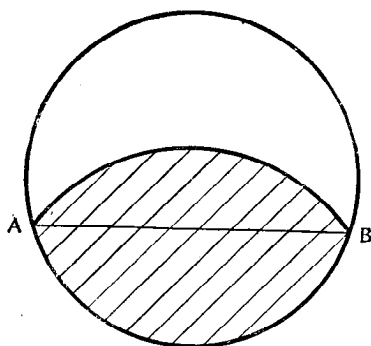


Рис. 8.4.

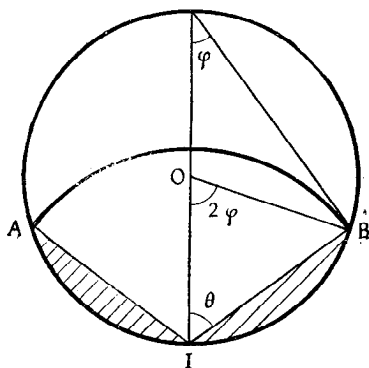


Рис. 8.5.

Решение

Самое трудное в этой задаче — не ошибиться при составлении уравнения.

Пусть θ — половина угла AIB, опирающегося на хорду AB, общую для двух окружностей (рис. 8.5):

$$\operatorname{tg}(\theta) = \frac{BC}{IB} = \frac{\sqrt{4R^2 - D^2}}{D}.$$

Функция ATAN2 позволяет вычислить значение θ .

Площадь сектора IAB окружности с центром в точке I и радиусом D равна

$$S1 = \theta D.$$

Прибавляя к S1 площадь заштрихованных сегментов (рис. 8.5), получим площадь выпаса. Чтобы определить эту площадь S2, вычислим сначала площадь сектора OAIВ окружности с центром в точке O и радиуса R, в который входит заштрихованный сегмент. Площадь равна

$$S3 = 2\varphi R, \quad \text{где} \quad \varphi = \frac{\pi}{2} - \theta,$$

т. е.

$$S3 = (\pi - 2\theta) R.$$

Вычтем из S3 площадь двух треугольников AOI и OIB.

$$\text{Площадь AOI} = \text{площадь OIB} = (R \sin \theta) \frac{D}{2},$$

и, следовательно,

$$S2 = (\pi - 2\theta) R - RD \sin \theta.$$

Тогда площадь выпаса равна

$$S = S1 + S2,$$

$$S(D) = \theta D + (\pi - 2\theta) R - RD \sin \theta.$$

1-й метод

Из рис. 8.5 видно, что площадь S, равная $\pi R^2/2$, получится при $D > R$. Поэтому достаточно табулировать функцию S(D), изменяя D от R до 2R с шагом H. Вначале значение H можно выбрать довольно большим, а затем повторно

табулировать функцию между двумя ближайшими к решению значениями D с более мелким шагом H .

Этому методу соответствует принципиальная блок-схема, изображенная на рис. 8.6.

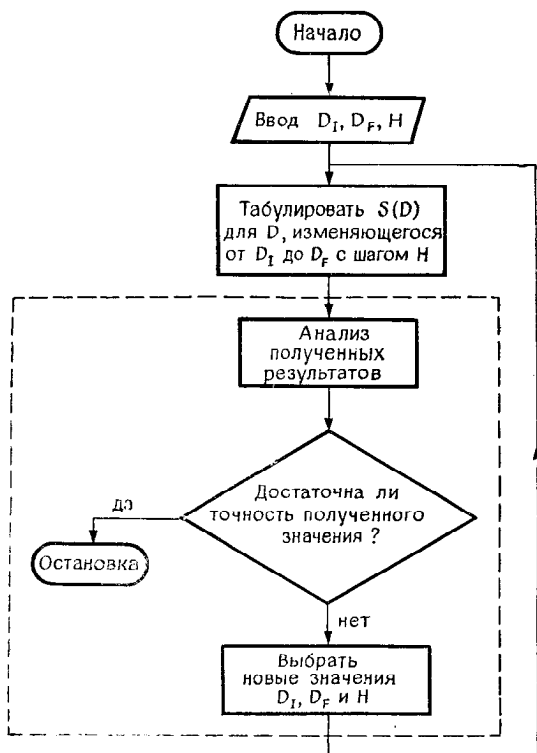


Рис. 8.6.

Та часть действий, которая выделена пунктиром, может быть выполнена человеком или может быть запрограммирована.

Если работа ведется за пультом в режиме разделения времени, то эти действия очень хорошо выполняются вручную. В этом случае мы можем перейти к подробной блок-схеме, изображенной на рис. 8.7.

Следующая программа соответствует предыдущей блок-схеме, но обладает меньшей гибкостью. Между тем она

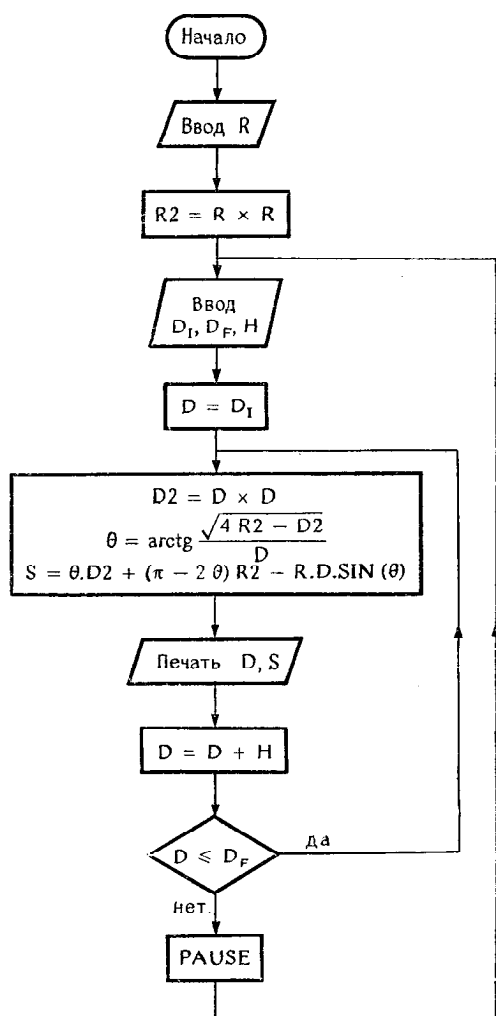


Рис. 8.7.

позволяет найти приближенное значение $D = 11,6$, весьма близкое к точному решению.

```
DATA PI,R/3.14159267,10./
D=11
H=0.1
R2=R*R
10 D2=D*D
TETA=ATAN2(SQRT(4.*R2-D2),D)
S=TETA*D2+(PI-2.*TETA)*R2-R*SIN(TETA)*D
WRITE(1,20)D,S
20 FORMAT(3H D=,F5.2,4X,2HS=,G15.7/)
D=D+H
IF(D,LE.12.)GO TO 10
STOP
END
```

Результат выполнения программы:

D=11.00	S=	144.2050
D=11.10	S=	146.3828
D=11.20	S=	148.5669
D=11.30	S=	150.7570
D=11.40	S=	152.9527
D=11.50	S=	155.1540
D=11.60	S=	157.3603
D=11.70	S=	159.5714
D=11.80	S=	161.7869
D=11.90	S=	164.0067
D=12.00	S=	166.2304

STOP NUMBER 00

2-й метод

Можно попытаться найти прямое решение уравнения

$$S(D) = \pi R^2/2.$$

Необходимо решить следующую систему:

$$\theta = \arctg \frac{\sqrt{4R^2 - D^2}}{D},$$

$$S(D) = \theta D + (\pi - 2\theta)R - RD \sin \theta = \pi R^2/2.$$

Как и прежде, мы можем воспользоваться подпрограммой DICHO, рассмотренной ранее.

Приведенная ниже программа позволяет быстро получить значение

$$D = 11,582.$$

```

EXTERNAL S
CALL DICH0(S,10.,15.,0.001,0.02,D,L)
IF(L,LT,0)STOP 77
WRITE(1,20)D
20  FORMAT(3H D=,F7.3//)
STOP 00
END

SUBROUTINE DICH0(F,C,D,EPSI,ETA,X,L)
A=C
B=D
U=F(A)
V=F(B)
IF(U*V)20,20,10
10  L=-1
RETURN
20  X=(A+B)/2.
W=F(X)
IF(U*W)30,30,403
30  B=X
V=W
GO TO 50
40  A=X
U=W
50  IF(ABS(W).LE.EPSI) GO TO 60
IF(ABS(B-A).GT.ETA) GO TO 20
L=1
RETURN
60  L=0
RETURN
END

FUNCTION S(D)
DATA PI,R,R2/3.14159267,10.,100./
D2=D*D
TETA=ATAN2(SQRT(4.*R2-D2),D)
S=TETA*D2+(PI-2.*TETA)*R2-R*D*SIN(TETA)
S=S-PI*R2/2.
RETURN
END

```

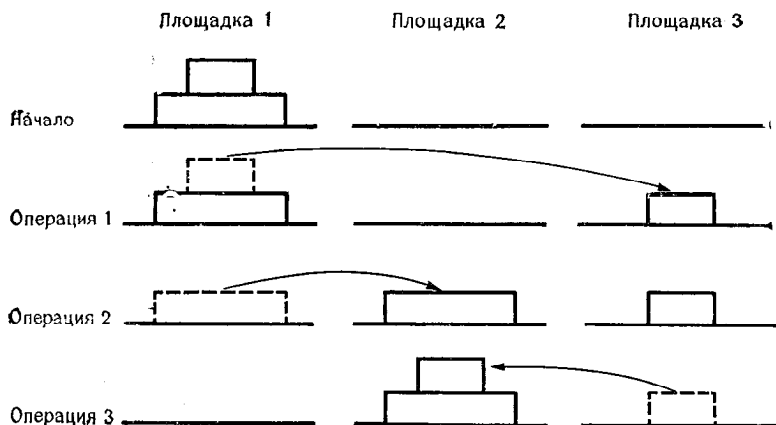
8.4. Ханойская башня¹⁾

Пирамидка (башня) состоит из n дисков, положенных один на другой так, что чем выше находится диск, тем меньше его диаметр.

¹⁾ Своим происхождением эта задача обязана индусской легенде, которая рассказывает, что в большом храме Бенареса бронзовая плита поддерживает три алмазных стержня, на один из которых бог нанизал во время сотворения мира 64 золотых диска. С тех пор день и ночь монахи, сменяя друг друга, каждую секунду перекладывают по одному диску согласно правилам, описанным в этом параграфе. Конец мира наступит тогда, когда все 64 диска будут перемещены, на что потребуется чуть больше 58 млрд. лет.

Требуется переместить эту башню, соблюдая следующие правила:

- за один раз можно перекладывать только один диск,
- нельзя класть диск на меньший по размерам,
- можно пользоваться только одной резервной площадкой.



Пример $n = 2$

Схема показывает, как можно переместить башню с площадки 1 на площадку 2

Рис. 8.8.

Задача

Как переместить башню с площадки 1 на площадку 2 при любом значении n ?

1. Найти метод решения с применением рекурсии, а затем метод, в котором рекурсия не используется.

2. Нарисовать блок-схему.

3. Написать программу.

Напомним, что на Фортране нельзя (за исключением особых случаев) писать «рекурсивные» подпрограммы.

Решение

Эта задача, весьма ординарная в математическом плане, интересна с точки зрения программирования, так как она является простой пример задач, которые очень легко решаются с использованием рекурсии, но представляют значительные трудности, когда рекурсия не допускается, что как раз и свойственно языку Фортран.

1-й метод

Легко найти решение при $n = 2$. Назовем $HANOI(A, B, n)$ процедуру переноса башни из n дисков с площадки A на площадку B .

Чтобы осуществить операцию HANOI (1, 2, 3), можно действовать таким образом:

- выполнить HANOI(1, 3, 2),
- перенести самый большой диск с площадки 1 на площадку 2,
- выполнить HANOI(3, 2, 2).

Это можно изобразить схематически следующим образом:

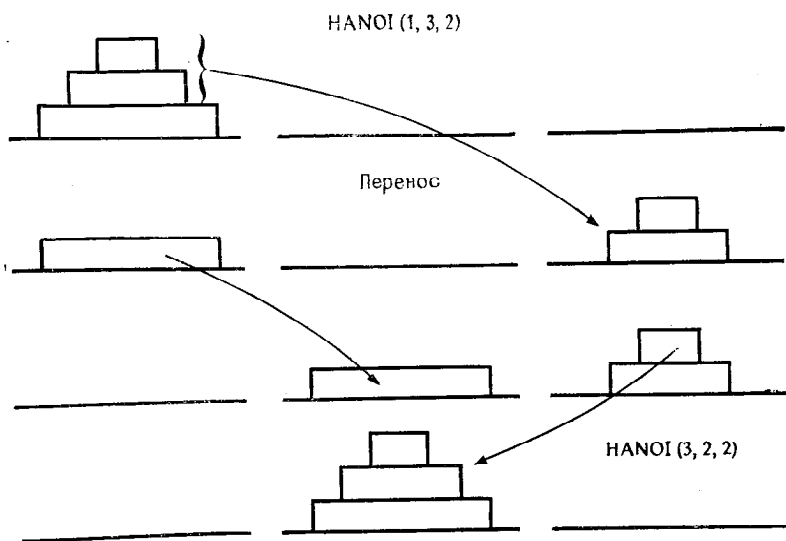


Рис. 8.9.

Операцию можно обобщить для произвольного n .

$A, B = 1, 2 \text{ или } 3 \quad C = 6 - A - B \Rightarrow C = 1, 2 \text{ или } 3$

$$\text{HANOI}(A, B, n) \begin{cases} \text{HANOI}(A, C, n-1), \\ \text{перенос } A \text{ в } B, \\ \text{HANOI}(C, B, n-1). \end{cases}$$

Далее мы увидим, что этот способ позволяет получить изящную рекурсивную формулировку.

Таким образом, чтобы осуществить операцию HANOI (1, 2, n), надо произвести нечетное число элементарных операций, а главной операцией является перенос самого большого диска с площадки 1 на площадку 2. Можно ли определить общее число предстоящих операций?

При $n = 2$ мы знаем, что нужно 3 операции;

при $n = 3$ требуется $3 + 1 + 3 = 7$ операций;

итак, $3^2 = 2^3 - 1$ и $7 = 2^3 - 1$. Следовательно, можно предположить, что для переноса башни с n дисками необходимо $2^n - 1$ операций. Это можно легко доказать, используя индукцию: указанное свойство выполняется при $n = 2$ (и 3), предположим, что оно справедливо при $n = p - 1$, и покажем, что оно выполняется при $n = p$:

$$\text{HANOI}(1, 2, p) \begin{cases} \text{HANOI}(1, 3, p-1) \Rightarrow 2^{p-1} - 1 \text{ операций,} \\ \text{перенос } 1, 2 \Rightarrow 1 \text{ операция,} \\ \text{HANOI}(3, 2, p-1) \Rightarrow 2^{p-1} - 1 \text{ операций.} \end{cases}$$

То есть требуется $2^{p-1} + 2^{p-1} - 1 = 2^p - 1$ операций, что доказывает свойство.

Таким образом, необходим массив из $2^n - 1$ строк и нескольких столбцов, в который записываются операции по мере того, как они выявляются. Когда массив будет заполнен, работа закончится. Мы уже знаем операцию, которая должна попасть в середину массива (позиция $2^n/2 = 2^{n-1}$).

Первые $2^{n-1} - 1$ строк будут соответствовать $\text{HANOI}(1, 3, n-1)$. Не разлагая пока $\text{HANOI}(1, 3, n-1)$, мы можем записать в середину этой зоны $\text{HANOI}(1, 3, n-1)$ и поступить аналогично по отношению ко второй части массива, который теперь примет вид такой, как на рис. 8.10.

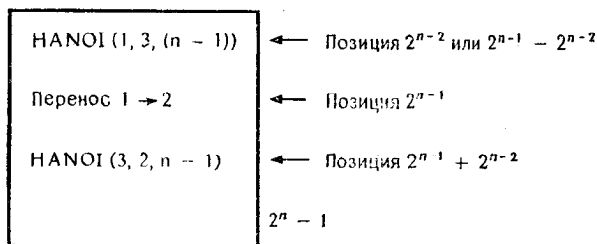


Рис. 8.10

Чтобы этот массив удобно обрабатывался программой на Фортране, разделим его на 3 столбца. Столбцы 2 и 3 будут соответственно указывать исходную площадку и целевую площадку. Первый столбец играет особую роль. Будем интерпретировать значения элементов $T(I)$ и столбца 1 следующим образом:

если $T(I) = 0$, то строка I не заменена,

если $T(I) = 1$, то задан перенос диска с площадки, указанной $D(I)$, на площадку, указанную $A(I)$,

если элемент $T(I) > 1$, то задана операция $HANOI(D(I), A(I), T(I))$, которая должна быть разложена на элементарные операции.

Предыдущий массив будет представлен в памяти так, как на рис. 8.11.

I	T(I)	D(I)	A(I)
1	⋮		
2^{n-2}	$n-1$	1	3
2^{n-1}	1	1	2
$2^{n-2} + 2^{n-2}$	$n-1$	3	2
$2^n - 1$	⋮		

Рис. 8.11.

Начиная с этого момента работа заключается в осуществлении всех разложений до тех пор, пока все $T(I)$ не станут равными 1.

На практике массив T содержит значения N . Следовательно, чтобы осуществить разложение операции $HANOI$ порядка N на две операции $HANOI$ порядка $N-1$ и один простой перенос, достаточно указать в подпрограмме $HANOI$ значение индекса I . То есть I является единственным параметром этой подпрограммы, поскольку массивы T , D , A в общей (COMMON) области.

Пример выполнения программы (см. блок-схемы на рис. 8.12 и 8.13, а также соответствующие им листинги):

1	2	5	16	1	2
			17	3	1
1	1	2	18	3	2
2	1	3	19	1	2
3	2	3	20	3	1
4	1	2	21	2	3
5	3	1	22	2	1
6	3	2	23	3	1
7	1	2	24	3	2
8	1	3	25	1	2
9	2	3	26	1	3
10	2	1	27	2	3
11	3	1	28	1	2
12	2	3	29	3	1
13	1	2	30	3	2
14	1	3	31	1	2
15	2	3	STOP	NUMBER	00

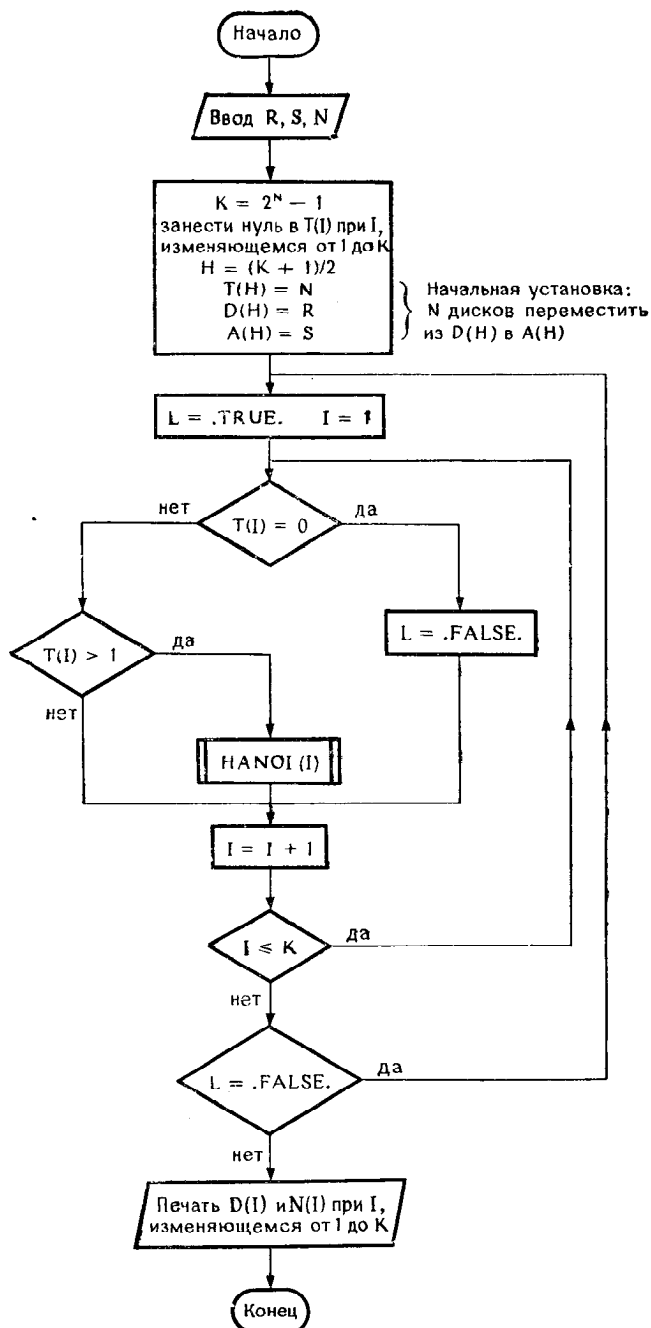


Рис. 8.12.

Подпрограмма HANOI(I):

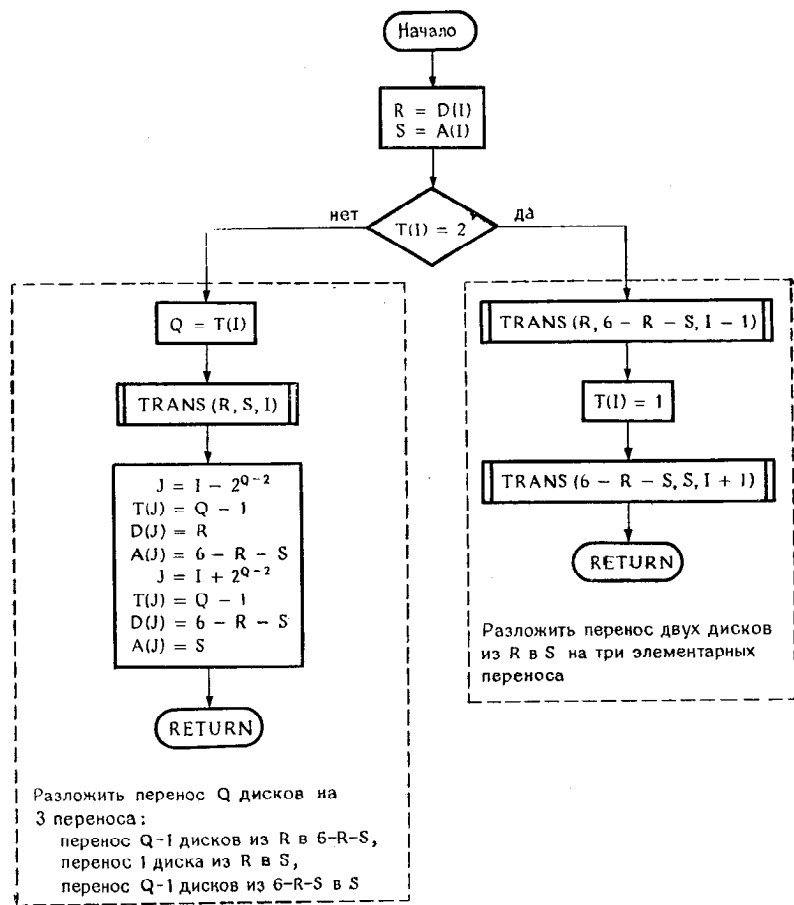


Рис. 8.13.


```

INTEGER T(255), D(255), A(255), R, S, H, H1
LOGICAL L
COMMON T, D, A
READ(2, 40) R, S, N
K=2*N-1
DO 10 I=1, K
10  T(I)=0
    H=(K+1)/2
    T(H)=N
    D(H)=R
    A(H)=S
15  L=.TRUE.
    DO 20 I=1, K
        IF(T(I).EQ.0) L=.FALSE.
        IF(T(I).GT.1) CALL HANOI(I)
20  CONTINUE
    IF(.NOT.L) GO TO 15
    WRITE(1, 50) (I, D(I), A(I), I=1, K)
40  FORMAT(2I2, I3)
50  FORMAT(15, 2I3)
    STOP
END

```

```

SUBROUTINE HANOI(I)
INTEGER T(255), D(255), A(255), R, S, Q
COMMON T, D, A
R=D(I)
S=A(I)
IF(T(I).NE.2) GO TO 10
CALL TRANS(R, 6-R-S, I-1)
T(I)=1
CALL TRANS(6-R-S, S, I+1)
RETURN
10  Q=T(I)
    CALL TRANS(R, S, I)
    J=I-2*(Q-2)
    T(J)=Q-1
    D(J)=R
    A(J)=6-R-S
    J=I+2*(Q-2)
    T(J)=Q-1
    D(J)=6-R-S
    A(J)=S
    RETURN
END

```

Разложение перевода
при $n=2$ на 3 перевода

HANOI (R, 6 - R - S, Q - 1).

HANOI (6 - R - S, S, Q - 1).

```

SUBROUTINE TRANS(R, S, I)
INTEGER T(255), D(255), A(255), R, S
COMMON T, D, A
T(I)=1
D(I)=R
A(I)=S
RETURN
END

```

8.5. Задача о восьми ферзях на шахматной доске

Эта классическая задача формулируется следующим образом: разместить 8 ферзей на шахматной доске так, чтобы ни один ферзь при этом не был «под боем».

При использовании компьютера условие слегка усложняется: необходимо найти все решения этой задачи.

В качестве примера на рис. 8.14 показано одно из решений.

Задача

Нарисовать блок-схему и написать программу, позволяющую получить все решения, такие, в которых ферзь на 1-й вертикали находится с 1 по 4 клетки; другие решения получаются с помощью симметрии.

Замечание. Имея в виду, что как правильные могут рассматриваться только те решения, в которых на каждой вертикали находится всего один ферзь, мы можем использовать массив D из 8 элементов, а относительные положения ферзя в вертикальном ряду I задавать значениями $D(I)$.

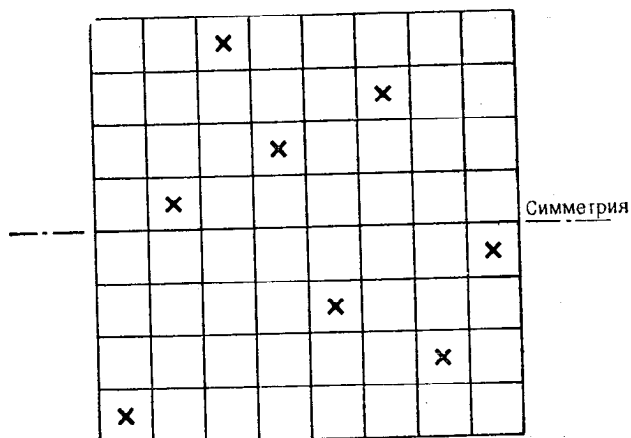


Рис. 8.14.

Рис. 8.14 соответствует решению $D(1) = 1$, $D(2) = 5$ и т. д., $D(8) = 4$.

Решение

Два ферзя не «бьют» друг друга, если:

а) $D(I) \neq D(J)$

ферзи не находятся на одной горизонтальной линии,

- б) $D(J) - D(I) \neq J - I$ ферзи не должны быть на одной диагонали под углом 45° ,
 в) $D(J) - D(I) \neq I - J$ ферзи не должны быть на одной диагонали под углом -45° .

Эти условия можно записать компактнее:

$$\begin{cases} D(I) \neq D(J), \\ |D(J) - D(I)| \neq J - I, \text{ где } J > I. \end{cases}$$

Чтобы получить решения, мы можем действовать следующим образом: при I , изменяющемся от 1 до 8:

1. Установить $D(I) = 1$.

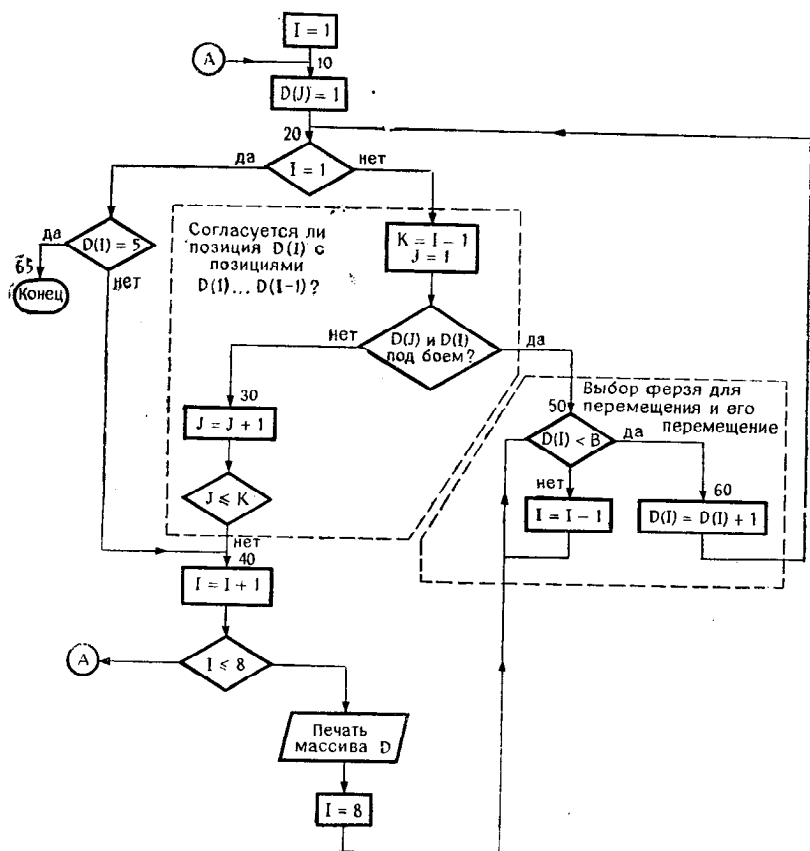


Рис. 8.15.

```

INTEGER D(8)
I=1
10 D(I)=1
20 IF(I.EQ.1)IF(D(I)-5)40,65,65
   K=I-1
   DO 30 J=1,K
     IF(D(I).EQ.D(J))GO TO 50
     IF(ABS(D(I)-D(J)).EQ.(I-J))GO TO 50
30 CONTINUE
40 I=I+1
   IF(I.LF.8)GO TO 10
   WRITE(1,70)D
   I=8
50 IF(D(I).LT.8)GO TO 60
   I=I-1
   GO TO 50
60 D(I)=D(I)+1
   GO TO 20
65 STOP
70 FORMAT(8I3)
END

```

Результат выполнения программы:

1	5	8	6	3	7	2	4
1	6	8	3	7	4	2	5
1	7	4	6	8	2	5	3
1	7	5	8	2	4	6	3
2	4	6	8	3	1	7	5
2	5	7	1	3	8	6	4
2	5	7	4	1	8	6	3
2	6	1	7	4	8	3	5
2	6	8	3	1	4	7	5
2	7	3	6	8	5	1	4
2	7	5	8	1	4	6	3
2	8	6	1	3	5	7	4
3	1	7	5	8	2	4	6
3	5	2	8	1	7	4	6
3	5	2	8	6	4	7	1
3	5	7	1	4	2	8	6
3	5	8	4	1	7	2	6
3	6	2	5	8	1	7	4
3	6	2	7	1	4	8	5
3	6	2	7	5	1	8	4
3	6	4	1	8	5	7	2
3	6	4	2	8	5	7	1
3	6	8	1	4	7	5	2
3	6	8	1	5	7	2	4
3	6	8	2	4	1	7	5
3	7	2	8	5	1	4	6
3	7	2	8	6	4	1	5
3	8	4	7	1	6	2	5
4	1	5	8	2	7	3	6
4	1	5	8	6	3	7	2
4	2	5	8	6	1	3	7
4	2	7	3	6	8	1	5
4	2	7	3	6	8	5	1
4	2	7	5	1	8	6	3
4	2	8	5	7	1	3	6
4	2	8	6	1	3	5	7
4	6	1	5	2	8	3	7
4	6	8	2	7	1	3	5
4	6	8	3	1	7	5	2
4	7	1	8	5	2	6	3
4	7	3	8	2	5	1	6
4	7	5	2	6	1	3	8
4	7	5	3	1	6	8	2
4	8	1	3	6	2	7	5
4	8	1	5	7	2	6	3
4	8	5	3	1	7	2	6

2. Проверить, что ферзь $D(I)$ не бьет ферзя с $D(1)$ по $D(I-1)$;

если он бьет хотя бы одного ферзя, то перейти к 3, если он не бьет ни одного ферзя, то:

- если $I = 8$, решение получено, надо его напечатать, после чего продолжить работу,
- если $I < 8$, увеличить I на 1 и перейти к 1.

3. Продвигать ферзя $D(J)$, стремясь

- по возможности увеличивать J , соблюдая условие $J \leq I$,
- с учетом условия $D(J) \leq 8$ установить $I = J$, вернуться к 2.

Эта теоретическая схема нуждается в детализации, прежде чем ее можно будет запрограммировать. Например, в 2 перед проверками необходимо убедиться, что $I \geq 2$. Кроме того, если $I = 1$, то надо посмотреть, выполняется ли равенство $D(I) = 5$, и в таком случае остановиться.

Все это учтено в блок-схеме, изображенной на рис. 8.15.

Программирование по этой блок-схеме не составляет никакого труда. Важно, однако, не попасть в «ловушку», связанную с использованием ДО-цикла с управляющей переменной I , поскольку возникает опасность войти в пределы ДО-цикла, не сделав предварительно начальной установки.

Две проверки $I = 1$ и $D(I) = J$ удобно объединить в одной строке, составленной из логической инструкции IF, за которой следует арифметическая инструкция IF.

8.6. Задача о восьми ферзях; исключение избыточных решений

Расположение восьми ферзей на шахматной доске можно в зависимости от порядка чтения представить в нескольких списках координат, которые в конечном итоге эквивалентны.

Пример

1 5 8 6 3 7 2 4	}	Эти записи соответствуют одному решению, представ- ленному на рис. 8.14.
1 7 5 8 2 4 6 3		
3 6 4 2 8 5 7 1		
8 2 4 1 7 5 3 6		
6 3 5 7 1 4 2 8		
8 4 1 3 6 2 7 5		
5 7 2 6 3 1 4 8		

Предыдущая программа исключает четыре последних решения, но оставляет еще два избыточных решения.

Задача

Изменить предыдущую программу так, чтобы исключались все избыточные решения. Для этого можно:

а) проверить, что $D(8) > D(1)$, и, если условие не выполняется, исключить решение, которое соответствует симметрии по отношению к оси АВ (рис. 8.16);

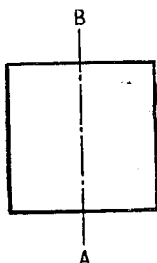


Рис. 8.16.

б) проверить, что последовательности, полученные при повороте на 90° влево или вправо, предпочтительнее, чем последовательности, полученные в обычном случае.

Замечание. Таким образом, предполагается, что известен переход от массива $D(8)$ к массивам $DG(8)$ или $DD(8)$, которые получаются из первого при повороте на 90° соответственно влево или вправо.

Решение

Чтобы определить массивы DG или DD с помощью вращения, достаточно применить следующие формулы:

$$\begin{cases} I \\ D(I) \end{cases} \Rightarrow \begin{cases} R = D(I), \\ DD(R) = 9 - I \end{cases}$$

(см. рис. 8.17),

$$\begin{cases} I \\ D(I) \end{cases} \Rightarrow \begin{cases} L = 9 - D(I), \\ DG(L) = I, \end{cases}$$

(см. рис. 8.18).

Это вычисление делается для каждой точки $(I, D(I))$.

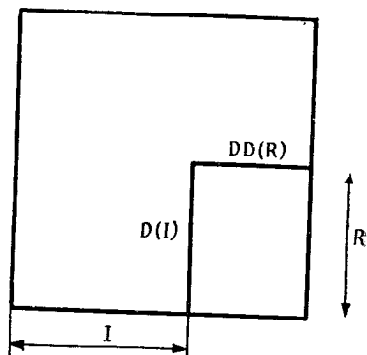


Рис. 8.17.

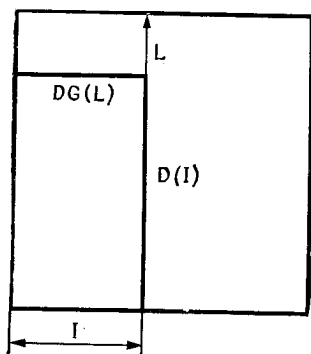


Рис. 8.18.

Чтобы оценить, соответствует ли $DG(I)$ или $DD(I)$ лучшей последовательности, чем $D(I)$, можно действовать следующим образом.

Вычислить

$$X1 = DD(8) + DD(7) \cdot 8 + DD(6) \cdot 8^2 + \dots + DD(1) \cdot 8^7,$$

$$X2 = DD(1) + DD(2) \cdot 8 + DD(3) \cdot 8^2 + \dots + DD(8) \cdot 8^7,$$

$$X = \min(X1, X2).$$

Сравнить X с Z , вычислив Z по формуле

$$Z = D(8) + D(7) \cdot 8 + D(6) \cdot 8^2 + \dots + D(1) \cdot 8^7.$$

Если $X < Z$, то найденное решение следует отбросить.

Так же поступают с $DG(I)$.

Этот метод немного тяжеловесен, но он легко программируется, если применяется схема Горнера. В результате получается блок-схема, изображенная на рис. 8.19.

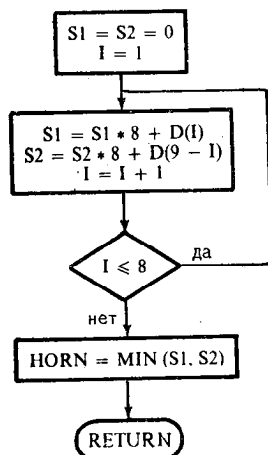


Рис. 8.19

```

INTEGER D(8)
I=1
10 D(I)=1
20 IF(I.EQ.1)IF(D(I)-5)40,65,65
   K=I-1
   DO 30 J=1,K
     IF(D(I).EQ.D(J))GO TO 50
     IF(ABS(D(I)-D(J)).EQ.(I-J))GO TO 50
30 CONTINUE
40 I=I+1
   IF(I.LE.8)GO TO 10
   IF(D(8).GT.D(1))WRITE(1,70)D
   I=8
50 IF(D(I).LT.8)GO TO 60
   I=I-1
   GO TO 50
60 D(I)=D(I)+1
   GO TO 20
65 STOP
70 FORMAT(8I3)
END

```

1	5	8	6	3	7	2	4
1	6	8	3	7	4	2	5
1	7	4	6	8	2	5	3
1	7	5	8	2	4	6	3
2	4	6	8	3	1	7	5
2	5	7	1	3	8	6	4
2	5	7	4	1	8	6	3
2	6	1	7	4	8	3	5
2	6	8	3	1	4	7	5
2	7	3	6	8	5	1	4
2	7	5	8	1	4	6	3
2	8	6	1	3	5	7	4
3	1	7	5	8	2	4	6
3	5	2	8	1	7	4	6
3	5	7	1	4	2	8	6
3	5	8	4	1	7	2	6
3	6	2	5	8	1	7	4
3	6	2	7	1	4	8	5
3	6	2	7	5	1	8	4
3	6	8	1	5	7	2	4
3	6	8	2	4	1	7	5
3	7	2	8	5	1	4	6
3	7	2	8	6	4	1	5
3	8	4	7	1	6	2	5
4	1	5	8	2	7	3	6
4	2	5	8	6	1	3	7
4	2	7	3	6	8	1	5
4	2	8	5	7	1	3	6
4	2	8	6	1	3	5	7
4	6	1	5	2	8	3	7
4	6	8	2	7	1	3	5
4	7	3	8	2	5	1	6
4	7	5	2	6	1	3	8
4	8	1	3	6	2	7	5
4	8	5	3	1	7	2	6


```

INTEGR D(8),DD(8),DG(8),R
I=1
10 D(I)=1
20 IF(I.EQ.1)IF(D(I)-5)40,65,65
K=I-1
D0 30 J=1,K
    IF(D(I).EQ.D(J))G0 T0 50
    IF(ABS(D(I)-D(J)).EQ.(I-J))G0 T0 50
30 CONTINUE
40 I=I+1
    IF(I.LE.8)G0 T0 10
    IF(D(8).LT.D(1))G0 T0 49
D0 42 I=1,8
    R=D(I)
    L=9-R
    DG(L)=I
42 DD(R)=9-I
X=HORN(DD,8)
Y=HORN(DG,8)
Z=HORN(D,8)
IF(X.LT.Z .OR. Y.LT.Z)G0 T0 49
WRITE(1,70)D
49 I=8
50 IF(D(I).LT.8)G0 T0 60
I=I-1
G0 T0 50
60 D(I)=D(I)+1
G0 T0 20
65 STOP
70 FORMAT(8I3)
END

```

Пример выполнения программы:

```

FUNCTION HORN(D,N)
INTEGER D(N)
N1=N+1
FN=FLOAT(N)
S1=0.
S2=0.
DO 10 I=1,N
  N1I=N1-I
  S1=S1*FN+FLOAT(D(I))
  S2=S2*FN+FLOAT(D(N1I))
HORN=AMIN1(S1,S2)
RETURN
END

```

1	5	8	6	3	7	2	4
1	6	8	3	7	4	2	5
2	4	6	8	3	1	7	5
2	5	7	1	3	8	6	4
2	5	7	4	1	8	6	3
2	6	1	7	4	8	3	5
2	6	8	3	1	4	7	5
2	7	3	6	8	5	1	4
2	7	5	8	1	4	6	3
3	5	2	8	1	7	4	6
3	5	8	4	1	7	2	6
3	6	2	5	8	1	7	4

ОГЛАВЛЕНИЕ

От редактора перевода	5
Предисловие	7
Символы, используемые в блок-схемах	9
Символы, используемые в блок-схемах для обозначения данных	10
Глава 1. Блок-схемы	11
1.0. Введение	11
1.1. Задача об автомате и лабиринте	11
1.2. Задача о поездах и мухе	14
1.3. Предварительный анализ задачи	14
1.4. Первая блок-схема	18
1.5. Подробный анализ отрезков пути	19
1.6. Окончательная блок-схема	20
1.7. Методология решения научных задач с помощью вычислительной машины	21
1.8. Советы по составлению блок-схемы	23
1.9. Советы по конструированию программы	25
Глава 2. Упражнения с целыми числами	29
2.1. Таблица Пифагора	29
2.2. НОК двух целых чисел	33
2.3. Совершенные числа	35
2.4. Поиск простых чисел	38
2.5. Разложение числа на простые множители	42
Глава 3. Простые упражнения, ориентированные на численный анализ	48
3.1. Константа Эйлера, первая задача	48
3.2. Константа Эйлера, вторая задача	52
3.3. Последовательность Фибоначчи и золотое сечение	54
3.4. След матрицы	57
3.5. Определитель треугольной матрицы	59
3.6. Транспонирование квадратной матрицы	61
3.7. Решение уравнения $f(x) = x$ методом итерации	64
3.8. Решение уравнения методом хорд и методом деления пополам	67
Глава 4. Упражнения, ориентированные на численный анализ	75
4.1. Вычисление длины кривой	75
4.2. Вычисление интеграла методом трапеций	78
4.3. Вычисление определенного интеграла методом Симпсона	82
4.4. Вычисление определенного интеграла методом Ветдлля	84
4.5. Решение системы линейных уравнений методом Гаусса	85

4.6. Интегрирование дифференциального уравнения методом Рунге — Кутты	90
Глава 5. Упражнения по статистике	96
5.1. Среднее, дисперсия, среднее квадратичное отклонение	96
5.2. Простая линейная регрессия	99
5.3. Анализ генератора случайных чисел	103
Глава 6. Финансовые расчеты	108
6.1. Платежи по займу	108
6.2. Расчет процента	112
Глава 7. Различные простые упражнения	115
7.1. Сортировка в оперативной памяти	115
7.2. Редактирование таблицы преобразования	122
7.3. Слияние двух массивов	124
7.4. Поиск в массиве	127
Глава 8. Различные более сложные упражнения	130
8.1. Определение дня недели: список пятниц, приходящихся на 13-е число	130
8.2. Автоматический расчет сдачи	134
8.3. Задача о козе	140
8.4. Ханойская башня	145
8.5. Задача о восьми ферзях на шахматной доске	153
8.6. Задача о восьми ферзях; исключение избыточных решений	156

Издательство «Мир» готовит к выпуску

Математическое моделирование. Пер. с англ., 16 л.,
1 р. 50 к.

Книга занимает особое положение в современной научной литературе. В ней сделана попытка дать студентам, специализирующимся по прикладным наукам, представление о том, как «делается» современная наука. Отдельные главы посвящены конкретным математическим моделям: таким, как управление движением, истечение жидкости, сверление отверстий лазером, анализ напряжений, развитие популяций, планирование и т. д. Каждая глава заканчивается списком задач для самостоятельной работы.

Широта охвата материала и разнообразие примеров делают книгу полезной для начинающих исследователей по прикладной математике и для научных работников различных специальностей.

Если Вы желаете приобрести эту книгу, оставьте в книжном магазине предварительный заказ. Своевременное оформление заказа гарантирует Вам приобретение книги.

Издательство «Мир» готовит к выпуску

ЛЬЮИС Ф., РОЗЕНКРАНЦ Д., СТИРНЗ Р. Теоретические основы проектирования компиляторов. Пер. с англ., 33 л., 2 р. 70 к.

В книге известных американских специалистов излагаются математические понятия и методы теории автоматов и формальных грамматик, лежащие в основе проектирования компиляторов, и показывается, как их применять на практике. Применение теории детально продемонстрировано на примере компиляторов для учебного языка программирования. Разработанный авторами метод позволил им включить в синтаксический блок значительную часть того, что обычно относится к семантике (генерации кода). Изложение строгое, но неформальное, доступное читателю, не имеющему специальной математической подготовки.

Книга рекомендуется широкому кругу системных программистов и студентов соответствующего профиля (особенно инженерных вузов).

Если Вы желаете приобрести эту книгу, оставьте в книжном магазине предварительный заказ. Своевременное оформление заказа гарантирует Вам приобретение книги.

Издательство «Мир» готовит к выпуску

ПРАТТ Т. Языки программирования. Разработка и реализация. Пер. с англ., 34 л., 2 р. 80 к.

Книга посвящена систематическому изложению языков программирования. В первой ее части вводится система понятий и критериев, позволяющих исследовать самые различные языки с единой точки зрения. Главное внимание уделяется семантике языков, т. е. структурам данных, операциям, структурам управления и организации памяти. Во второй части рассматриваются семь наиболее распространенных языков программирования: Фортран, Алгол-60, Кобол, ПЛ/I, Лисп, Снобол-4, АПЛ.

Книга рассчитана на широкий круг программистов. Она будет полезным учебным пособием при изучении языков программирования в вузах, поможет профессиональным программистам более рационально выбирать языки для конкретных приложений.

Если Вы желаете приобрести эту книгу, оставьте в книжном магазине предварительный заказ. Своевременное оформление заказа гарантирует Вам приобретение книги.

Издательство «Мир» готовит к выпуску

ГИЛМАН Л., РОУЗ А. Курс АПЛ; диалоговый подход.
Пер. с англ. 27 л., 2 р. 30 к.

Книга посвящена описанию АПЛ/360 — универсального языка программирования высокого уровня. В ней излагаются основные понятия языка, дается их обобщение, обсуждаются различные «особые» случаи. Она знакомит с реализацией АПЛ на машинах ИБМ серий 360 и 370, организацией файловых систем и особенностями различных версий языка. Все понятия иллюстрируются примерами программ, часто встречающихся на практике. В конце каждой главы приводятся задачи. Книгу удобно использовать для изучения языка в диалоговом режиме — каждая глава соответствует одному сеансу работы с ЭВМ.

Книга будет полезной и интересной разработчикам математического обеспечения ЭВМ и программистам различной квалификации.

Если Вы желаете приобрести эту книгу, оставьте в книжном магазине предварительный заказ. Своевременное оформление заказа гарантирует Вам приобретение книги.

УВАЖАЕМЫЙ ЧИТАТЕЛЬ!

Ваши замечания о содержании книги, ее оформлении, качестве перевода и другие просим присылать по адресу: 129820, Москва, И-110, ГСП, 1-й Рижский пер., д. 2, издательство «Мир».

Ж.-П. ЛАМУАТЬЕ
УПРАЖНЕНИЯ ПО ПРОГРАММИРОВА-
НИЮ НА ФОРТРАНЕ IV

Редактор А. А. Бряндинская
Художник А. В. Шипов
Художественный редактор
В. И. Шаповалов
Технический редактор Н. И. Борисова
Корректор А. П. Сизова

ИБ № 1016

Сдано в набор 23.01.78. Подписано к печати 18.07.78. Литературная гарнитура. Высокая печать. Бумага тип. № 3 60×90¹/₁₆, бум. л. 5, 25, 10, 50 печ. л. Уч.-изд. л. 7, 61. Изд. № 1/9486. Цена 60 коп. Тираж 72 000 экз. Заказ № 566.

ИЗДАТЕЛЬСТВО «МИР»

Москва, 1-й Рижский пер., 2

Ордена Трудового Красного Знамени Ленинградская типография № 2 имени Евгении Соколовой «Союзполиграфпрома» при Государственном комитете Совета Министров СССР по делам издательств, полиграфии и книжной торговли. 198052, Ленинград, Л-52, Измайловский проспект, 29